Digital Logic

# MODULE 6
## FUNCTIONS OF COMBINATIONAL LOGIC

**Objectives:**

1. To introduce a functions of digital circuits.
2. To show the design of the functions.
3. To introduce how to use an MSI chip of the functions.



CamScanner

Digital Logic

# FUNCTIONS OF COMBINATIONAL LOGIC

- Basic Adders
- Parallel Binary Adder
- Comparators
- Decoders
- Encoders
- Multiplexers (Data Selector)
- DeMultiplexers (Data Distributor)
- Code Converter
- Parity Generators / Checkers

- Any digital circuit may consist one or more of these functions
- Because these functions are common, it is produced as a single IC
- The IC is a Medium Scale Integration (MSI) chip
    - The IC helps to make circuit development faster because there is no need to redesign all the functions from scratch
- It is important to know how to use the IC



CamScanner

# Basic Adder

Digital Logic

- Basic adder operation is fundamental to the study of digital system.
- Types of basic adder :
  - Half Adder
  - Full Adder
- Basic rules for binary addition

| |
|---|
| 0+0= 0 |
| 0+1= 1 |
| 1+0= 1 |
| 1+1=10 |

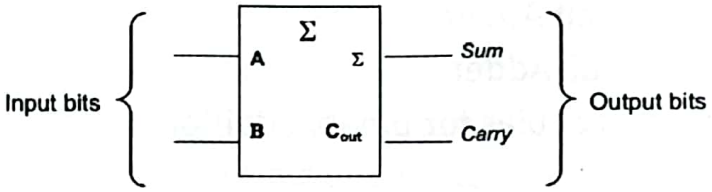
- Adder is a basic circuit in a digital arithmetic operation.
- As mentioned in Module 1, all arithmetic operations can be implemented by using an adder.



CamScanner

# Half Adder

- Accepts two binary digits on its inputs and produces two binary digits on its outputs, a sum bit and a carry bit
- It meant for adding 1-bit number




$$Sum = A \oplus B$$

- Sum output is 1, if A and B are not equal.

$$C_{out} = AB$$

- Output carry is 1 only when A and B are 1s.

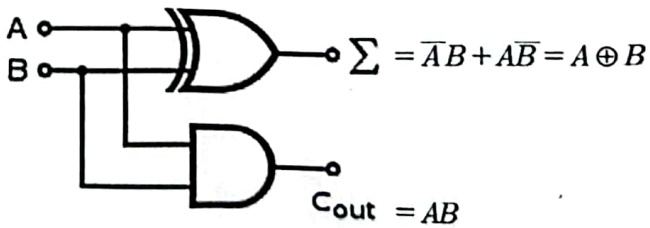
The Design:

| A | B | Carry, $C_{out}$ | Sum |
|---|---|---|---|
| 0 | 0 | 0 | 0 |
| 0 | 1 | 0 | 1 |
| 1 | 0 | 0 | 1 |
| 1 | 1 | 1 | 0 |

From the truth table:

Sum, $\sum = \overline{A}B + A\overline{B} = A \oplus B$

Carry, $C_{out} = AB$

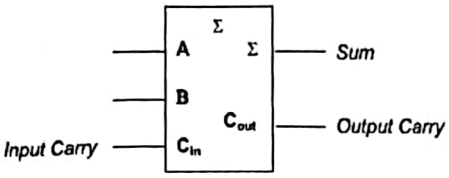




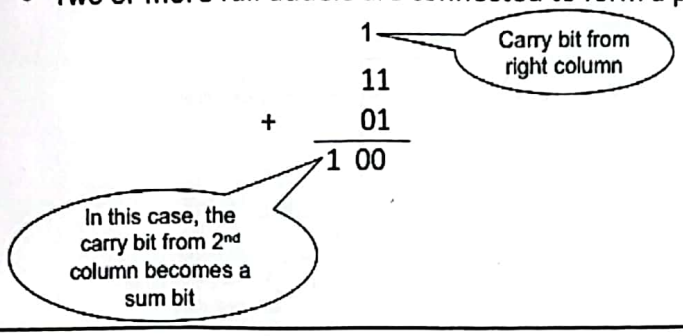

CamScanner

# Full Adder..

- Accepts two input bits and **an input carry** and generates a sum output and an output carry.
- Suitable for adding more than 1 bits by cascading the full adder




- Two or more full adders are connected to form a parallel adders.

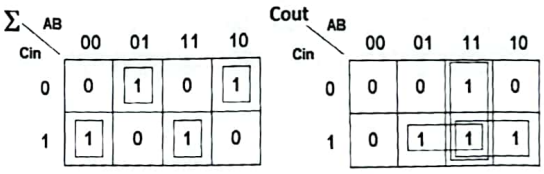



**The Design:**

**Step 1 :** Produce a Truth table

| Cin | A | B | Cout | Σ |
|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 1 | 0 | 1 |
| 0 | 1 | 0 | 0 | 1 |
| 0 | 1 | 1 | 1 | 0 |
| 1 | 0 | 0 | 0 | 1 |
| 1 | 0 | 1 | 1 | 0 |
| 1 | 1 | 0 | 1 | 0 |
| 1 | 1 | 1 | 1 | 1 |

**Step 2:** Map the K-Map

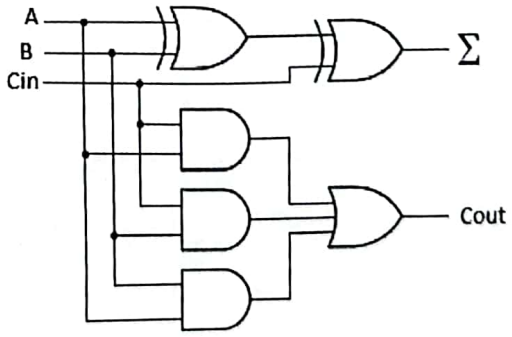



**Step 3:** Simplify K-Map & get the equation

$$\Sigma = (A \oplus B) \oplus Cin$$

$$Cout = AB + ACin + BCin$$

**Step 4 :** Draw the circuit



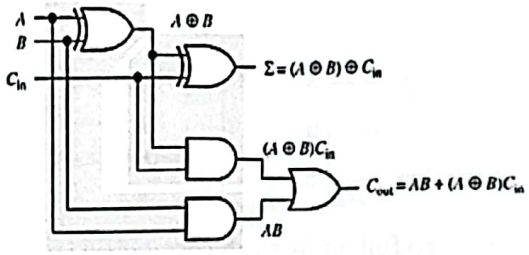


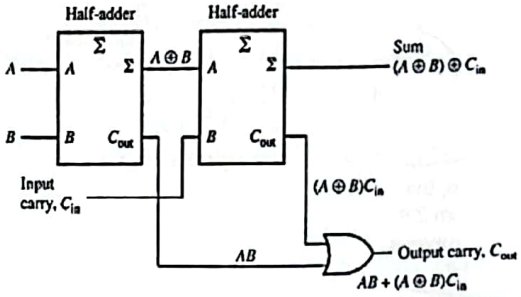

CamScanner

# ..Full Adder



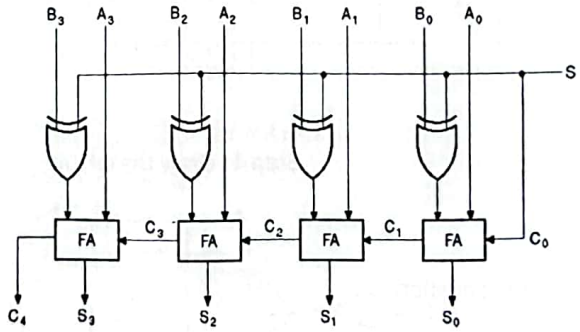
- Full Adder actually can be built by using 2 Half Adder
  - Half Adder is the one shaded



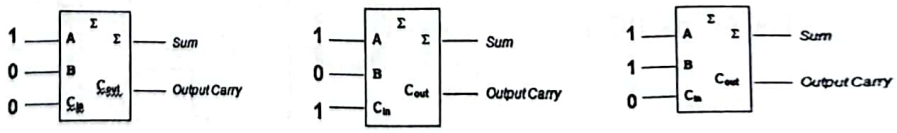




**Adder/Subtractor Circuit**

- The circuit implement the 2's complement arithmetic
- If S=0 implement **A + B**, no changes to B because B⊕S = B⊕0 = B, therefore full adder will do A+B
- If S =1 implement **A - B**, -B will be the 2's complement of B because B⊕1 = $\overline{B}$ which is the 1's complement of B. To get the 2's complement of B we have to add 1 by connecting $C_0$ to S so that $C_0$ = S = 1, therefore full adder will do ***A + 2's complement of B*** which is **A-B**




**DIY:** For each of the three full adders, determine the outputs for the inputs shown





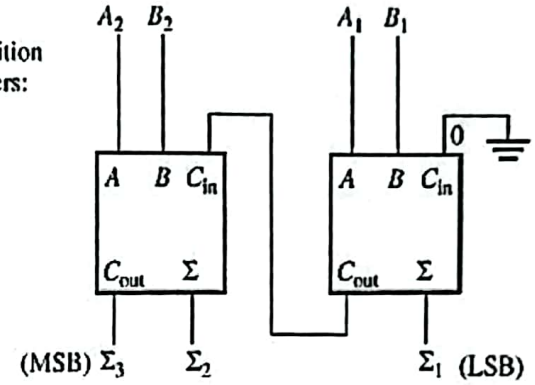

CamScanner

# Parallel Adder

- Two or more full adders are connected together to form a parallel adder
  - It is use to add more than 1 bit
  - Example : 2 bit parallel adder

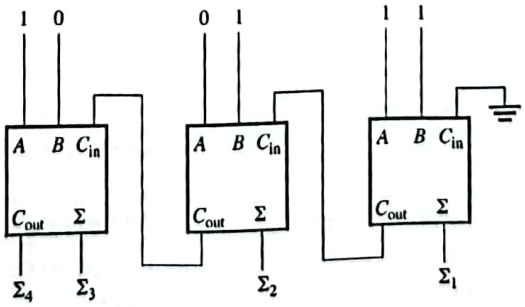
General format, addition of two 2-bit numbers:

$$\begin{array}{r} A_2A_1 \\ +\ B_2B_1 \\ \hline \Sigma_3\Sigma_2\Sigma_1 \end{array}$$




- Addition is done from right to left, therefore the leftmost bit doesn't have a $C_{in}$, signified by a ground symbol (which has a logic value of 0).
- To add the Least Significant Bit (LSB), either:
  - ✓ use a half adder which doesn't have a $C_{in}$, or
  - ✓ use a full adder but must make the $C_{in}$ equals to 0.
- All other bits position can only be added by using a full adder because there is a possibility of $C_{in}$ from the previous bit addition.
- From the example in the slide, the addition is for two bit number but the answer is 3 bits number.

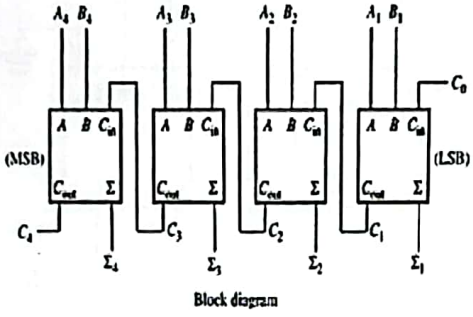
**DIY :** Determine the sum generated by the 3-bit parallel adder below and show the intermediate carries when the binary numbers 101 and 011 are being added



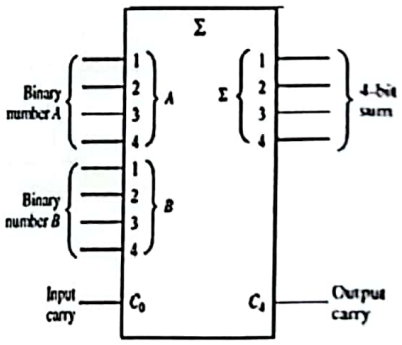

CamScanner

# MSI chip: 4-Bit Parallel Adder

- 74LS283 is a MSI chip that can do 4 bits addition
- Only the first $C_{in}$ and the last $C_{out}$ exist as a pin on the IC, all other carry pins are connected internally
- Internally, the IC is connected as in the block diagram



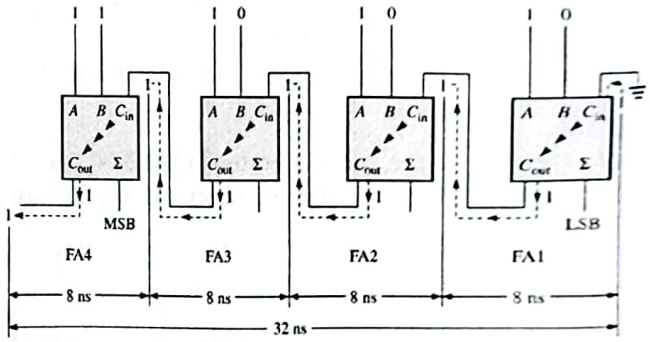

Block diagram




Logic symbol

Two type of parallel adder, based on how it handle a carry:

1. **Ripple Carry Adder**
   - Carry output of each full-adder is connected to carry in of the next order stage of full adder
   - Carry accumulate from the lowest order bit to the highest order bit
   - Speed of addition limited by the time required for the carries to propagate (or ripple) through all stages
   - Circuit simpler than look-ahead carry adder.




*A 4-bit parallel binary ripple carry adder showing "worst-case" carry propagation delays*

2. **Look-Ahead Carry Adder**
   - Speed up the addition process by eliminating ripple carry delay
   - It anticipates the output carry of each stage , and based on the inputs, produces the output carry by either **carry generation** or **carry propagation**



CamScanner

# Adder: Application example

- A voting system

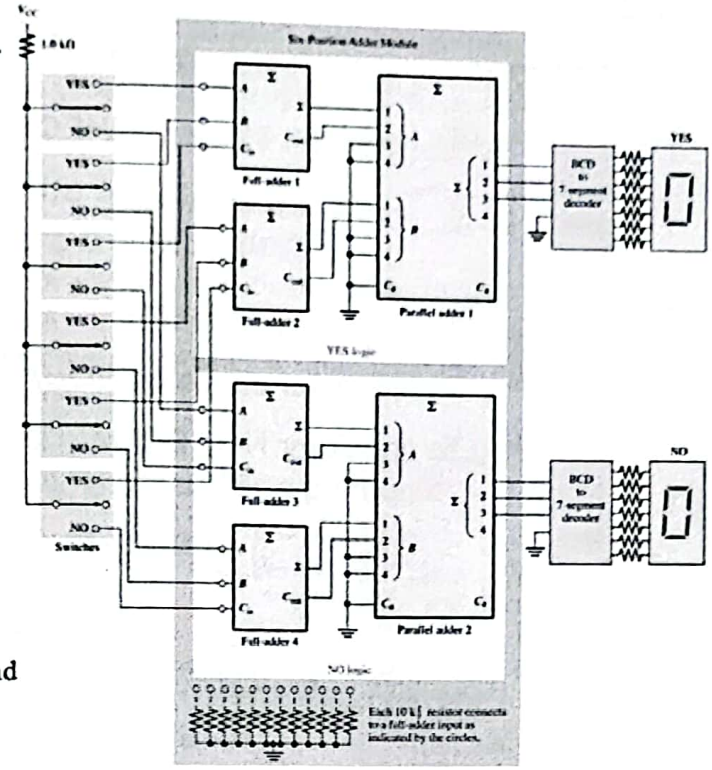



**Figure** A voting system using full-adders and parallel binary adders.

- The system will determine how many voter vote for YES and NO.
- Each voter press a switch which is an input to the system.
- Each input to the full adder is connected to ground through a 100KΩ resistor, this is to make the input LOW (0) if voter abstain ( don't want to choose YES or NO) -- this connection is not shown on the schematic:
  - If a user flips the switch to YES, the input to YES logic will become HIGH (1).
  - If a user flips the switch to NO, the input to NO logic will be HIGH (1).
- Each full adder can add 3 votes, the possible output is 00, 01, 10, 11.
- The output of the full adder is sent to parallel adder, where the output of the two full adder will be added.
- The output was sent to 7 segment display.
- To display the digit, a 7 segment decoder is used.

- **DIY**

a) If the sequence of voting from top to bottom is Y, N, N, Y, N, N, what will be displayed at the 7 segment display?

b) Trace the logic value at each input and output of the IC during that voting

c) What kind of 7 segment display is use in the circuit?

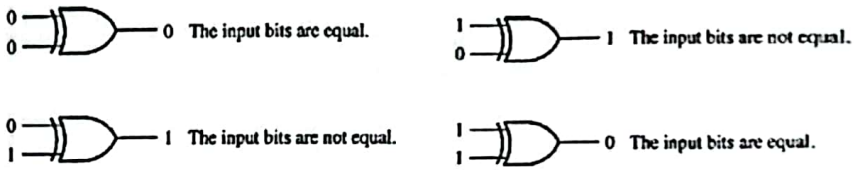
d) Why one of the input to 7-segment decoder is connected to ground ? Which pin ?



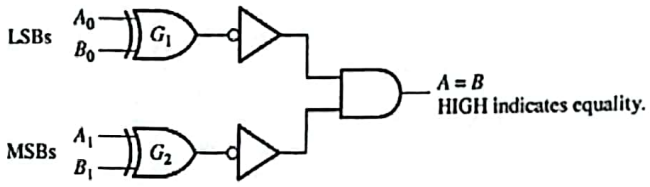
CamScanner

# Comparator

- Compares two *n*-bit binary values to determine whether the value is equal, lower or higher
  - To check for single bit **INEQUALITY** – use **XOR**




  - To check for single bit **EQUALITY** – use a **XNOR** gate
  - To check for N bit EQUALITY, combine the result of comparing an N single bit comparator




General format: Binary number $A \rightarrow A_1A_0$
Binary number $B \rightarrow B_1B_0$

**The Design:**

Design a single bit comparator that compares X and Y and determine whether $X = Y$, $X < Y$ or $X > Y$

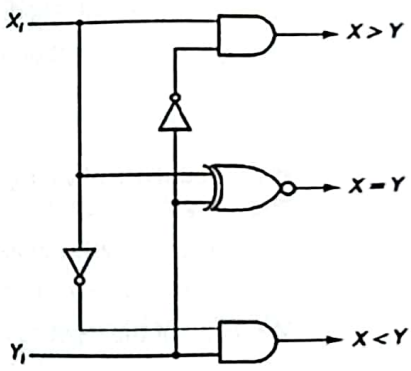
1) The truth table:

| $X_i$ | $Y_i$ | $X > Y$ | $X = Y$ | $X < Y$ |
|---|---|---|---|---|
| 0 | 0 | 0 | 1 | 0 |
| 0 | 1 | 0 | 0 | 1 |
| 1 | 0 | 1 | 0 | 0 |
| 1 | 1 | 0 | 1 | 0 |

2) From truth table, the output equations:

$$X > Y \rightarrow XY$$
$$X = Y \rightarrow \overline{X}\,\overline{Y} + XY = X \odot Y$$
$$X < Y \rightarrow \overline{X}Y$$

3) The circuit:
Design a separate circuit for each output and finally combine them.

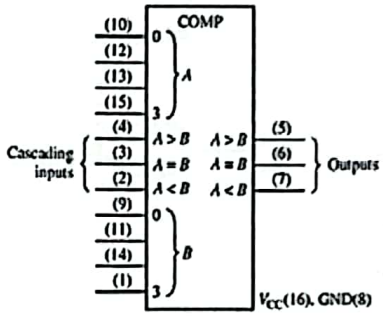





CamScanner

# MSI chip: 4-bit Comparator

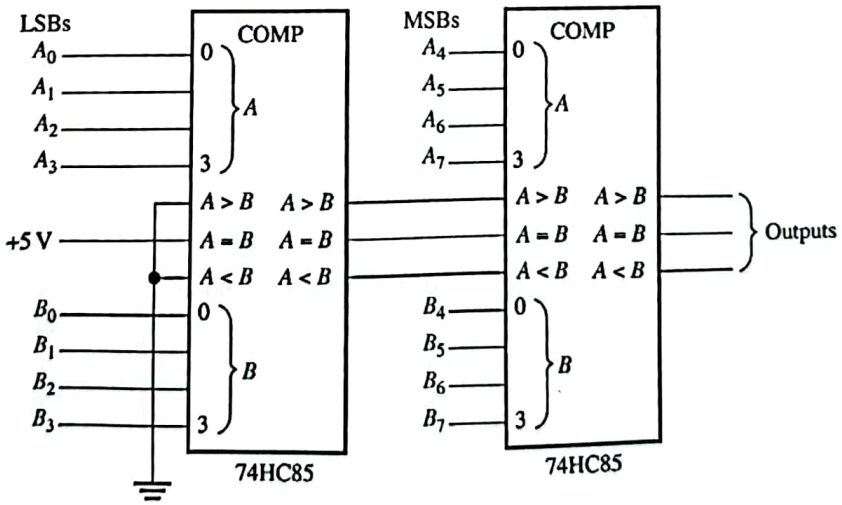
- 74LS85 is an MSI chip that can compare 4 bits value
- It can indicates whether the two number greater, equal or lower than the other
- Its operation: compare the MSB first
  - If $A_3$= 1 and $B_3$ = 0 then A > B set without comparing the lower bit
  - If $A_3$= 0 and $B_3$ = 1 then A < B set without comparing the lower bit
  - If $A_3$ = $B_3$ then A2 and B2 will be compared, until $A_0$ and $B_0$, if all bits are equal, only then A= B set
- 74LS85 also has a cascading input, to allow cascading an IC for more bits comparison




**Example :**

Comparing 8 bits value

- Each IC can only compare 4 bits, therefore requires 2 ICs in cascaded connection.
- For cascaded connection:
  - ✓ Output of A>B, A=B, A< B must be connected to the corresponding input of the next stage
  - ✓ For lowest order comparator, A=B must be HIGH while A>B and A<B must be connected to LOW

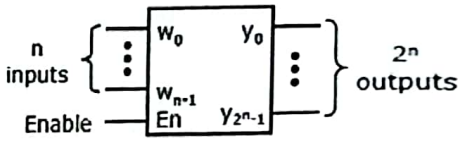





CamScanner

# Decoder

- Function
  - detect the presence of a specified combination of bits (code) on its inputs
  - indicate the presence of that code by a specified output level.
- Decoder has
  - *n* input lines to handle *n* bits
  - from 1 to $2^n$ output lines to indicate the presence of one or more n-bit combinations.
  - An enable input (En) is used to disable the outputs
    - If En=0, none of the decoder outputs is asserted
    - If En=1, one of the outputs is asserted according to the valuation of the inputs




There are several types of decoder. This note only discuss the following types:

- Binary Decoder
- MSI Chip: 4-Bits Decoder
- BCD to Decimal Decoder
- MSI chip: Binary to 7-Segments Decoder
- Combinational Logic using Decoder

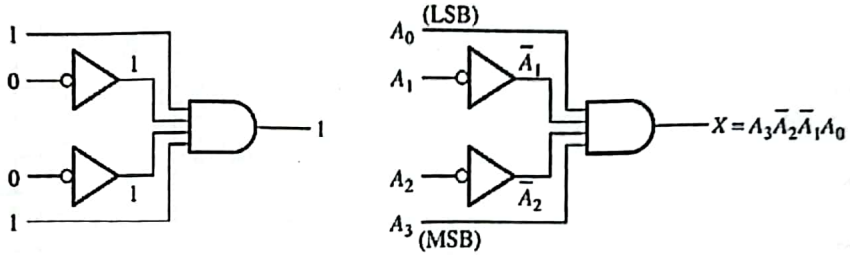


CamScanner

# Binary Decoder ..

Digital Logic

- A decoder that recognize a binary pattern at its input and activate the output whenever that pattern occurs
  - Use AND if we want the output to be HIGH
  - Use NAND if we want the output to be LOW
- Example : A decoder that produces a logic HIGH if $1001_2$ occurs at the input




**The Design:**

1. Determine what kind of output desired:
   - If HIGH use AND
   - If LOW use NAND
2. Determine the binary pattern at the input:
   - Draw a bubble at the input if input 0
   - Draw a straight horizontal line at the input if input 1

**DIY:**

a) Design a binary decoder that detect an input of $1011_2$ and produces a logic 0 if that logic value exist at the input.

b) An active low device is activated by an output of a decoder. The device will be activated if the decoder detect $110011010_2$, design the decoder circuit.

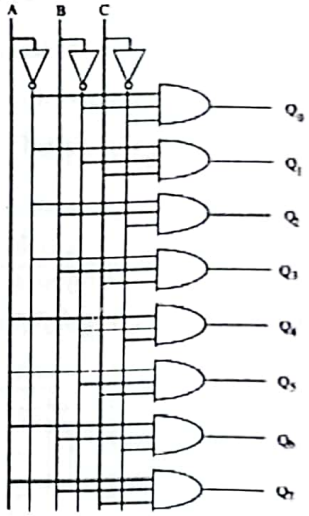
CamScanner

# ..Binary Decoder

- Sometimes it is good to have a general purpose decoder that can decode all the possibility of binary pattern at the input
  - Such a decoder has multiple output but its output will only be active if a unique binary pattern exist at the input
  - Eg: 3 bits Decoder

| A | B | C | $Q_0$ | $Q_1$ | $Q_2$ | $Q_3$ | $Q_4$ | $Q_5$ | $Q_6$ | $Q_7$ |
|---|---|---|---|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 1 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 1 | 0 | 1 | 0 | 0 | 0 | 0 | 0 | 0 |
| 0 | 1 | 0 | 0 | 0 | 1 | 0 | 0 | 0 | 0 | 0 |
| 0 | 1 | 1 | 0 | 0 | 0 | 1 | 0 | 0 | 0 | 0 |
| 1 | 0 | 0 | 0 | 0 | 0 | 0 | 1 | 0 | 0 | 0 |
| 1 | 0 | 1 | 0 | 0 | 0 | 0 | 0 | 1 | 0 | 0 |
| 1 | 1 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 1 | 0 |
| 1 | 1 | 1 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 1 |

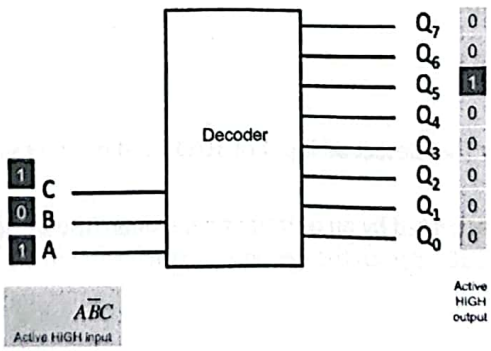
Truth Table for 3 bits decoder




- The design steps is the same as in the previous slide
- AND is used because we want an active high output
- The difference is that all possible input is listed in the truth table, and all the output is also listed
- The each input there will be only one HIGH output
- For an active LOW output, there will be only one LOW for each input
- The equation for each output is determine

$$Q_0 = \overline{A}\,\overline{B}\,\overline{C}, Q_1 = \overline{A}\,\overline{B}C, \ldots . Q_6 = AB\overline{C}, Q_7 = ABC$$

- From the equation the circuit was build. For the above circuit the input is active HIGH and the output is active HIGH
- **Example**: Binary decoder.




**Example:**

If the input = 101,

the output $Q_5$ = 1

**Note:**
Only one output will active with a unique binary pattern at input

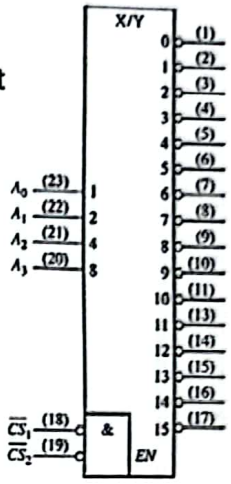


CamScanner

# MSI Chip: 4-Bits Decoder ..

Digital Logic

- Decoder can be called **n to 2$^n$** or **1 of 2$^n$** for the same decoder
- The common MSI decoder chip
  - 2 to 4 (1 of 4), 3 to 8 (1 of 8) and 4 to 16 (1 of 16)
- 74HC154 is a 1 of 16 decoder
  - It has 4 active high input and 16 active low output
  - It also has 2 active low enable (EN) pin
    - This pin is used to select the IC if we use more than one chip
    - To use the IC, this two pin must be connected to logic LOW




**The function table** – to descibe the function of the device

| | | INPUTS | | | | OUTPUTS | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| $\overline{E}_0$ | $\overline{E}_1$ | $A_0$ | $A_1$ | $A_2$ | $A_3$ | $\overline{Y}_0$ | $\overline{Y}_1$ | $\overline{Y}_2$ | $\overline{Y}_3$ | $\overline{Y}_4$ | $\overline{Y}_5$ | $\overline{Y}_6$ | $\overline{Y}_7$ | $\overline{Y}_8$ | $\overline{Y}_9$ | $\overline{Y}_{10}$ | $\overline{Y}_{11}$ | $\overline{Y}_{12}$ | $\overline{Y}_{13}$ | $\overline{Y}_{14}$ | $\overline{Y}_{15}$ |
| H | H | X | X | X | X | H | H | H | H | H | H | H | H | H | H | H | H | H | H | H | H |
| H | L | X | X | X | X | H | H | H | H | H | H | H | H | H | H | H | H | H | H | H | H |
| L | H | X | X | X | X | H | H | H | H | H | H | H | H | H | H | H | H | H | H | H | H |
| L | L | L | L | L | L | L | H | H | H | H | H | H | H | H | H | H | H | H | H | H | H |
| L | L | H | L | L | L | H | L | H | H | H | H | H | H | H | H | H | H | H | H | H | H |
| L | L | L | H | L | L | H | H | L | H | H | H | H | H | H | H | H | H | H | H | H | H |
| L | L | H | H | L | L | H | H | H | L | H | H | H | H | H | H | H | H | H | H | H | H |
| L | L | L | L | H | L | H | H | H | H | L | H | H | H | H | H | H | H | H | H | H | H |
| L | L | H | L | H | L | H | H | H | H | H | L | H | H | H | H | H | H | H | H | H | H |
| L | L | L | H | H | L | H | H | H | H | H | H | L | H | H | H | H | H | H | H | H | H |
| L | L | H | H | H | L | H | H | H | H | H | H | H | L | H | H | H | H | H | H | H | H |
| L | L | L | L | L | H | H | H | H | H | H | H | H | H | L | H | H | H | H | H | H | H |
| L | L | H | L | L | H | H | H | H | H | H | H | H | H | H | L | H | H | H | H | H | H |
| L | L | L | H | L | H | H | H | H | H | H | H | H | H | H | H | L | H | H | H | H | H |
| L | L | H | H | L | H | H | H | H | H | H | H | H | H | H | H | H | L | H | H | H | H |
| L | L | L | L | H | H | H | H | H | H | H | H | H | H | H | H | H | H | L | H | H | H |
| L | L | H | L | H | H | H | H | H | H | H | H | H | H | H | H | H | H | H | L | H | H |
| L | L | L | H | H | H | H | H | H | H | H | H | H | H | H | H | H | H | H | H | L | H |
| L | L | H | H | H | H | H | H | H | H | H | H | H | H | H | H | H | H | H | H | H | L |

**Note**

1. H = HIGH voltage level
   L = LOW voltage level
   X = don't care

CamScanner

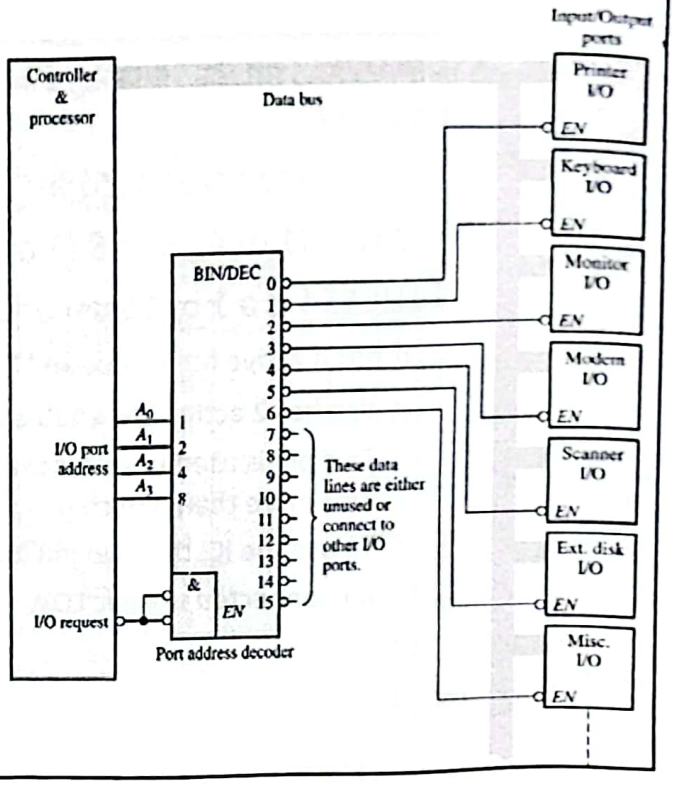
# MSI Chip: 4-Bits Decoder ..



**Example:**

- **Address Decoder**

  Decoder is use a lot to decode an address of a computer system.




- In a computer system every devices is given an address (like an ID) to identify the devices
- Address must be unique for a device, some devices uses a range of addresses
- Decoder is used to decode the address
- Whenever a computer try to access the I/O device it will issue an I/O request by making that signal LOW
  - this will enable an active low enable pin of a decoder
- Next the address of an I/O device will be sent by the computer as $A_3A_2A_1A_0$
- This address will be decoded and activate one of the output (becomes LOW)
  - If $A_3A_2A_1A_0 = 0110$, then output 6 will be LOW, all other output are HIGH
  - The LOW output will enable an active LOW device at address 6

**DIY**

a) Given $A_3A_2A_1A_0 = 0011$, which device will be enabled?

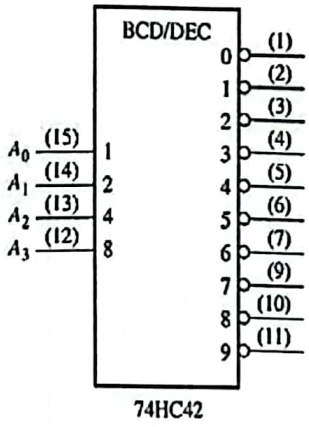
b) Why only one device enabled at any one time?

CamScanner

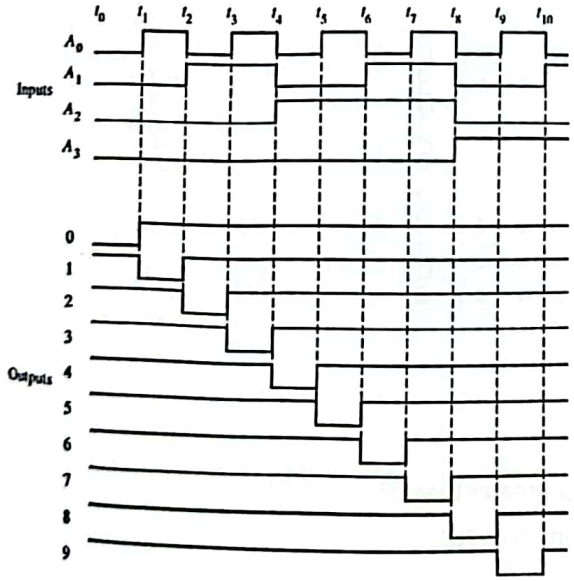
# BCD to Decimal Decoder

Digital Logic

- Convert from BCD to decimal
  - Frequently referred as 1 of 10 decoder
  - The same as binary decoder except only 10 decoding gates required
    - An active LOW output MSI Chip is 74HC42




The timing diagram of 74HC42, It shows that only one output active (LOW) for a BCD input




The decoding function , where each output has a unique function

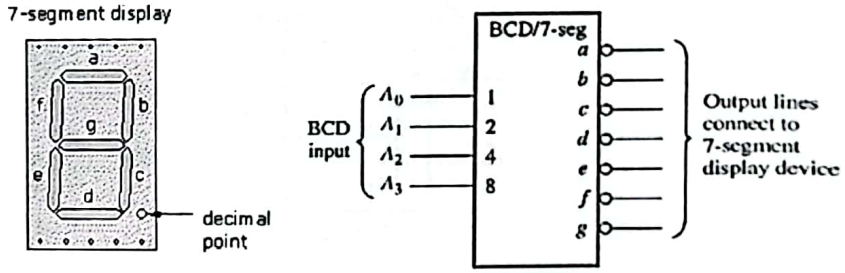
| Decimal digit | $A_3$ | $A_2$ | $A_1$ | $A_0$ | Decoding Function |
|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 | $\bar{A}_3\bar{A}_2\bar{A}_1\bar{A}_0$ |
| 1 | 0 | 0 | 0 | 1 | $\bar{A}_3\bar{A}_2\bar{A}_1A_0$ |
| 2 | 0 | 0 | 1 | 0 | $\bar{A}_3\bar{A}_2A_1\bar{A}_0$ |
| 3 | 0 | 0 | 1 | 1 | $\bar{A}_3\bar{A}_2A_1A_0$ |
| 4 | 0 | 1 | 0 | 0 | $A_3\bar{A}_2\bar{A}_1\bar{A}_0$ |
| 5 | 0 | 1 | 0 | 1 | $A_3\bar{A}_2\bar{A}_1A_0$ |
| 6 | 0 | 1 | 1 | 0 | $A_3\bar{A}_2A_1\bar{A}_0$ |
| 7 | 0 | 1 | 1 | 1 | $\bar{A}_3A_2A_1A_0$ |
| 8 | 1 | 0 | 0 | 0 | $A_3\bar{A}_2\bar{A}_1\bar{A}_0$ |
| 9 | 1 | 0 | 0 | 1 | $A_3\bar{A}_2\bar{A}_1A_0$ |

CamScanner

# MSI chip: Binary to 7-Segments Decoder

Digital Logic

- Receive a BCD code at the input and lit the proper LED of the 7-segment display based on the BCD value
- The display shows the decimal numbers 0-9
- The individual segments making up a 7-segment display are identified by letters, from *a* to *g*
  - 2 type of 7-segment display
    - Common anode – 0 at it input will lit the segment therefore require a decoder with an active LOW output
    - Common cathode- 1 at it input will lit the segment therefore require a decoder with an active HIGH output




- Truth table for an active HIGH output 7-segment decoder
  - Can drive a common cathode 7-segment display

| BCD Inputs | | | | Segment Outputs | | | | | | | Display |
|---|---|---|---|---|---|---|---|---|---|---|---|
| D | C | B | A | a | b | c | d | e | f | g | |
| 0 | 0 | 0 | 0 | 1 | 1 | 1 | 1 | 1 | 1 | 0 | 0 |
| 0 | 0 | 0 | 1 | 0 | 1 | 1 | 0 | 0 | 0 | 0 | 1 |
| 0 | 0 | 1 | 0 | 1 | 1 | 0 | 1 | 1 | 0 | 1 | 2 |
| 0 | 0 | 1 | 1 | 1 | 1 | 1 | 1 | 0 | 0 | 1 | 3 |
| 0 | 1 | 0 | 0 | 0 | 1 | 1 | 0 | 0 | 1 | 1 | 4 |
| 0 | 1 | 0 | 1 | 1 | 0 | 1 | 1 | 0 | 1 | 1 | 5 |
| 0 | 1 | 1 | 0 | 1 | 0 | 1 | 1 | 1 | 1 | 1 | 6 |
| 0 | 1 | 1 | 1 | 1 | 1 | 1 | 0 | 0 | 0 | 0 | 7 |
| 1 | 0 | 0 | 0 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 8 |
| 1 | 0 | 0 | 1 | 1 | 1 | 1 | 1 | 0 | 1 | 1 | 9 |

**DIY**

a) What will be displayed if the input to the decoder greater than 9?

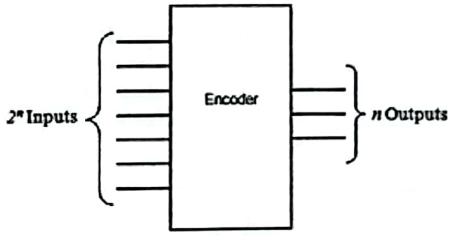
b) Produce a truth table for an active LOW 7-segment decoder



CamScanner

# Encoder

- Encoder does the opposite of the decoder
  - E.g: convert Decimal to BCD
- A ***binary encoder encodes information (data) from*** $2n$ *inputs into an n-bit code (output)*
  - Exactly one of the inputs should have a value of one
  - The outputs represent the binary number that identifies which input is equal to 1
- Encoders reduce the number of bits needed to represent given information



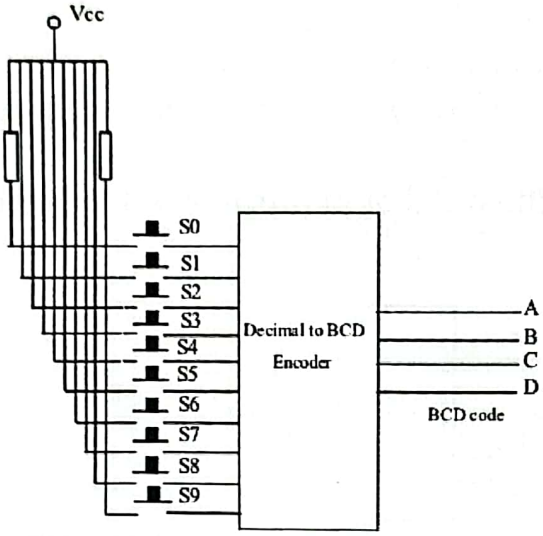

In this note we will discuss the following encoders

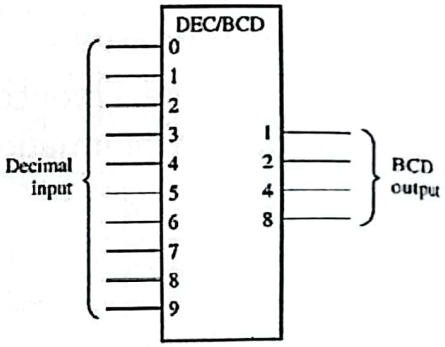
- Decimal to BCD Encoder
- Priority Encoder

CamScanner

# Decimal-to-BCD Encoder ..

- 10 input (Decimal) and 4 output (BCD)
  - Also called 10 line to 4 line encoder
  - Convert a decimal value to a BCD code




There are 10 input at the output but only one active at a time and each decimal value will be converted to a BCD as shown in the truth table

| S0 | S1 | S2 | S3 | S4 | S5 | S6 | S7 | S8 | S9 | $A_3$ | $A_2$ | $A_1$ | $A_0$ |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 1 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 0 | 1 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 1 |
| 0 | 0 | 1 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 1 | 0 |
| 0 | 0 | 0 | 1 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 1 | 1 |
| 0 | 0 | 0 | 0 | 1 | 0 | 0 | 0 | 0 | 0 | 0 | 1 | 0 | 0 |
| 0 | 0 | 0 | 0 | 0 | 1 | 0 | 0 | 0 | 0 | 0 | 1 | 0 | 1 |
| 0 | 0 | 0 | 0 | 0 | 0 | 1 | 0 | 0 | 0 | 0 | 1 | 1 | 0 |
| 0 | 0 | 0 | 0 | 0 | 0 | 0 | 1 | 0 | 0 | 0 | 1 | 1 | 1 |
| 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 1 | 0 | 1 | 0 | 0 | 0 |
| 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 1 | 1 | 0 | 0 | 1 |

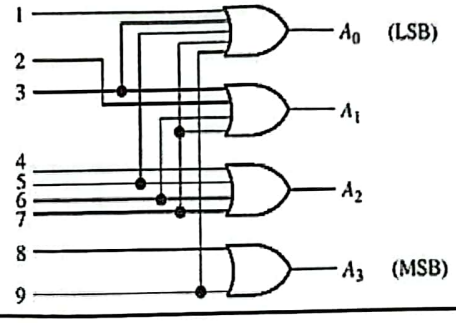


CamScanner

# .. Decimal-to-BCD Encoder

Digital Logic

- From truth table
  - $A_0$ HIGH if switch 1,3,5,7,9 HIGH
  - $A_1$ HIGH if switch 2,3,6,7 HIGH
  - $A_2$ HIGH if switch 4,5,6,7 HIGH
  - $A_3$ HIGH if switch 8,9 HIGH
    - Therefore OR all the input to make the corresponding output HIGH
- A 0-digit input is not needed because the BCD outputs are all LOW when there are no HIGH inputs




**DIY**

Suppose, HIGH are applied to input 2 and 9 of the circuit.

a) What are the states of the output lines?

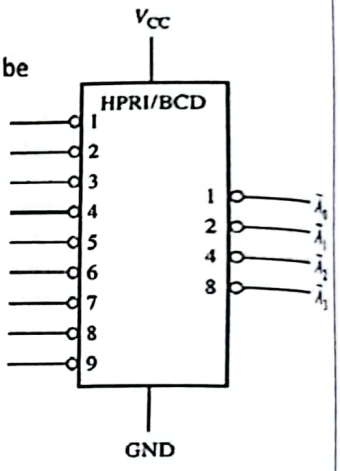
b) Does this represent a valid BCD code?

c) What is the restriction on the encoder logic?

CamScanner

# MSI chip: Priority Encoder

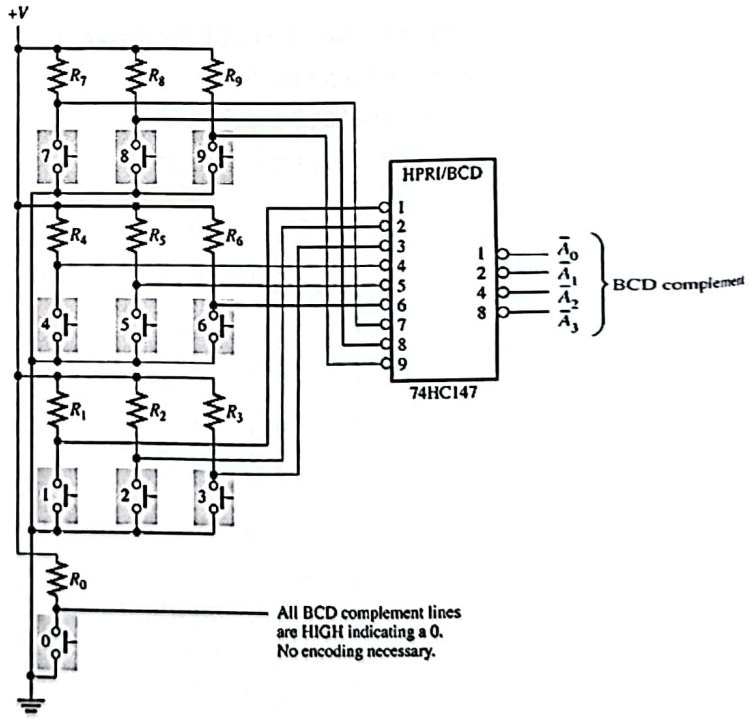
- IC 74HC147 perform the same encoding function but with additional feature
  - Produce BCD output corresponding to the higher decimal value at the input and ignore the lower value
    - HPRI – highest value input has priority
  - Eg:
    - if pin 9 and 5 LOW at the same time, only pin 9 will be recognize because it has a higher priority
    - The output will be 0110 for 9 (because of active low at the output)




**An Application:**

**Keyboard Encoder**

- 10 keys at the input , from 0 .. 9
- If no key is pressed , input to the encoder HIGH and its output all HIGH (1111) which is 0 as active LOW
- If a key is pressed, e.g. key 4 then pin 4 input will be LOW.
  - This will cause the output to be 1011 ( 4 as active LOW)





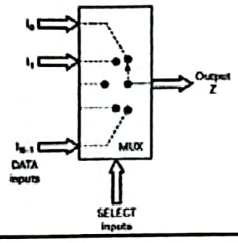

CamScanner

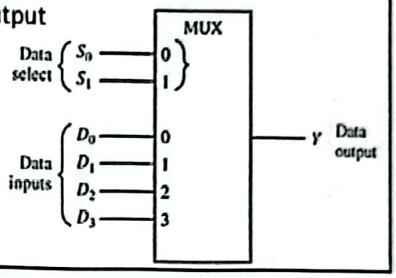
# Multiplexers (Data Selector)

- A multiplexer (MUX) is a device that allows digital information from several sources to be routed onto a single line for transmission over that line to a common destination.
  - The basic multiplexer has several data-input lines and a single output line, ***Many to One***
  - It also has data-select inputs, which permit digital data on any one of the inputs to be switched to the output line.
    - Select line control which input routed to output

$$\text{number of select bits} = \frac{\log(\text{number of input})}{\log 2}$$

- MUX are also known as data selector
  - Because it can select which data to sent to the output




**The Design:**

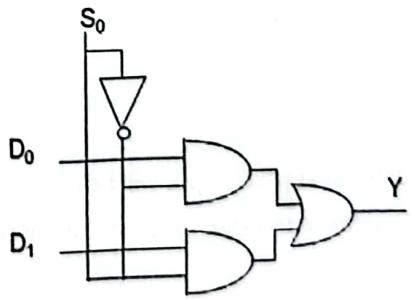
When $S_0 = 0$, $Y = D_0$ and when $S_0 = 1$, $Y = D_1$

- Truth table is produced from the specification.
- But truth table can be very long, a more compact form is on the right side table

| $S_0$ | $D_1$ | $D_0$ | Y |
|---|---|---|---|
| 0 | 0 | 0 | 0 |
| 0 | 0 | 1 | 1 |
| 0 | 1 | 0 | 0 |
| 0 | 1 | 1 | 1 |
| 1 | 0 | 0 | 0 |
| 1 | 0 | 1 | 0 |
| 1 | 1 | 0 | 1 |
| 1 | 1 | 1 | 1 |

| $S_0$ | Data Output |
|---|---|
| 0 | $Y=D_0$ |
| 1 | $Y=D_1$ |

The circuit equation: $Y = \overline{S_0} D_0 + S_1 D_1$



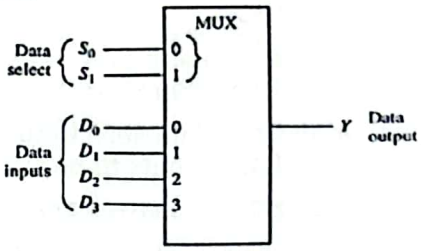


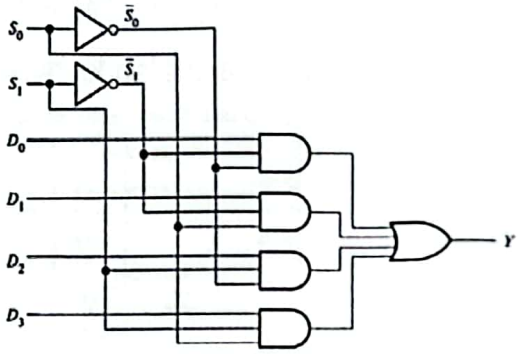

CamScanner

# Multiplexer 1-of-4

- Able to route 4 inputs to 1 output
  - Requires 2 bits data selector

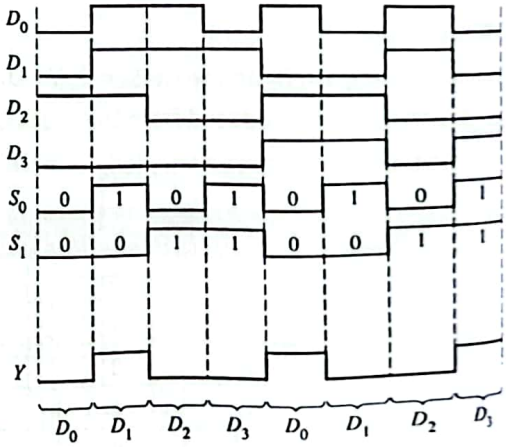
| Data-select input | | Data output Y |
|---|---|---|
| $S_1$ | $S_0$ | |
| 0 | 0 | $D_0$ |
| 0 | 1 | $D_1$ |
| 1 | 0 | $D_2$ |
| 1 | 1 | $D_3$ |



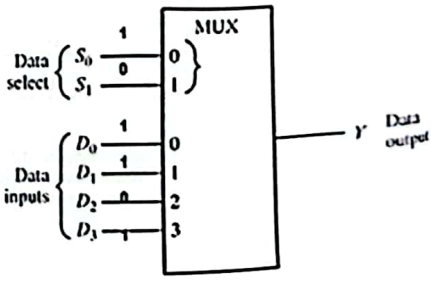

**Timing Diagram to show the working of 1 of 4 Multiplexer**

- The output will depend on the value of selector
- $S_1$ is the MSB and $S_0$ the LSB




**DIY:**

1. Determine which input will be routed to the output?
2. Determine the logic value at the output.

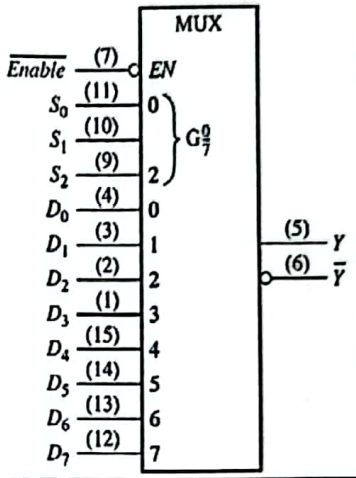





CamScanner

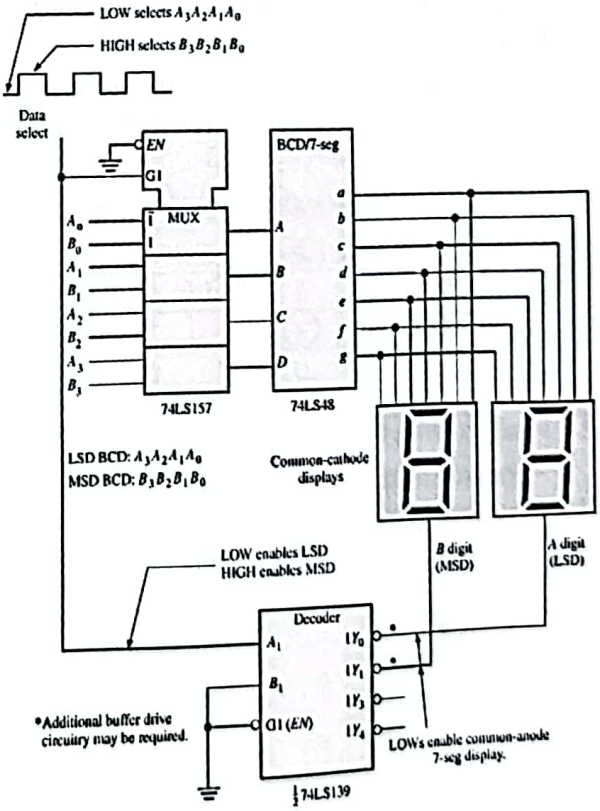
# MSI Chip: Multiplexer

- Besides Data input and selector pin , IC usually has an enable pin
  - For IC 74LS151
    - A 1 of 8 MUX
    - The enable pin is active LOW, therefore logic 0 must be applied to this pin to make it work
      - If logic HIGH applied to this pin the output will be High Impedance (Hi-Z), ie the output pin disconnected from the output
        -- the output is neither 0 nor 1
    - It has an inverted and a non inverted output




**Application:**

- One application is to alternately switch on a display by using a MUX
- To lit the LED 2 sets 4 bit data which is A3..A0 and B3 .. B0 need to be sent to 7 segment decoder
- Which set send to the decoder will be determine by data select pin , if LOW set A will be send and if HIGH set B will be send
  - ✓ By alternately turn on the display it will be seen as if those two display controlled simultaneously
- Seven segment is a common cathode therefore need a logic 0 to display a number
  - ✓ That logic 0 is controlled by a the output of the decoder, if $1Y_0$ =0 A digit will be displayed, else if $1Y_1$ =0 then B-digit will be displayed
  - ✓ Which output will be 0 determine by the input A1 which is connected to data select



# Example

Implement the function F (A,B,C,D) = Σm(1,3,4,11,12,13,14,15) using a multiplexer

- 4 inputs variables therefore use MUX with 3 selector pin (8x1 MUX)
- Connect A, B and C to selector pin where A is the MSB.

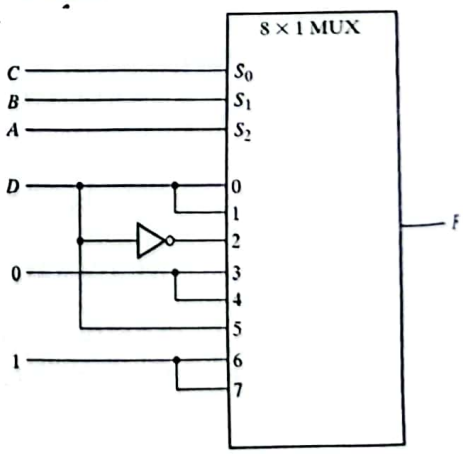
| A | B | C | D | F | |
|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 | F = D |
| 0 | 0 | 0 | 1 | 1 | |
| 0 | 0 | 1 | 0 | 0 | F = D |
| 0 | 0 | 1 | 1 | 1 | |
| 0 | 1 | 0 | 0 | 1 | F = D' |
| 0 | 1 | 0 | 1 | 0 | |
| 0 | 1 | 1 | 0 | 0 | F = 0 |
| 0 | 1 | 1 | 1 | 0 | |
| 1 | 0 | 0 | 0 | 0 | F = 0 |
| 1 | 0 | 0 | 1 | 0 | |
| 1 | 0 | 1 | 0 | 0 | F = D |
| 1 | 0 | 1 | 1 | 1 | |
| 1 | 1 | 0 | 0 | 1 | F = 1 |
| 1 | 1 | 0 | 1 | 1 | |
| 1 | 1 | 1 | 0 | 1 | F = 1 |
| 1 | 1 | 1 | 1 | 1 | |

- Produce a truth table and determine the output logic for F
- Draw a line to make a group based on the value of ABC =0,1,2 .. 7
- For each group relate the value of F and D
  - If equal then F=D
  - if complement then F=D'
  - if F always 0 then F =0
  - if F always 1 then F=1
- Connect the value of F from each group to pin of the MUX that has the corresponding value of ABC
  - E.g. ABC=101, F=D then connect D to pin 5 (which is 101)

- By using the other method produce the following table. Take note the value of each table read as ABCD
  - The value is different from the previous example because here we connect the LSB to the input pin of MUX while in the previous example MSB is connected to the input of the MUX

| | A'B'C' | A'B'C | A'BC' | A'BC | AB'C' | AB'C | ABC' | ABC |
|---|---|---|---|---|---|---|---|---|
| | $I_0$ | $I_1$ | $I_2$ | $I_3$ | $I_4$ | $I_5$ | $I_6$ | $I_7$ |
| D' | 0 | 2 | (4) | 6 | 8 | 10 | (12) | (14) |
| D | (1) | (3) | 5 | 7 | 9 | (11) | (13) | (15) |
| | D | D | D' | 0 | 0 | D | 1 | - |

- Finally connect the circuit
  → Doesn't matter which method is use the final answer will be the same



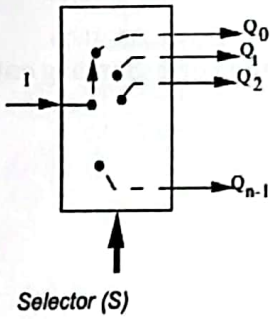



CamScanner

# Demultiplexer (DEMUX)

Digital Logic

- A DEMUX is basically reverse the multiplexing function.
- It takes digital information from one line and distributes it to a given number of output lines, **one to many.**
- For this reason, DEMUX is also known as a data distributor.
- Decoder can also be used as DEMUX.




| $S_1$ $S_0$ | $Q_3$ $Q_2$ $Q_1$ $Q_0$ |
|---|---|
| 0 0 | 0 0 0 1 |
| 0 1 | 0 0 1 0 |
| 1 0 | 0 1 0 0 |
| 1 1 | 1 0 0 0 |

**The Design**

- Truth Table

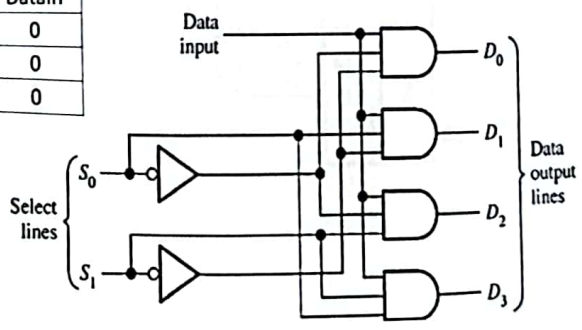
| $S_1$ | $S_0$ | DataIn | $D_3$ | $D_2$ | $D_1$ | $D_0$ |
|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 1 | 0 | 0 | 0 | 1 |
| 0 | 1 | 0 | 0 | 0 | 0 | 0 |
| 0 | 1 | 1 | 0 | 0 | 1 | 0 |
| 1 | 0 | 0 | 0 | 0 | 0 | 0 |
| 1 | 0 | 1 | 0 | 1 | 0 | 0 |
| 1 | 1 | 0 | 0 | 0 | 0 | 0 |
| 1 | 1 | 1 | 1 | 0 | 0 | 0 |

$$D_3 = S_1 S_0 \text{ DataIn}$$
$$D_2 = S_1 \overline{S_0} \text{ DataIn}$$
$$D_1 = \overline{S_1} S_0 \text{ DataIn}$$
$$D_0 = \overline{S_1}\,\overline{S_0} \text{ DataIn}$$

- A more compact Table:

| $S_1$ | $S_0$ | $D_3$ | $D_2$ | $D_1$ | $D_0$ |
|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 | DataIn |
| 0 | 1 | 0 | 0 | DataIn | 0 |
| 1 | 0 | 0 | DataIn | 0 | 0 |
| 1 | 1 | DataIn | 0 | 0 | 0 |

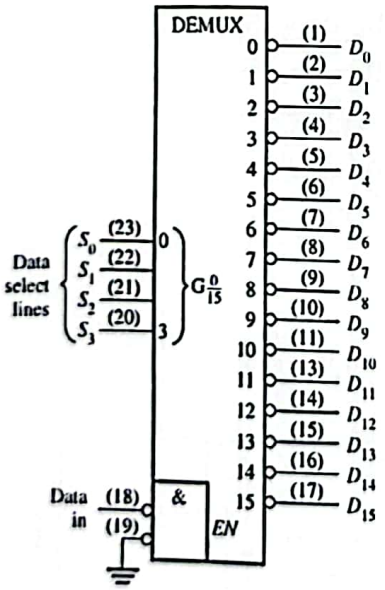
- The circuit:



# MSI Chip: Decoder as DeMUX

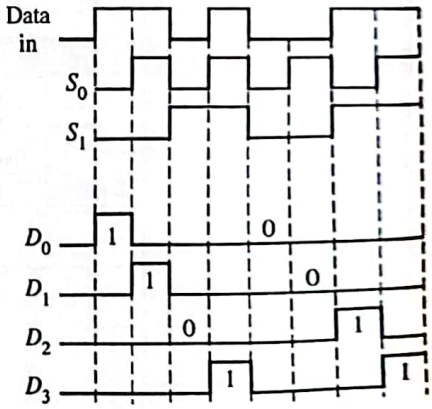
- A Decoder can be use as a DeMUX
  - Decoder 74HC154 can become a DeMUX when
    - Input of a decoder becomes a data select line
    - DataIn is connected to one of the enable pin of the IC
      - The other enable pin permanently connected to ground because it is an active LOW
      - If DataIn = 1, chip disable so that all output = 1 (because active low)
      - If DataIn = 0, chip will be enable,the corresponding output of the data selector will be 0

The circuit connection of a decoder from demux




| $S_1$ $S_0$ | $Q_3$ $Q_2$ $Q_1$ $Q_0$ |
|---|---|
| 0 0 | 0 0 0 1 |
| 0 1 | 0 0 1 0 |
| 1 0 | 0 1 0 0 |
| 1 1 | 1 0 0 0 |

The timing diagram to show the working of a 1-to-4 demux or as a 2-to-4 decoder






CamScanner

# Code Converter

- The purpose of ***code converter circuits is to*** convert from one type of code to another type of code
  - For example:
    - A 3-to-8 decoder converts from a binary number to a one-hot encoding at the output
    - A 8-to-3 encoder performs the opposite
- Many different types of code converter circuits can be constructed
  - One common example a a BCD-to7-segment decoder as explained in a decoder section

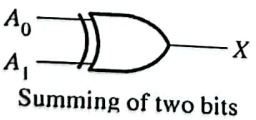
CamScanner

# Parity Generators / Checkers

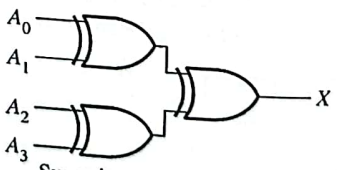


- Errors can occur as digital codes are being transferred from one point to another within a digital system.
  - The errors take the form of undesired changes in the bits that make up the coded information; that is, a 1 can change to a 0, or a 0 to a 1.
  - When an error occurs undetected, it can cause serious problem
- Basic Parity Logic
  - The sum (disregarding carries) of an even number of 1s is always 0, and the sum of an odd number of 1s is always 1.
    - Therefore, to determine if a given code has even parity or odd parity, all bits in the code are summed.
  - A parity bit indicates if the number of 1s in a code is even or odd for the purpose of error detection.

- To determine whether no of 1's or 0's odd or even
  - Use Modulo 2 summation
  - Device that can do Modulo 2 is XOR
  - If number of input bits
    - EVEN ( $A_1A_0 = 00$ or $A_1A_0 = 11$) then X =0
    - ODD ($A_1A_0 = 01$ or $A_1A_0 = 10$) then X = 1



Summing of two bits



Summing of four bits



CamScanner

# MODULE 7
## LATCHES AND FLIP-FLOPS



CamScanner

# Module 7: Latches & Flip-flops

Digital Logic

- Introduction to Latch & Flip-Flop
- Latch
  - S-R Latch
  - Gated S-R Latch
  - Gated D Latch
- Flip-flops
  - S-R Flip-Flop
  - D Flip-Flop
  - J-K Flip-Flop
  - T Flip-Flop

Objectives of this module are:

1 To introduce the various types of Latches and Flip-Flop.
2 To explain the characteristic of Latches and Flip-Flop.
3 To illustrate the timing diagram associated with each Latches and Flip-Flop operations.

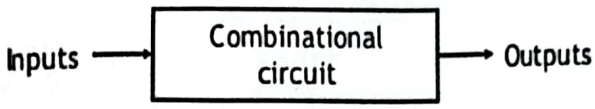


CamScanner

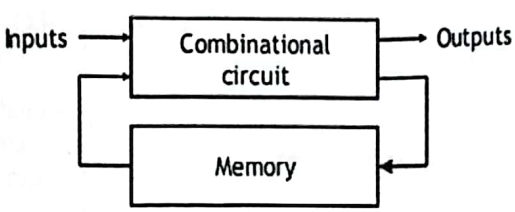
# Sequential Vs Combinational Logic Circuit



Combinational circuit



Sequential circuit



Combinational Logic Circuit:

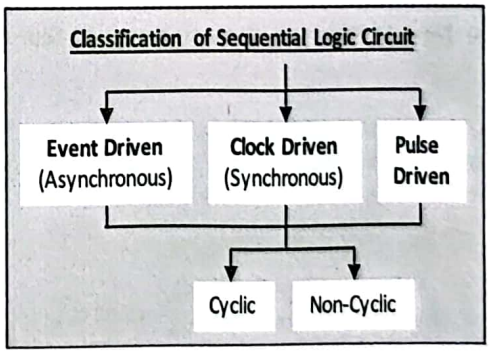
- the output depend on the input
- it has no memory element, therefore it cannot memorize the previous output

Sequential Logic Circuit:

- it has memory element
- the output depend on the input, as well as the previous output from the feedback elements.

Sequential Logic circuits can be divided into 3 main categories:

1. **Clock Driven** - Synchronous Circuits that are synchronized to a specific clock signal.
2. **Event Driven** - Asynchronous Circuits that react or change state when an external event occurs.
3. **Pulse Driven** - Which is a Combination of Synchronous and Asynchronous.




Sequential logic circuits that return back to their original state once reset, i.e. circuits with loops or feedback paths are said to be "Cyclic" in nature.



CamScanner

# Introduction: Latch



- **Latch** is a type of temporary storage device
- It has two stable states (bi-stable)
- It is level sensitive, or level-triggered.
  - outputs are dependent on the voltage level applied (HIGH or LOW), not on any signal transition.

- Type of Latch :
  - i) S-R
  - ii) Gated S-R
  - iii) Gated D
  - **S means *Set*, R means *Reset***

> ***INFO :***
> A ***bi-stable*** state is one with *two-stable* output states.

- A circuit incorporating latches has state; its output may depend not only on its current input, but also on its previous inputs. Such a circuit is described as sequential logic.
- Latch and flip-flop are a type of temporary storage device whereby they can store one bit of information.
- Latches and flip-flops are bi-stable (that has two stable states i.e. either LOW or HIGH state).
- The main difference between latches and flip-flops is the method used for changing their state.



CamScanner

# Introduction: Flip-flop (FF)

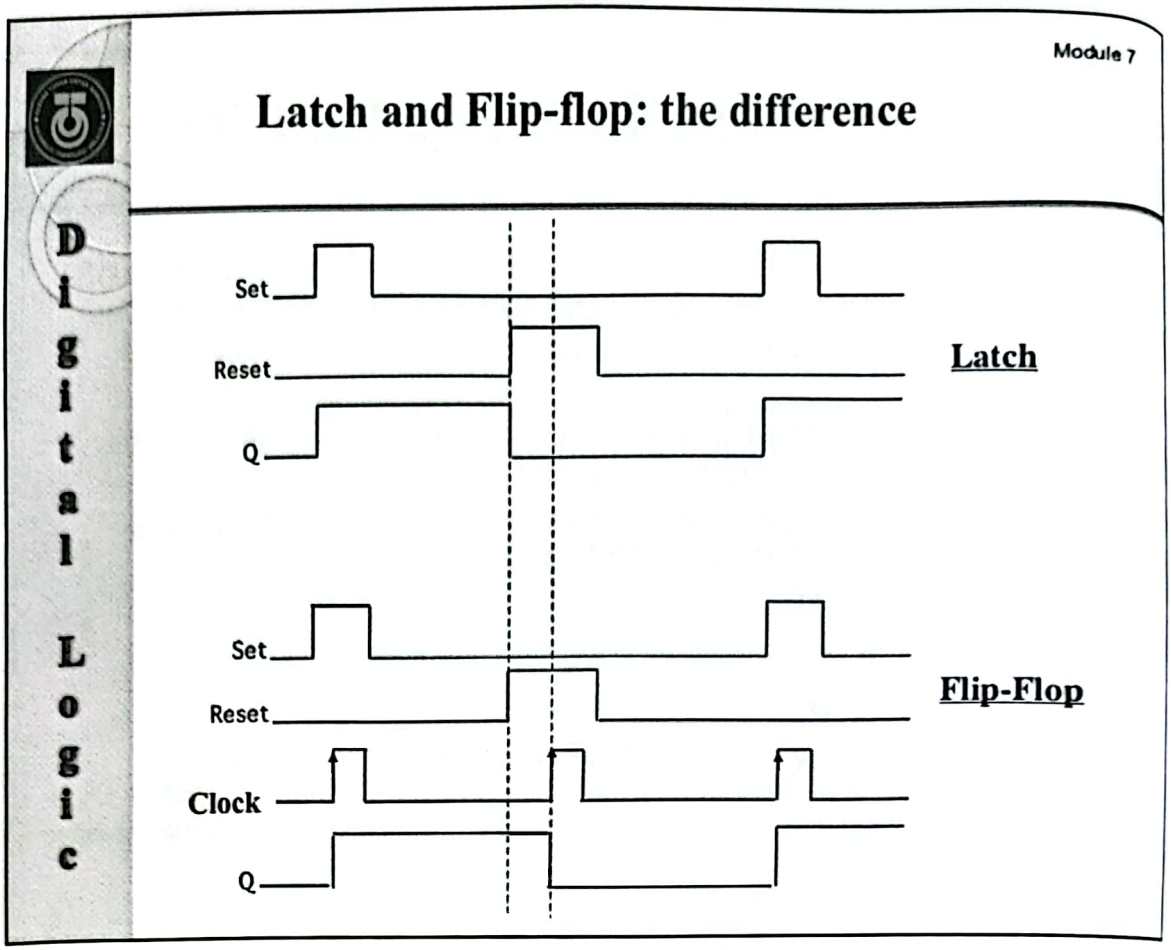
- **FF** are synchronous bi-stable storage devices capable of storing one bit, where the output state only changes at a specified point on a triggering input called the Clock (CLK).
- FF are edge-triggered, means that the output changes on the transition of a clock signal
  - This may either be a LOW-to-HIGH (rising edge) or a HIGH-to-LOW (falling edge) transition.
- Type of FF :
  - i) S-R
  - ii) D
  - iii) J-K
  - iv) T

***INFO :***
Rising edge = positive edge
Falling edge = negative edge

CamScanner

# Latch and Flip-flop: the difference






**Explanation:**

- The output of latch, Q is determined only by the input Set and Reset. If Set is HIGH and Reset is LOW, Q will be HIGH. If Set is LOW and Reset is HIGH, Q will be LOW.

- As for the Flip-flop, another input is added, which is the Clock. The purpose of the Clock is for synchronization. We can see that when Set is LOW, Reset is HIGH, the output Q is still in HIGH state until the next occurrence of the clock positive edge.

- The difference between a latch and a flip-flop is the way it makes the output change state
  - For latch: any changes to the input will have a direct impact to the output
  - For Flip-flop : changes at the input will have an impact only at the clock edge

*** Please note : the explanation is different for a gated latch, that we will cover later.*



CamScanner

# Latches

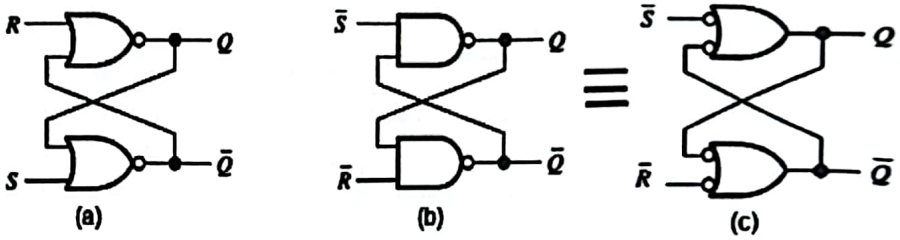
Digital Logic

S-R Latch
Gated S-R Latch
Gated D Latch

CamScanner

# S-R Latch

- The output of each gate is connected to an input of the other gate
- This produces a regenerative feedback (cross-coupled)
- Can be build using
  - NOR - active HIGH
  - NAND – active LOW




Active-HIGH input S-R latch

Active-LOW input $\bar{S}$-$\bar{R}$ latch

When using static gates as building blocks, the most fundamental latch is the simple SR latch, where S and R stands for SET and RESET. It can be constructed from a pair of cross-coupled NOR (Not OR) logic gates. The stored bit is present on the output marked Q.

Circuit (a) Active-HIGH input S-R latch :

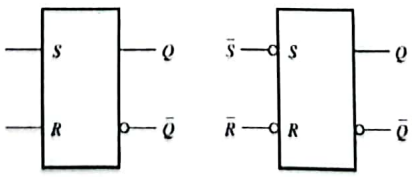
- use NOR gate
- Input R and S
- Output Q and Q'

Circuit (b) Active-LOW input S'-R' latch :

- use NAND gate
- Input S' and R'
- Output Q and Q'

Circuit (c) is equivalent to circuit (b), the only difference is symbol used in the schematic. The symbol in (c) is the dual symbol of NAND gate. This symbol can represent the active LOW input better.

Symbol for :




(a) Active-HIGH input S-R latch

(b) Active-LOW input $\bar{S}$-$\bar{R}$ latch



CamScanner

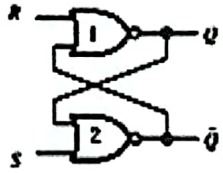
# SR Truth-Tables (Active HIGH)

**Active-HIGH**

| INPUTS | | OUTPUTS | | COMMENTS |
|---|---|---|---|---|
| $S$ | $R$ | $Q$ | $\overline{Q}$ | |
| 0 | 0 | NC | NC | No change. Latch remains in present state |
| 0 | 1 | 0 | 1 | Latch RESET |
| 1 | 0 | 1 | 0 | Latch SET |
| 1 | 1 | 0 | 0 | Invalid condition |

- The Q and Q' outputs are supposed to be in **opposite** states.
- When S and R are both equal to **0**, the Latch outputs "***Not Change***" in their prior states.
- **Q=1** and Q'=0 is defined as ***Set*** (by making S=1 and R=0)
- Q=0 and **Q'=1** is conversely defined as ***Reset*** (by making S=0 and R=1)
- If Q and Q' happen to be forced to the same state (both 0 or both 1), that state is referred to as ***invalid.***

Active High S-R Latch:




We use **NOR Gate, which has the characteristic of an output of 0, if any of the input is 1.**

→ From the circuit : $Q = \overline{R + \overline{Q}}$

$\overline{Q} = \overline{S + Q}$

**State 1:** S = 1, R = 0. Because S = 1, the output of Gate 2 which is Q', must be 0.

- Output from Q' fed back to Gate 1 which make the input both 0. Therefore the output Q = 1.
- This state is called **SET**

**State 2:** S = 0, R = 1. Because R = 1, the output of Gate 1 which is Q, must be 0.

- Output from Q fed back to Gate 2 which make the input both 1. Therefore the output Q = 0.
- This state is called **RESET**

**State 3:** S = 0, R = 0.

Assume initial state Q = 1, Q' = 0.
Q = (R+Q')' and Q' = (S+Q)',
therefore Q = 1 when R = 0 and Q' = 0,
Q' = 0 when S = 0 and Q = 1

Assume initial state Q = 0, Q' = 1.
Q = (R+Q')' and Q' = (S+Q)'
therefore Q = 0 when R = 0 and Q' = 1
Q' = 1 when S = 0 and Q = 0

- we can conclude that there is **no change** in the output
- This state is called **No Change** or **Hold**

**State 4:** S = 1, R = 1. Assume initial state Q = 1, Q' = 0.

Q = (R+Q')' and Q' = (S+Q)', when R = 1 and Q' = 0 then Q will be 0 and S = 1 and Q = 1 then Q' = 0

- This state is not allowed because Q' must be the complement of Q
- This state is called the **invalid state**



CamScanner

# SR Truth-Tables (Active LOW)

**Active-LOW**

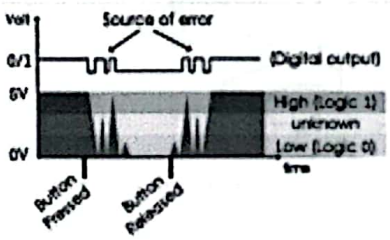
| INPUTS | | OUTPUTS | | COMMENTS |
|---|---|---|---|---|
| $\overline{S}$ | $\overline{R}$ | $Q$ | $\overline{Q}$ | |
| 0 | 0 | 1 | 1 | Invalid condition |
| 0 | 1 | 1 | 0 | Latch SET |
| 1 | 0 | 0 | 1 | Latch RESET |
| 1 | 1 | NC | NC | No change. Latch remains in present state |

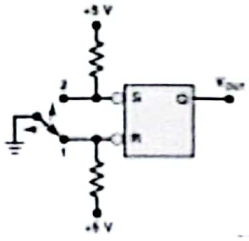
- The Q and Q' outputs are supposed to be in opposite states.
- **Q=1** and Q'=1 is defined as Invalid condition (by making S'=0 and R'=0)
- **Q=1** and Q'=0 is defined as Set condition (by making S'=0 and R'=1)
- Q=0 and **Q'=1** is conversely defined as ***Reset*** (by making S'=1 and R'=0)
- When S' and R' are both equal to **1**, the Latch outputs ***"Not Change"*** from their prior states.

**Example of an application:**

→ Switch debouncer

Switch is a mechanical device, although when press it makes an electrical contact, actually it makes multiple "contact and release" before the permanent contact. The effect is that we will get a multiple input of 0 and 1, where it should be only one 0 or 1.



To overcome the problem, use an active low SR latch. S' and R' input connected to +5V, this will provide a logic HIGH to the input. A switch is connected to ground that provides a logic LOW.

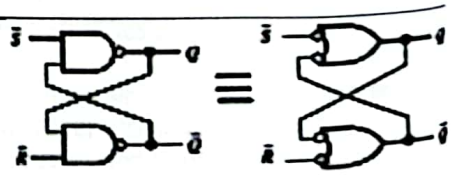
Let say a switch at position 1, it will make R' = 0 and S' = 1, latch will reset and Q becomes 0.

When the switch move to position 2 but at a position 1 and 2 (not in contact with 2 yet) S' = R'= 1 and the state is HOLD and the output stays at 0, even though the switch bounce and there is the possibility of 0 and 1, it wont affect the output because R'= 0 will make Q = 0 and a R'= 1 will hold the previous value which is 0.

The same argument apply to position 2, when the switch make a contact, S' = 0 and Q will be 1. Even though it bounce, S' will have a value of 1 and 0, but Q will not change because S'= 0 will make Q-1 and S'=1 will be a HOLD state because R' = 1.

**DIY:**

An active LOW SR Latch can be build by using NAND gate. Explain the operation of the circuit using the same argument as the explanation of an active HIGH SR latch.

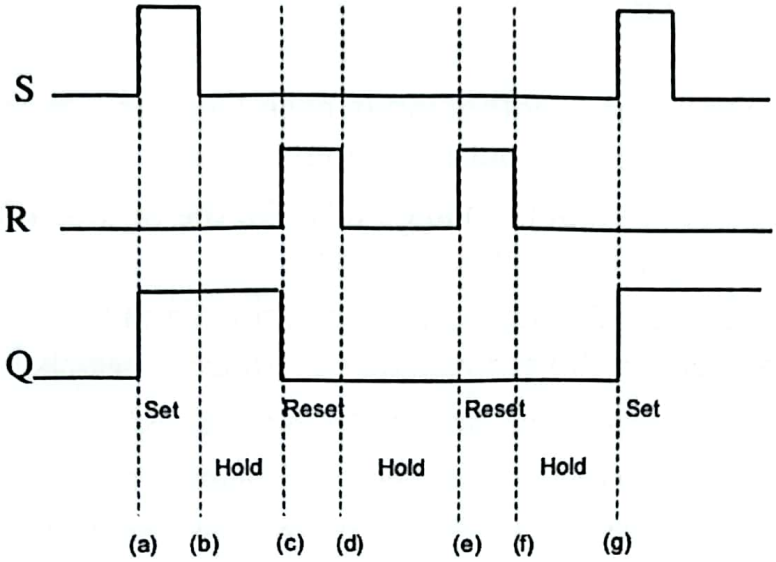


*Hint: It will be easy if you use the equivalent circuit (negative OR) on the right side.*



CamScanner

# Example : S-R Latch (active HIGH)




State Table for active HIGH SR Latch based on the above waveform

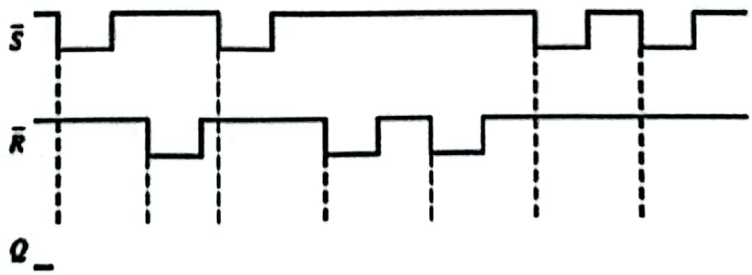
| Location | S | R | Q | State |
|---|---|---|---|---|
| (a) | 1 | 0 | 1 | SET |
| (b) | 0 | 0 | 1 | HOLD |
| (c) | 0 | 1 | 0 | RESET |
| (d) | 0 | 0 | 0 | HOLD |
| (e) | 0 | 1 | 0 | RESET |
| (f) | 0 | 0 | 0 | HOLD |
| (g) | 1 | 0 | 1 | SET |

**DIY:** Do the following:

a) Draw the missing dotted line for every signals change.

b) Draw waveform for Q.

c) Label the output state.






CamScanner

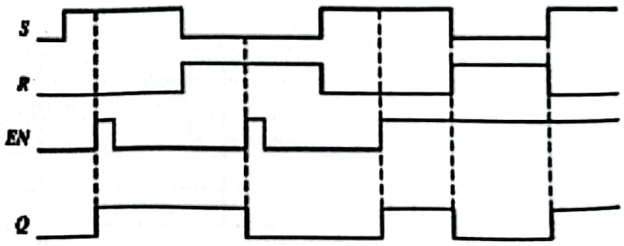
# Gated S-R Latch

- S-R latch is *asynchronous*, which means that the output changes very soon after the input S and R change
- Gated S-R has another input, called **enable** (EN), that control when the output should change
- When this input is enabled (EN =1), the output will depend on the value of S and R.
- If EN=0, the SR latch is disable, input S and R doesn't have an effect on the output, the output does not change.
- EN pin has a higher priority than pin S and R
  - SR input will only has an effect to the output if pin EN =1
  - The EN pin can be use to prevent the occurrence of invalid state at the output.

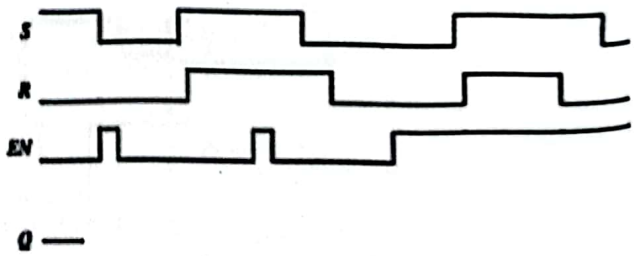
Truth Table for Gated SR Latch operation

| EN | S | R | Output | Disable / Enable |
|---|---|---|---|---|
| 0 | X | X | No Change | Disable |
| 1 | 0 | 0 | No Change | Enable |
| 1 | 0 | 1 | Q = 0 | Enable |
| 1 | 1 | 0 | Q = 1 | Enable |
| 1 | 1 | 1 | Invalid | Enable |

Timing diagram that shows the effect of EN input



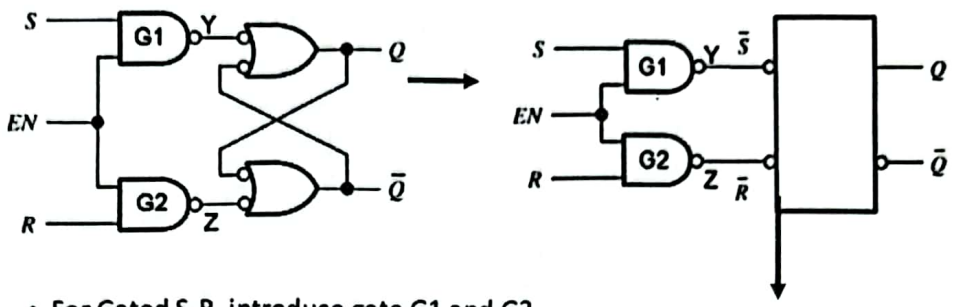

**DIY:** Draw the output Q waveform.



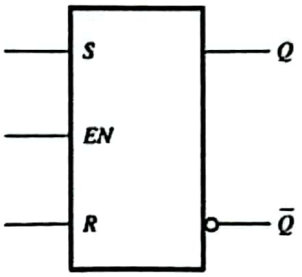



CamScanner

# Gated S-R Latch: Circuit and Symbol




- For Gated S-R, introduce gate G1 and G2
  - behaves as a gate that control whether to allow S to Y and R to Z or to make Y=Z=1.
  - Y = (S.EN)' and Z = (R.EN)'
    - If EN = 0, Y =1 and Z=1
    - If EN=1, Y =S' and Z = R' this will change an active low SR to an active HIGH SR flip-flop




**DIY:**

Referring to logic diagram above, fill in the table. Assume initially Q = 0 and Q' = 1

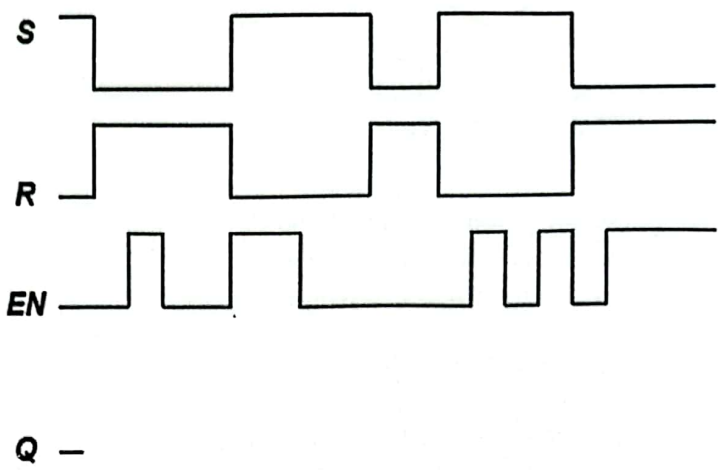
| EN | S | R | Y | Z | Q |
|---|---|---|---|---|---|
| 0 | 0 | 0 | | | |
| 0 | 0 | 1 | | | |
| 0 | 1 | 0 | | | |
| 0 | 1 | 1 | | | |
| 1 | 0 | 0 | | | |
| 1 | 0 | 1 | | | |
| 1 | 1 | 0 | | | |
| 1 | 1 | 1 | | | |



CamScanner

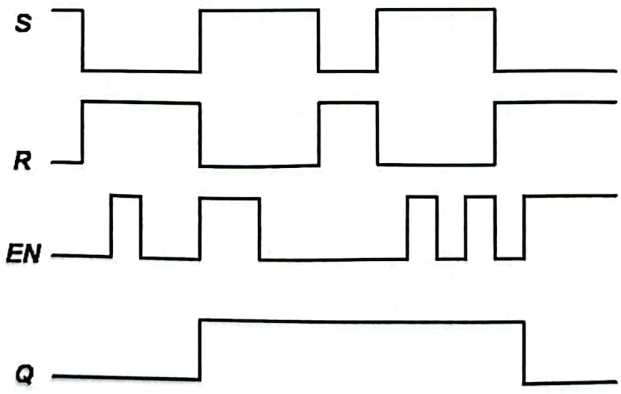
# Exercise: Gated S-R Latch

Draw waveform for Q.




**DIY:**

Add the vertical line for EN waveform. Deduce the understanding of the Q waveform.

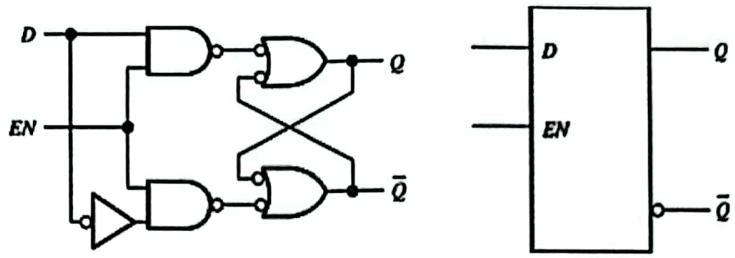





CamScanner

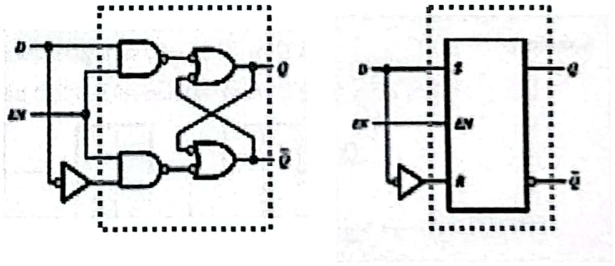
# Gated D Latch

## Logic Symbol & Logic Diagram




Logic diagram  Logic symbol

The Gated SR in the dotted rectangle on the left can be represented by a symbol in the dotted rectangle on the right. From the Gated SR, we can construct the Gated D latch by adding an inverter.



The purpose of the inverter is to make sure that R is the complement of S and R will never be the same as S. So that, we have only two condition at the input of SR.

**Condition 1:** D = 0, Therefore S = 0, R = 1 which make Q = 0

**Condition 2:** D = 1, Therefore S = 1, R = 0 which make Q = 1

Just like gated SR, when EN = 0, the output does not change. BUT, if EN = 1, the output will depend on the value of input D.

So, we can conclude that Q = D when EN = 1.

The characteristic of the gated D latch can be represented in the truth table →

| EN | D | Output |
|---|---|---|
| 0 | 0 | No Change |
| 0 | 1 | No Change |
| 1 | 0 | Q = 0 |
| 1 | 1 | Q = 1 |

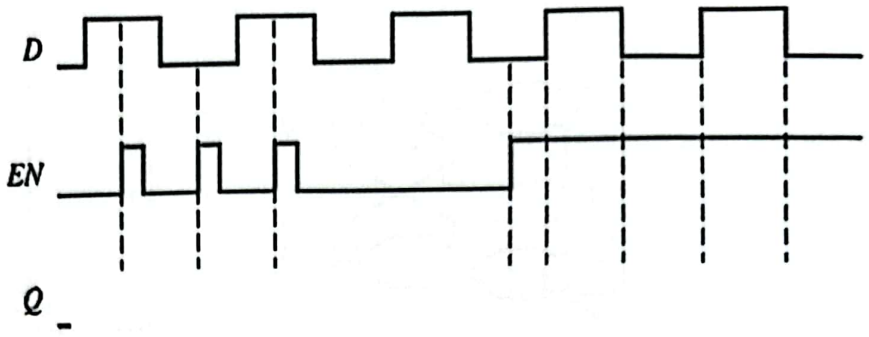


CamScanner

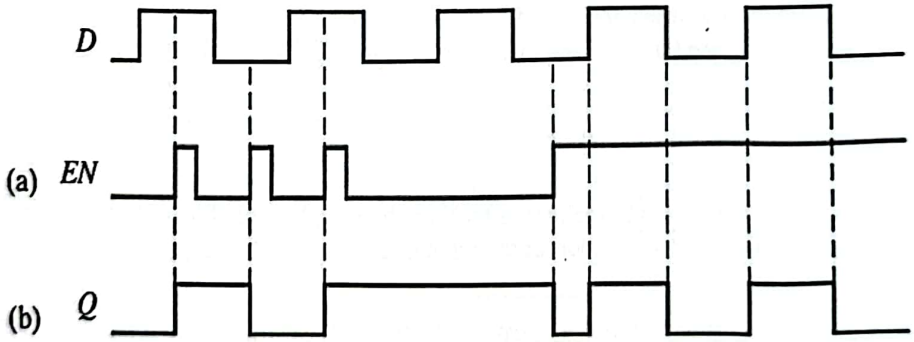
# Exercise: Gated D Latch



Draw waveform for Q.




**Solution:**

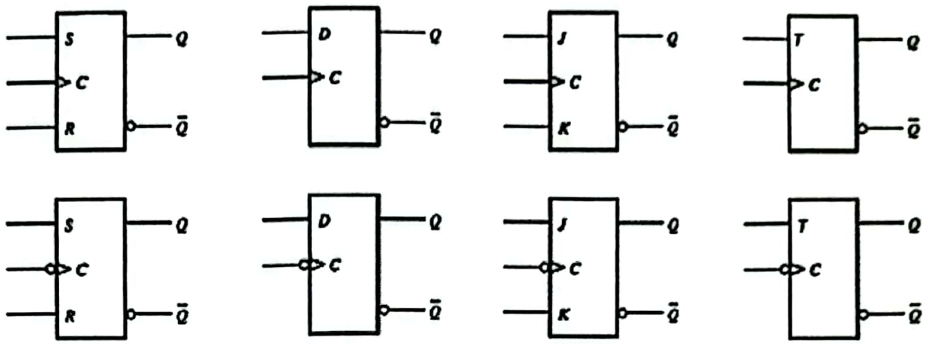





CamScanner

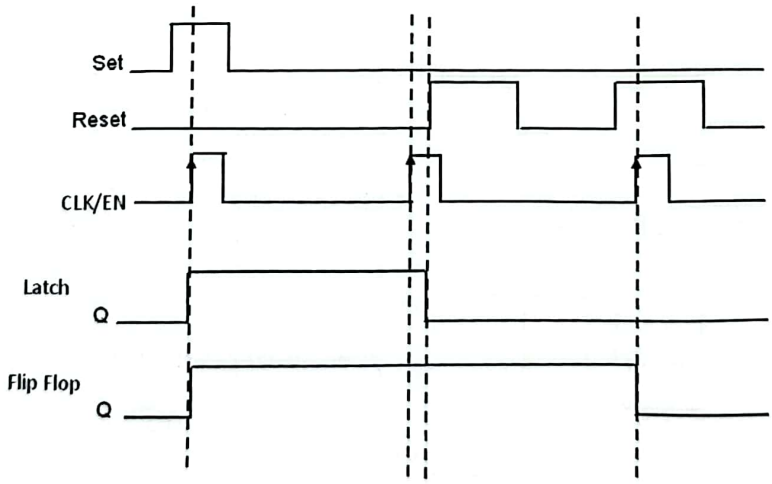
# Edge-Triggered FF

- Flip-flop is an important device in a sequential circuit




(top: positive edge-triggered; bottom: negative edge-triggered)

- The different between Gated Latch (EN) and Flip-Flop (CLK):
  - Gated Latch is activated by using a level triggered EN (i.e '0' or '1')
  - FF is activated by using edge triggered CLK (i.e 'transition from '0 to 1' or '1 to 0')

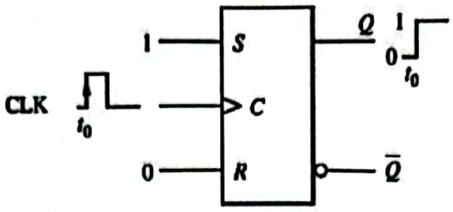



- Four type of FFs: SR (Set/Reset), D (Delay/Data), JK and T (Toggle)
- Triggering of the FF (CLK): +tve edge, –ve edge and Master-slave (obsolete)
- Latch and FF are the basic of sequential circuits.

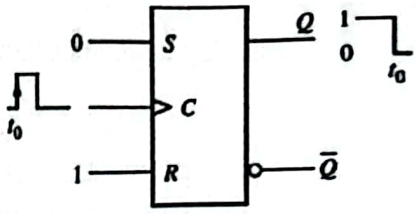


CamScanner

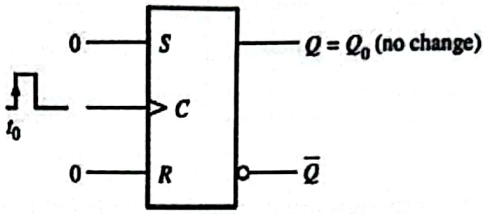
# S-R FF: operation




(a) $S = 1, R = 0$ flip-flop SETS on positive clock edge. (If already SET, it remains SET.)




(b) $S = 0, R = 1$ flip-flop RESETS on positive clock edge. (If already RESET, it remains RESET.)




(c) $S = 0, R = 0$ flip-flop does not change. (If SET, it remains SET; if RESET, it remains RESET.)

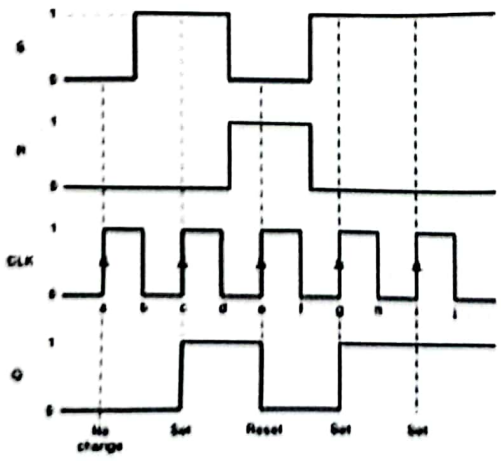
Truth Table for an **active HIGH S-R** flip-flop

| S | R | CLK | Output | | Comments |
|---|---|---|---|---|---|
| | | | Q | $\bar{Q}$ | |
| 0 | 0 | X | $Q_0$ | $\bar{Q}_0$ | No Change |
| 0 | 1 | ↑ | 0 | 1 | RESET |
| 1 | 0 | ↑ | 1 | 0 | SET |
| 1 | 1 | ↑ | ? | ? | Invalid |

TRUTH TABLE

↑ = clock transition LOW to HIGH
X = irrelevant (don't care)
$Q_0$ = output level prior to clock transition

Timing diagram of an **active HIGH S-R** flip-flop

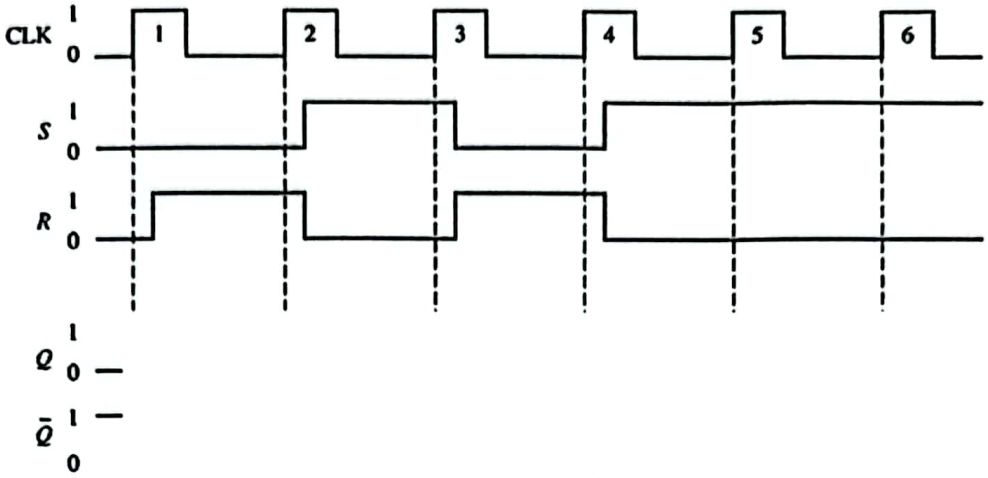



**Note :** In our class, the logic of input at clock edge is determine by looking at it logic value immediately before the clock edge. What actually happens in the real circuit will depend on the setup and hold time of the flip-flop.

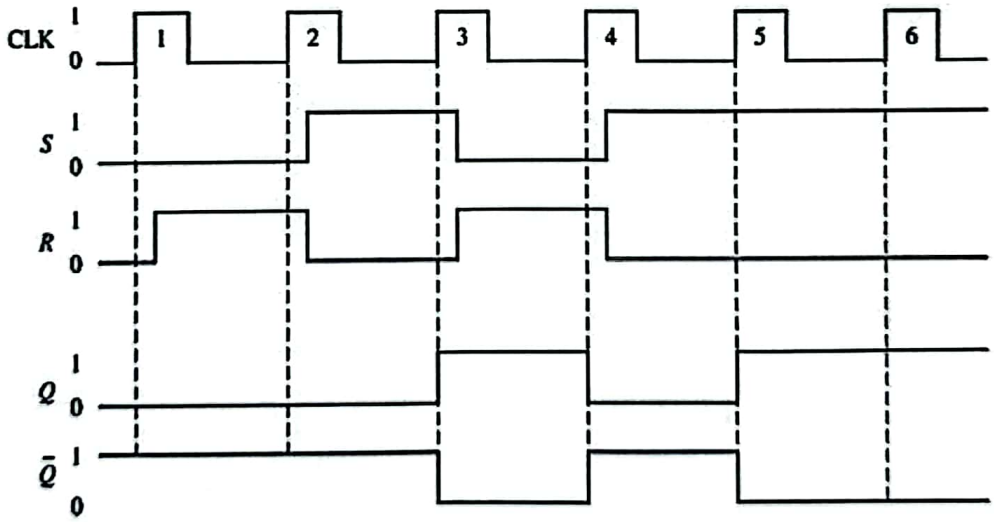


CamScanner

# Example : S-R FF




Solution

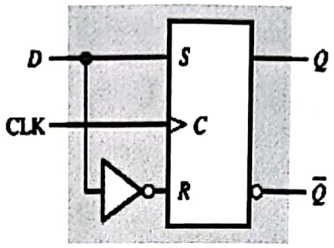


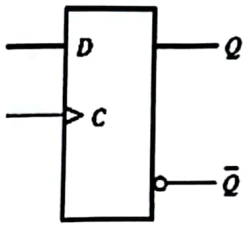



CamScanner

# D Flip-Flop

- D FF is useful when a single data bit (1 or 0) is to be stored
  - D can mean Data ( ability to store data) or Delay (to delay a signal by one clock cycle)
- The addition of an inverter to an S-R FF creates
- basic D FF where a positive edge-triggered type is shown.



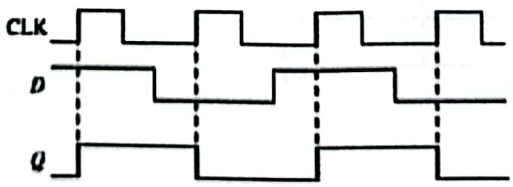




- These flip flops are very useful, as they form the basis for **shift registers** (that we will learn later)
- The advantage of the D flip-flop over the D-type latch is that it "captures" the signal at the moment the clock goes high, and subsequent changes of the data line do not influence Q until the next rising clock edge (for a positive edge D flip-flop)

D Flip-flop truth table:

| INPUTS | | OUTPUTS | | COMMENTS |
|---|---|---|---|---|
| D | CLK | Q | $\bar{Q}$ | |
| 1 | ↑ | 1 | 0 | SET (store a 1) |
| 0 | ↑ | 0 | 1 | RESET (store 0) |

Timing diagram for a positive edge D flip-flop




**Remember, Q follows D at the active or triggering clock edge**

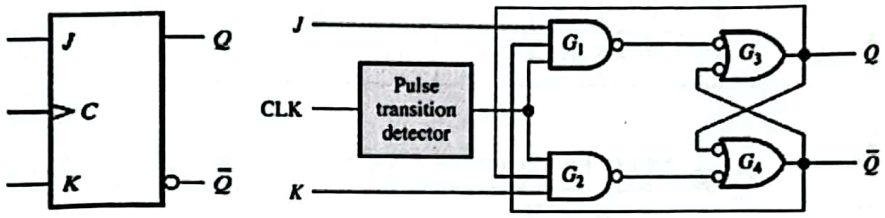


CamScanner

# J-K Flip Flop

- The JK FF is versatile and is a widely used type of FF.
- The difference between J-K and S-R is a J-K has **no invalid state** as SR.

Logic Symbol




The JK flip-flop is a universal flip-flop, because it can be configured to work as an SR flip-flop, a D flip-flop, or a T flip-flop.

The characteristic equation of the JK flip-flop is:

$$Q_{next} = J\overline{Q}_{prev} + \overline{K}Q_{prev}$$

The problem with active HIGH SR is when both of the inputs are HIGH (i.e. invalid state).

In JK flip-flop, J has the same characteristic as active HIGH S and K has the same characteristic as active HIGH R.

However, in JK FF when J = K = 1 and plug in into JK characteristic equation

- ✓ If $Q_{prev}$ =1 then $Q_{next}$ will be 0.
- ✓ If $Q_{prev}$ = 0, then $Q_{next}$ will be 1

Therefore, there is no **invalid state** BUT we have a new state called **TOGGLE**, where the next output value is NOT the previous output:

$$Q_{next} = \overline{Q}_{prev}$$



CamScanner

# Truth Table: J-K FF

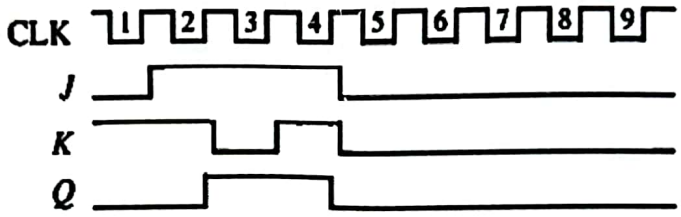
| J | K | CLK | Output | | Comments |
|---|---|---|---|---|---|
| | | | Q | $\overline{Q}$ | |
| 0 | 0 | ↑/↓ | $Q_0$ | $\overline{Q_0}$ | No Change |
| 0 | 1 | ↑/↓ | 0 | 1 | RESET |
| 1 | 0 | ↑/↓ | 1 | 0 | SET |
| 1 | 1 | ↑/↓ | $\overline{Q_0}$ | $Q_0$ | Toggle |

**TRUTH TABLE**

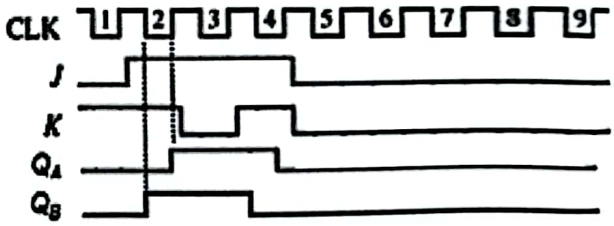
↑ = clock transition LOW to HIGH

$Q_0$= output level prior to clock transition

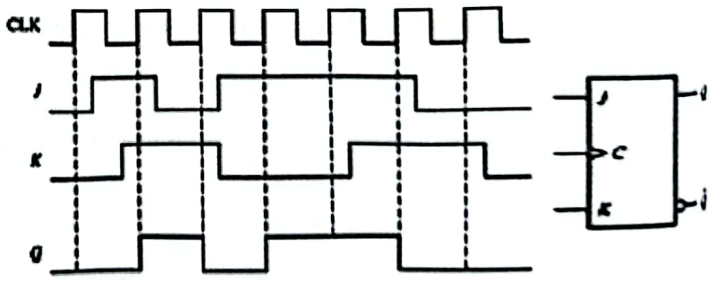
Positive or negative triggered ?



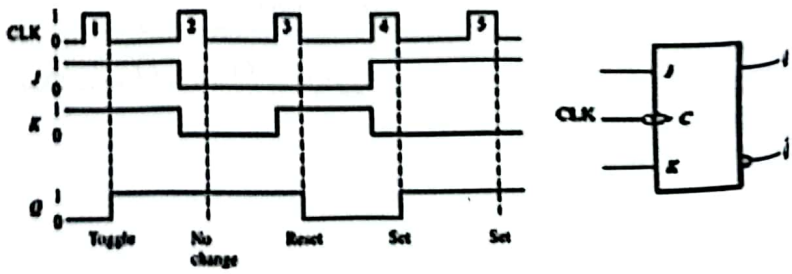

- The truth table above shows the output for positive edged-triggered J-K Flip-Flop.
- The output Q for a negative edged-triggered J-K Flip-Flop is shown in $Q_B$:




**Example 1** : Positive Edge




**Example 2**: Negative Edge

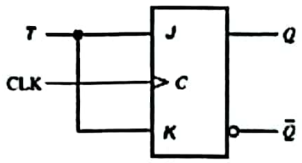




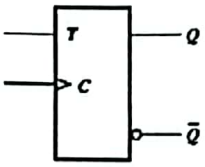

CamScanner

# T Flip Flop

- Also called as Toggle flip flop.
- Can be build from JK by connecting J and K to the same node.
- Therefore it will always make J=K. Thus, there is only 2 output states (i.e. either HOLD or TOGGLE).




Logic Diagram

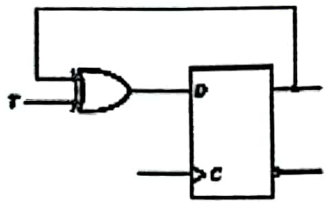



Logic Symbol

- Truth table for T Flip-flop

| T | $Q_{prev}$ | $Q_{next}$ | Comment |
|---|---|---|---|
| 0 | 0 | 0 | Hold state |
| 0 | 1 | 1 | |
| 1 | 0 | 1 | Toggle state |
| 1 | 1 | 0 | |

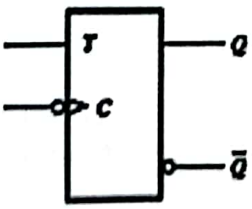
- D can be used to build a T flip-flop as in circuit →

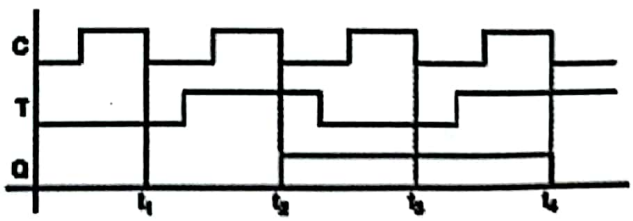



$$D = T \oplus Q = T\overline{Q} + \overline{T}Q$$

- The expression for D input
  - When T = 0 then D = Q, if T = 1, D = Q', it behaves as aT flip-flop.

- Timing diagram of a negative edge T Flip-flop



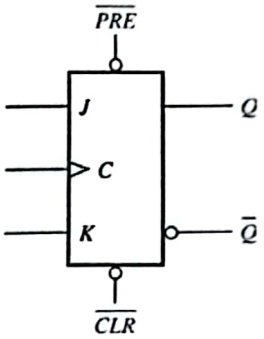



CamScanner

# FF Asynchronous Input

- Synchronous → inputs transferred on the triggering edge of the clock
- Asynchronous → inputs effect FF state independent of the clock
  - Normally labeled (depends on the manufacturer)
    - preset ($\overline{PRE}$) or direct set
    - clear ($\overline{CLR}$) or direct reset

| $\overline{PRE}$ | $\overline{CLR}$ | FF | MODE |
|---|---|---|---|
| 0 | 1 | SET | Asynchronous |
| 1 | 0 | RESET | Asynchronous |
| 1 | 1 | JK | Synchronous |




- Flip-flop input priority:

| Priority | FF Input |
|---|---|
| Highest | $\overline{PRE}$, $\overline{CLR}$ - asynchronous input |
| Medium | Clock |
| Lowest | SR, JK, D, T - synchronous input |

- Flip-flop asynchronous input has the highest priority.
  - Which means that if the asynchronous input active, the output will immediately change regardless the value of synchronous input or clock.

- There are two types of asynchronous input:
  - preset , $\overline{PRE}$ → to set initial value of 1 to the output
  - clear, $\overline{CLR}$ → to reset the output to 0

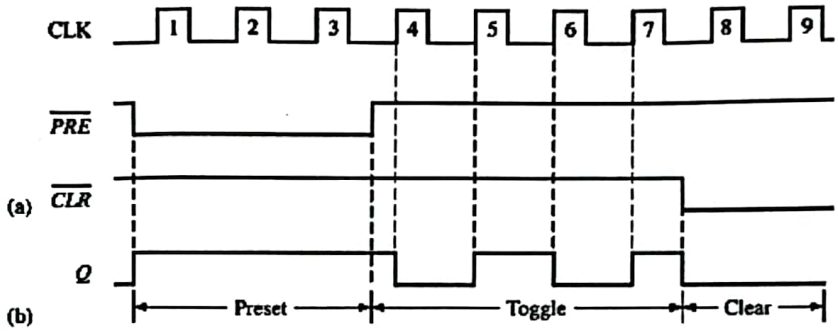
- Usually the asynchronous input are applied momentarily ( in a short period of time), because to allow the normal operation of the FF both asynchronous input must be deactivated (HIGH) .



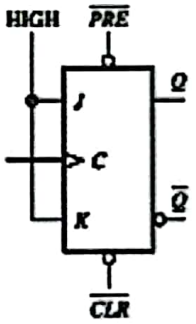
CamScanner

# Example: J-K FF

| clk pulse | $\overline{PRE}$ | $\overline{CLR}$ | J | K | FF State | Comment |
|---|---|---|---|---|---|---|
| 1,2,3 | 0 | | 1 | 1 | SET | JK inputs - dont care |
| 4,5,6, 7 | 1 | 1 | 1 | 1 | Toggle | Synchronous mode. |
| 8,9 | | 0 | 1 | 1 | RESET | JK inputs - dont care |




- The timing diagram is for JK Flip Flops connected as T FF, as shown in the figure →

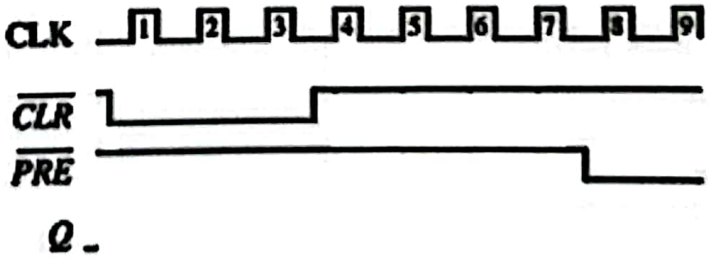



- From the timing diagram, it can clearly illustrates the asynchronous characteristic of the $\overline{PRE}$ and $\overline{CLR}$, where when the asynchronous input change the output will immediately change without waiting for the clock edge.

***Tips:***

- *If you are required to draw the timing diagram with an asynchronous input, draw output cause by the asynchronous input first. When draw this output ignore the edge of the clock.*
- *After that you can ignore the asynchronous input, draw the effect of a synchronous input and take note that the output will only change at the edge of the clock.*

**DIY:** Draw the output, Q waveform



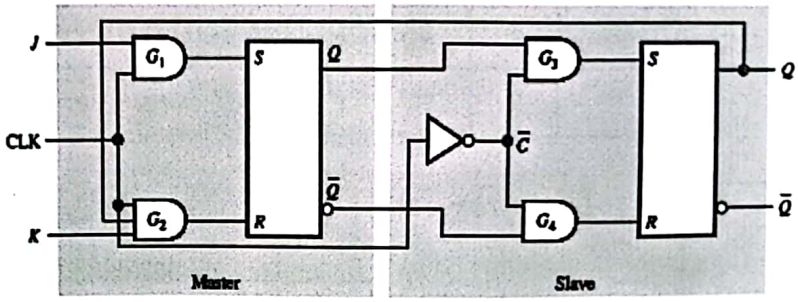



CamScanner

# Master Slave Flip Flops (JK Example)

Digital Logic

- This type of Flip-flop is already obsolete, the mention here is for the sake of completeness
- You must know it exist, where you might find in the old circuit board.




- There are 2 sections called as :
  - **Master section** - *external JK input* followed by a *gated latch*
  - **Slave section** - an *inputs which is the outputs of Master* and the *I inverted clock pulse*

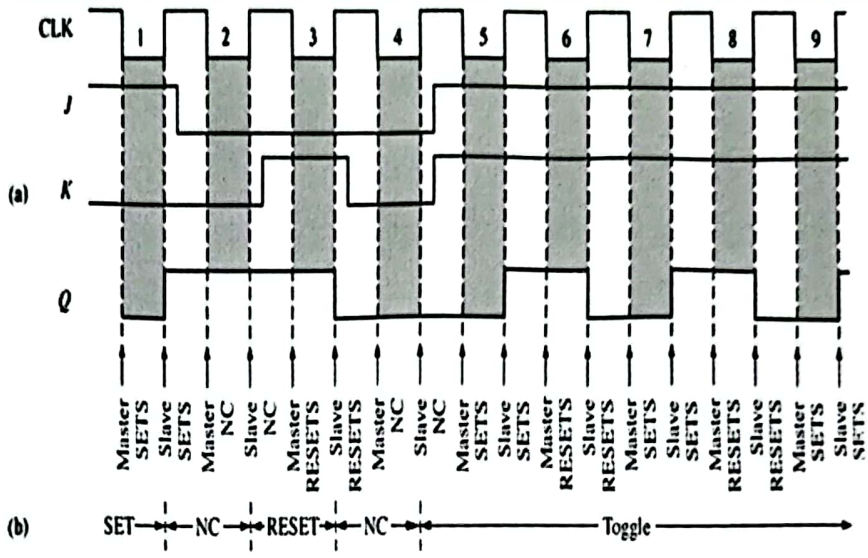
Master-slave flip-flop:

- Another class of flip-flop is the pulse-triggered or master-slave. These flip-flops are constructed from two separate flip-flops. The term pulse-triggered means that data are entered into the flip-flop on the leading edge of the clock pulse, but the output does not reflect the input state until the trailing edge of the clock pulse. This is due to the master flip-flop being rising edge triggered and the slave flip-flop being falling edge triggered as illustrated in the above slide.
- A type of clocked flip-flop consisting of master and slave elements that are clocked on complementary transitions of the clock signal.
- Data is only transferred from the master to the slave, and hence to the output, after the master-device outputs have stabilized.
- This eliminates the possibility of ambiguous outputs, which can occur in single-element flip-flops as a result of propagation delays of the individual logic gates driving the flip-flops.

CamScanner

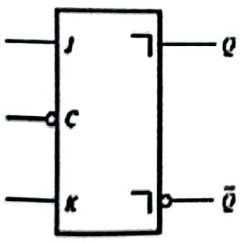
# Master Slave Flip Flops






**Operation of master-slave J-k flip-flop**

- The logic symbol for the master-slave flip-flop only indicates the initial inputs to the master and the outputs from the slave




Logic symbol for J-K master-slave flip-flop

| J | K | C | Q | $\bar{Q}$ | Operation |
|---|---|---|---|---|---|
| 0 | 0 | Pulse | $Q_0$ | $\bar{Q}_0$ | Hold (no change) |
| 0 | 1 | Pulse | 0 | 1 | Reset |
| 1 | 0 | Pulse | 1 | 0 | Set |
| 1 | 1 | Pulse | $\bar{Q}_0$ | $Q_0$ | Toggle |

Truth table for J-K master-slave flip-flop

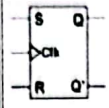
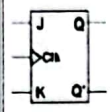
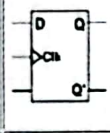
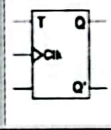
- J-K master slave flip-flop have the same characteristic with the edge triggered J-K , the only difference is how it was triggered and when the input evaluated and the effect appear at the output



CamScanner

# Summary of FF characteristic

Digital Logic

| FLIP-FLOP NAME | FLIP-FLOP SYMBOL | CHARACTERISTIC TABLE | CHARACTERISTIC EQUATION | EXCITATION TABLE |
|---|---|---|---|---|
| SR |  | S R $Q_{(next)}$<br>0 0 Q<br>0 1 0<br>1 0 1<br>1 1 ? | $Q_{(next)} = S + R'Q$<br>$SR = 0$ | Q $Q_{(next)}$ S R<br>0 0 0 X<br>0 1 1 0<br>1 0 0 1<br>1 1 X 0 |
| JK |  | J K $Q_{(next)}$<br>0 0 Q<br>0 1 0<br>1 0 1<br>1 1 Q' | $Q_{(next)} = JQ' + K'Q$ | Q $Q_{(next)}$ J K<br>0 0 0 X<br>0 1 1 X<br>1 0 X 1<br>1 1 X 0 |
| D |  | D $Q_{(next)}$<br>0 0<br>1 1 | $Q_{(next)} = D$ | Q $Q_{(next)}$ D<br>0 0 0<br>0 1 1<br>1 0 0<br>1 1 1 |
| T |  | T $Q_{(next)}$<br>0 Q<br>1 Q' | $Q_{(next)} = TQ' + T'Q$ | Q $Q_{(next)}$ T<br>0 0 0<br>0 1 1<br>1 0 1<br>1 1 0 |

- Characteristic table shows what the output will be for a given input
- Characteristic Equation allow us to determine what the output will be for a given input. Q is the previous input, to determine the output sometimes the previous output will need to be assumed.
- Excitation table can be used to determine what should be applied at the input to get certain output. This table will be use when we design a counter in the next module.

CamScanner

Digital Logic

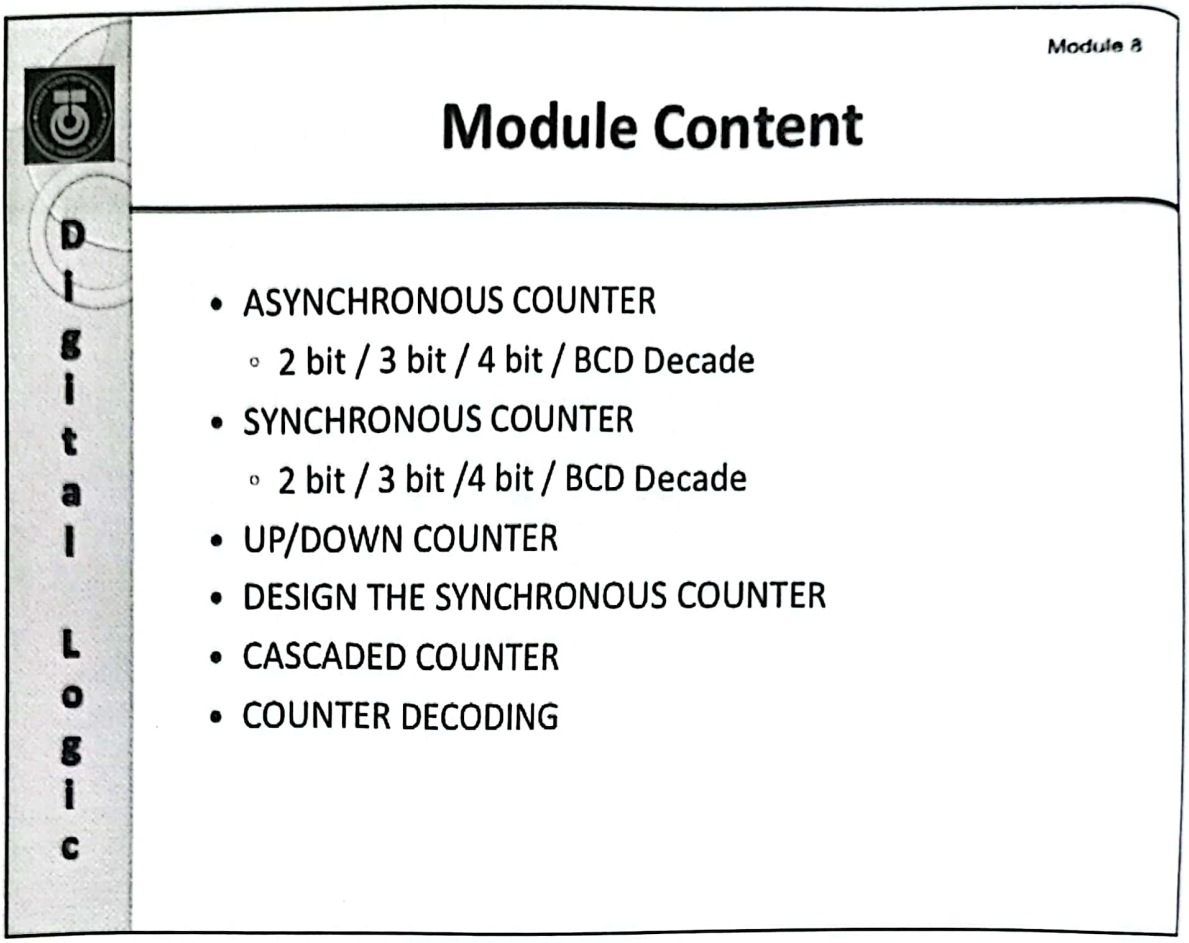
# MODULE 8
## COUNTERS

The objectives of this module are:

1. To distinguish the different between synchronous and asynchronous counters.
2. To illustrate the designing of counters .
3. To explain the application of counters.
4. To demonstrate the analysis of sequential circuits.



CamScanner




There is a variety of counters based on its construction, BUT in general it can be categorized into two categories Asynchronous and Synchronous counter. In both categories the counters can be differentiated by the following criterion:

1. Clock Trigger: Positive edged or Negative edged
2. Counts: Binary, Decade
3. Count Direction: Up, Down, or Up/Down

In the previous lectures, the characteristic of S-R, D, J-K and T flip-flops were discussed. These flip-flops can be connected together to perform certain operations.

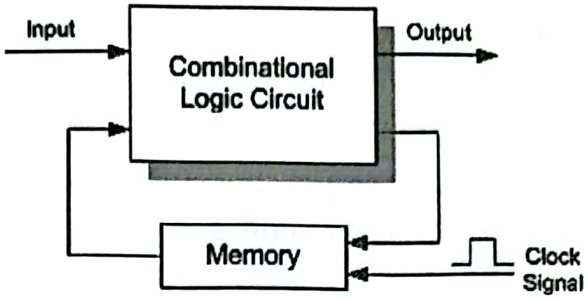
In the following lectures, we will use these flip-flops to construct a variety of counters. In addition we will also learned the method for analysing the different types of counters.



CamScanner

Digital Logic

# Recap: Sequential Circuit

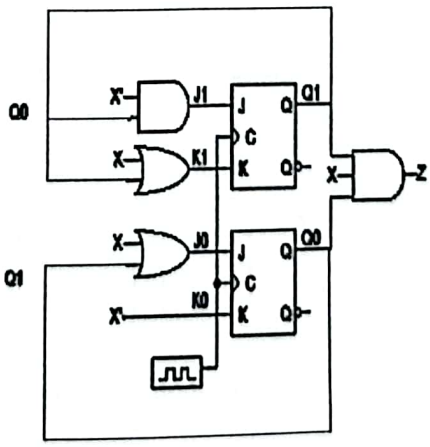



- General sequential circuit that consists of a combinational logic section and a memory section that is flip-flops.

- Memory elements are devices capable of storing binary information.
- Binary information stored at any given time defines the state of the sequential circuit.
- The input and the present state of the memory element determines the output.
- A sequential circuit is specified by a time sequence of inputs, outputs, and internal states.

How does sequential circuits look like?

Below is a sequential circuit with two JK flip-flops. There is input X and output Z. The values of the flip-flops (Q1Q0) form the state, or the memory, of the circuit. The flip-flop outputs also go back into the primitive gates on the left. This matches the abstract sequential circuit diagram in the slide.



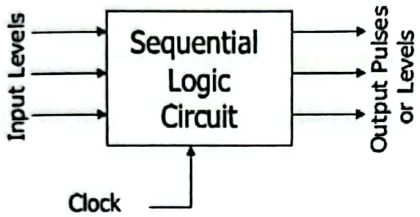



CamScanner

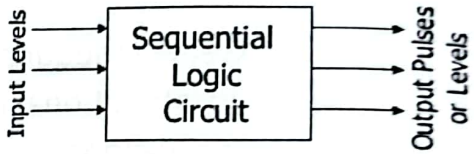
# Types of Sequential Circuits



- Asynchronous and Synchronous.
- Asynchronous sequential circuits do not use a common clock signal. The circuit operation is based on message (signal) passing from one stage (e.g. circuit) to another.
- Synchronous sequential circuits use clock inputs to synchronize the circuit operation.

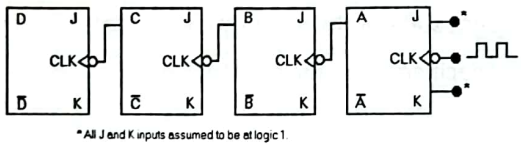



Synchronous




Asynchronous

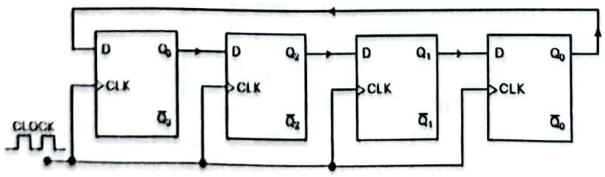
For asynchronous counters , not all flip flops are hooked to the same clock.




Asynchronous Counter

Asynchronous counters, also known as ripple counters, are not clocked by a common pulse and hence every flip-flop in the counter changes at different times. The flip-flops are usually clocked by the output pulse of the preceding flip-flop.

Synchronous counters have a common clock which results in all the flip-flops being triggered simultaneously.

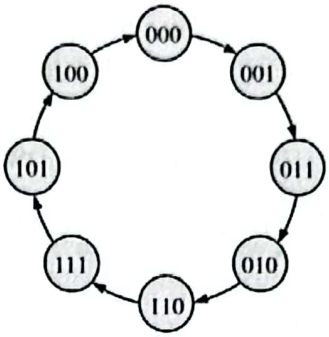



Synchronous Counter



CamScanner

# Counters




- Counters are sequential circuit that goes through a prescribed (predefine) sequence of states upon the application of input pulses.
- Sequence of states in a counter may follow a binary count or any other sequence.



- Counters can retain an output state after the input condition which brought about that state has been removed.
- Counter can have more than two states
  - value of a state is expressed as a multi digit binary number
  - the number `1's and `0's are usually derived from the outputs of internal flip-flops that make up the counter.
- The number of states a counter may have is limited only by the amount of electronic hardware that is available.
- The changes from one state to another is controlled by the input pulses (count pulses). It may be clock pulses or originated from an external source and may occur at prescribed intervals of time or at random.

- Counters are generally made up of flip-flops and logic gates.
  - Main types of flip-flops used in designing counter are J-K, T or D flip-flops.



CamScanner

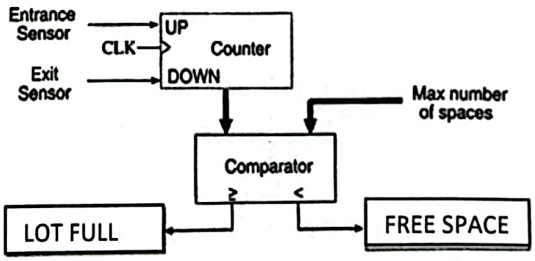
# Uses of Counters

- To count the number of times that a certain event takes place; the occurrence of event to be counted is represented by the input signal to the counter (see figure below).
- To control a fixed sequence of actions in a digital system
  - e.g gate A must be open before gate B.
- To generate timing signals
  - e.g a bomb can only be detonated 20 second after a button is pressed
- To generate clocks of different frequencies.
  - By dividing the frequency, we can get a different clock frequencies from the same source

An application of a counter:

1. **Parking Space**

   There are sensors at the entrance and exit of the parking lot. A car at the entrance will increment the counter and car exit from the parking lot will decrement the counter. The current number of car in the parking lot will be compared to the Max number of spaces that the parking lot has. If number of car less than max number of spaces then FREE SPACE display will lit else LOT FULL will be displayed.

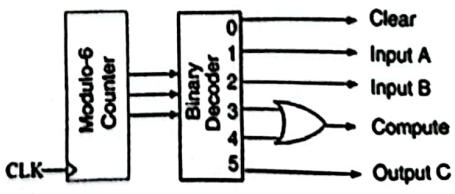



2. **Action sequencer**

   A counter can generate a number 0 to 5. Every positive edge of a clock the number will change to the next number. Every number generated by the counter will be decoded to perform some action. This will allow a sequence of actions to occurs as listed below.

**Sequence of actions:**

0: Clear all registers
1: Input A
2: Input B
3: Compute
4: Compute
5: Output C






CamScanner

# Count Sequence

- A counter may count up or count down or count up and down depending on the input control.
- Because of limited word length, the count sequence is limited.
  - For an *n*-bit counter, the range of the count is [0, $2^n$-1]. The count sequence usually repeats itself.
- When counting up, the count sequence goes in this manner: 0, 1, 2, ... $2^n$-2, $2^n$-1, 0, 1, ...etc.
- When counting down the count sequence goes in the same manner: $2^n$-1, $2^n$-2, ... 2, 1, 0, $2^n$-1, $2^n$-2, ... etc.

A counter can be constructed by an asynchronous circuit or by a synchronous circuit. With an asynchronous circuit, all the bits in the count do not all change at the same time. With a synchronous circuit, all the bits in the count change synchronously with the assertion of the clock.

A counter may count up or count down or count up and down depending on the input control.

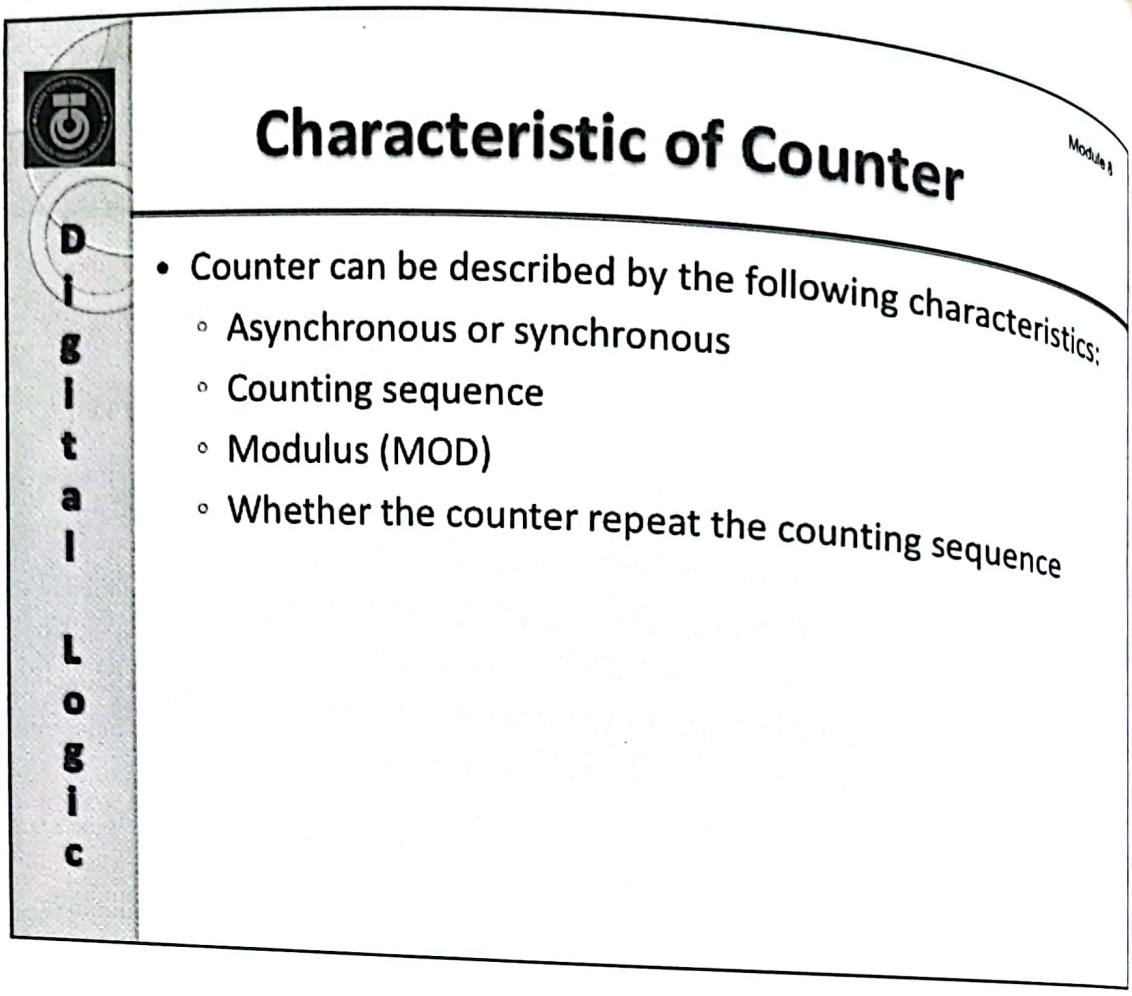
| 3-bit Up Counter | 3-bit Down Counter |
|---|---|
| 000 | 000 |
| 001 | 111 |
| 010 | 110 |
| 011 | 101 |
| 100 | 100 |
| 101 | 011 |
| 110 | 010 |
| 111 | 001 |

The complement of the count sequence counts in reverse direction. If the uncomplemented output counts up, the complemented output counts down. If the uncomplemented output counts down, the complemented output counts up.

| 3-bit Up Counter | Complement of the Count |
|---|---|
| 000 | 111 |
| 001 | 110 |
| 010 | 101 |
| 011 | 100 |
| 100 | 011 |
| 101 | 010 |
| 110 | 001 |
| 111 | 000 |



CamScanner

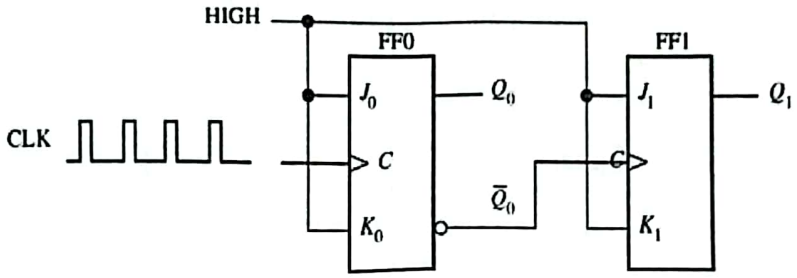



- Asynchronous or Synchronous
  - ✓ Whether the clock come from the same source or not
    - ❑ Synchronous – all clock from the same source
    - ❑ Asynchronous – clock for each flip-flop comes from a different source
- Counting sequence: Up or Down or Up-Down
  - ✓ Up or Down – unidirectional – the count sequence only one way.
  - ✓ Up-Down – bidirectional – the direction of the count can be change during operation.
- Modulus (MOD): the number of state that the counter can have.
  - ✓ e.g. MOD 4 ( 4 states with the following state 0, 1, 2, 3), MOD 9 (0, 1, 2, ... , 8).
  - ✓ the maximum count usually $2^N - 1$, where $N$ is the number of the FF, or for the truncated sequence the max count will be $< 2^N - 1$.
    E.g. MOD 9 (0, 1, 2, ..., 8) is truncated sequence.
- Whether the counter repeat the counting sequence:
  - ✓ Recycle – if the counter goes back to the first count after the maximum count
  - ✓ Saturated – the counter repeat the maximum count if count up or repeat the minimum if count down.

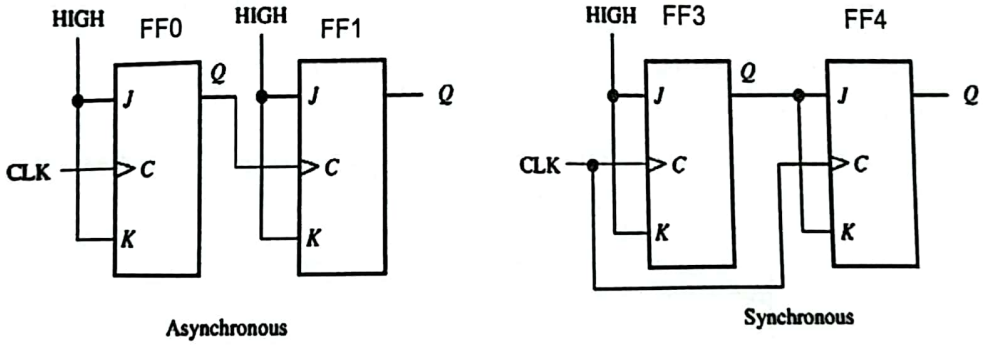
CamScanner

# Asynchronous Counter (AC)

- An AC is one in which the FF within the counter do not change states at exactly the same time because they do not have a common clock pulse.
- The clock input of an AC is always connected only to the LSB FF.
- The clock for the next FF come from the output of the previous FF
- An AC also known as **ripple** counter – message (signal) passing.




Comparison of Asynchronous and synchronous Counter. See the different clock connection!




Asynchronous counter operation:

- A two-bit asynchronous counter is shown on the left.
- The external clock is connected to the clock input of the first flip-flop (FF0) only.
- So, FF0 changes state at the positive edge of each clock pulse, but FF1 changes only when triggered by the positive edge of the Q output of FF0.
- Because of the inherent propagation delay through a flip-flop, the transition of the input clock pulse and a transition of the Q output of FF0 can never occur at exactly the same time.
- Therefore, the flip-flops cannot be triggered simultaneously, producing an asynchronous operation.

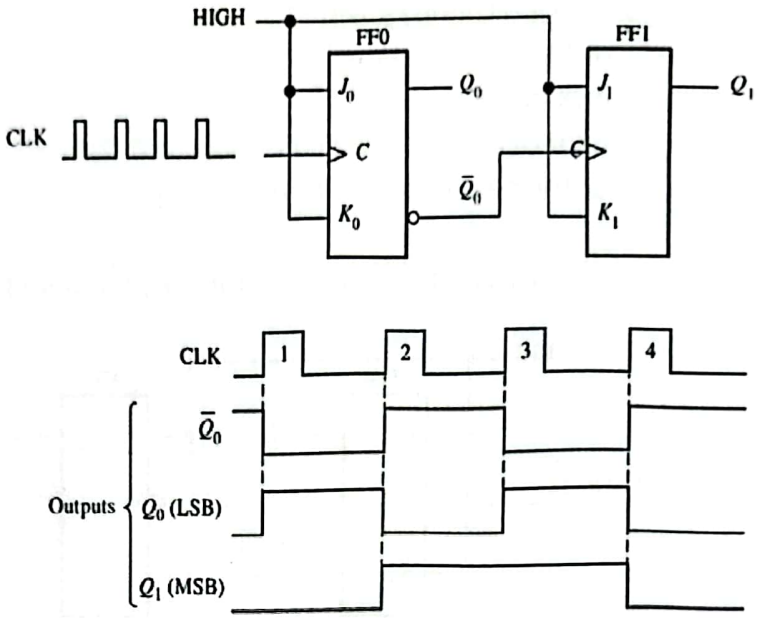
This asynchronous counter is slow because the cascaded clocking scheme:

- The clock source ripples from stage-to-stage
- The ripple effect is similar to that of a ripple carry adder circuit



CamScanner

# Count Up 2-Bit Asynchronous Counter




**Notice:**

1. Input JK for each FF is tied up together. Therefore J = K = 1 (HIGH). In other word, the JK FF function in a toggle mode that is every clock pulse Q will be complemented.
2. The clock of FF1 come from the complemented output of FF0, $\overline{Q}_0$
3. For FF0, every positive edge of the clock the output , $Q_0$ will toggle.
4. For FF1, the clock depend on $\overline{Q}_0$ and the output $Q_1$ will only toggle on the positive edge of $\overline{Q}_0$
5. The output of the circuit counter is read $Q_1 Q_0$

| CLOCK PULSE | $Q_1$ (MSB) (Bit 2) | $Q_0$ (LSB) (Bit 1) |
|---|---|---|
| Initially | 0 | 0 |
| 1 | 0 | 1 |
| 2 | 1 | 0 |
| 3 | 1 | 1 |
| 4 | 0 | 0 |

The 2-bit ripple counter circuit above has four different states, each one corresponding to a count value. Similarly, a counter with *n* flip-flops can have $2^n$ states. The number of states in a counter is known as its mod (modulo) number. Thus a 2-bit counter is a mod-4 counter.



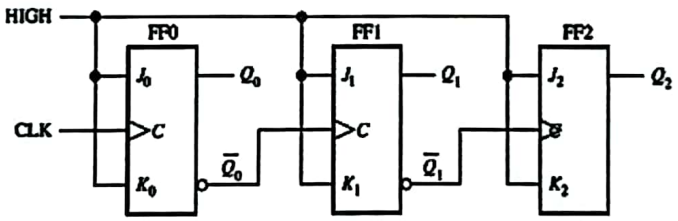
CamScanner

# 3-Bit Asynchronous Counter

| CLOCK PULSE | $Q_{2(MSB)}$ (Bit 3) | $Q_1$ (Bit 2) | $Q_{0(LSB)}$ (Bit 1) |
|---|---|---|---|
| Initially | 0 | 0 | 0 |
| 1 | 0 | 0 | 1 |
| 2 | 0 | 1 | 0 |
| 3 | 0 | 1 | 1 |
| 4 | 1 | 0 | 0 |
| 5 | 1 | 0 | 1 |
| 6 | 1 | 1 | 0 |
| 7 | 1 | 1 | 1 |
| 8 | 0 | 0 | 0 |

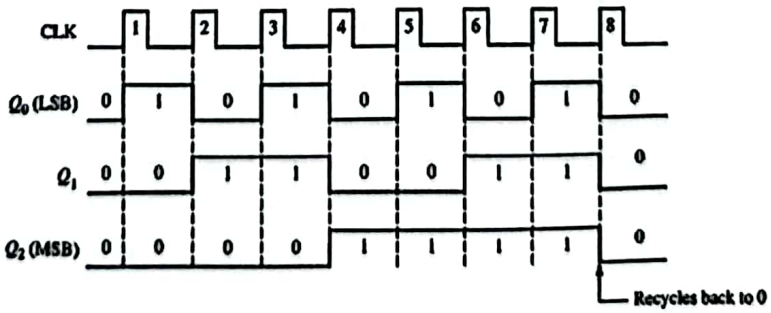
**3 bit asynchronous count up counter circuit**

- The circuit connection is the same as 2 bit asynchronous counter, for a 3 bit counter just add another flip-flop.




**The timing diagram of the 3 bit asynchronous counter.**

- The count change every positive edge of a clock.
- The counting sequence is 0, 1, 2, 3, 4, 5, 6, 7 and recycle back to 0

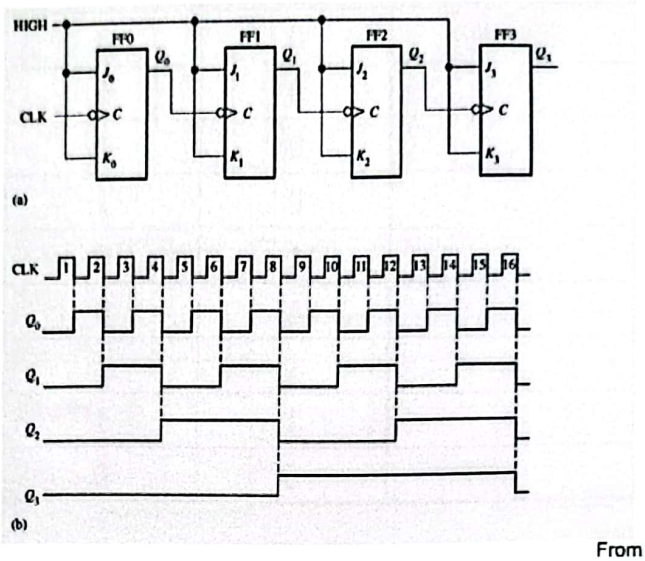





CamScanner

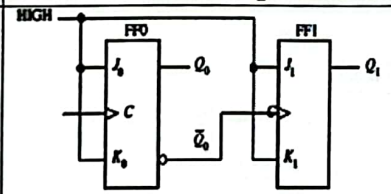
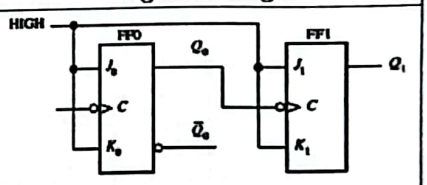
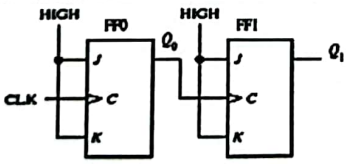
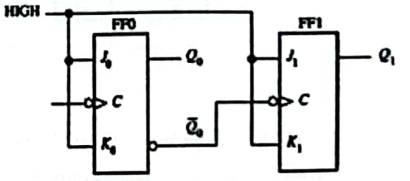
# 4-Bit Asynchronous Counter

- The counter is using a negative edge JK FF, but it is still a count up counter because clock is connected to Q.




From Floyd

Generalization of an Asynchronous Counter

| | Positive Edge | Negative Edge |
|---|---|---|
| Count Up |  |  |
| Count Down |  |  |

We can summaries as follows:

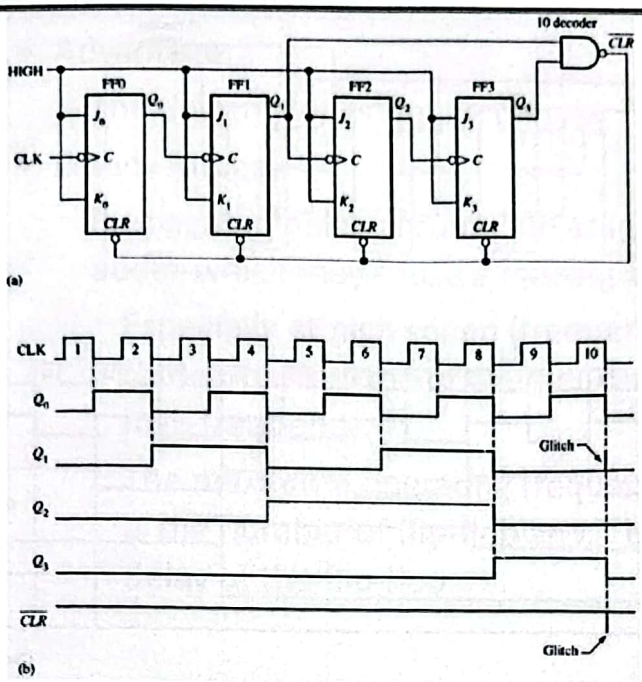
1. Basically, the circuit connection for AC is almost the same where the external clock connected to the LSB Flip Flop (FF).
2. Clock for each FF come from the previous FF, except for the first FF (that is FF0)
3. Every FF operates in Toggle mode (i.e. next output is a complement of the previous output).
   - To operate in toggle mod: **for RS:** R = Q' and S = Q, **for JK:** J = K = 1, **for D:** D = Q', **for T:** T = 1
4. The design connection is the same , if we want more bits just add more flip-flops.
5. The difference in connection will be determine whether we want to count UP or DOWN and by using which type of flip-flop – positive edge or negative edge.



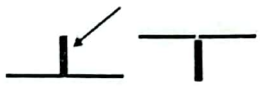
CamScanner

# Asynchronous Decade Counter






Notice that there is a glitch in Q1. The reason of this glitch is that Q1 must first go HIGH before the count 10 can be decoded. Several nanoseconds after the decoding gate goes LOW.



Previously we examined a binary counter that progresses through all of its possible states. Recall that the maximum number of possible states (maximum modulus) of a counter is $2^n$, where n is the number of flip-flops in the counter.

Counters can be designed to have a number of states in their sequence that is less than $2^n$. The resulting sequence is called a *truncated sequence*. One common modulus for counters with truncated sequences is ten. Such counters are referred to as *decade* counters. Furthermore, a counter with count a sequence of zero (0000) through nine (10001) is referred to as a *BCD decade* counter because its ten state sequence is the BCD code.

To obtain a truncated sequence it is necessary to force the counter to cycle before going through all of its normal states. For example, the BCD decade counter must recycle back to the state 0000 after state 1001. Note that a decade counter will require 4 flip-flops (three flip-flops are insufficient because $2^3 = 8$).

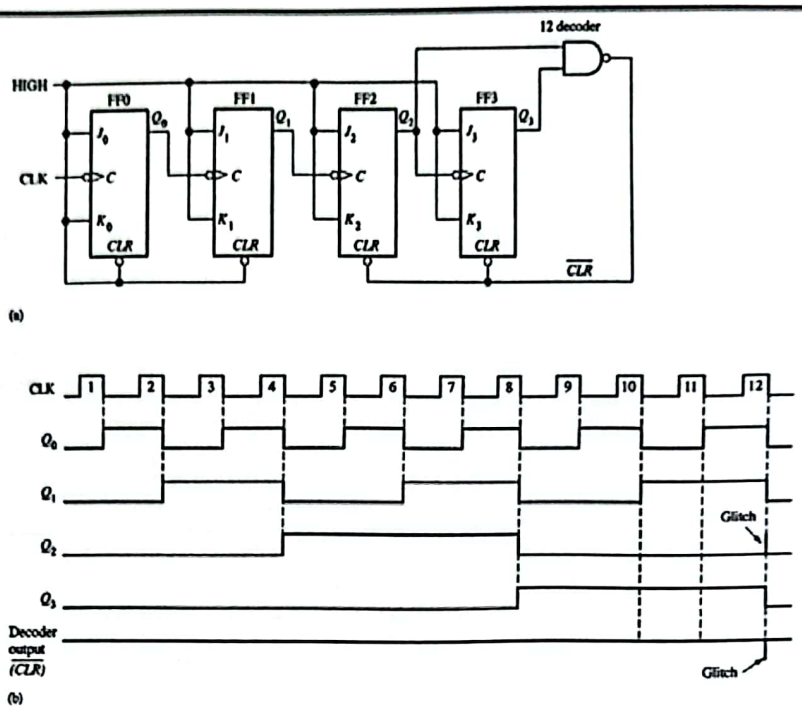
One method that can be used to achieve recycling after the count of nine (1001) is to decode ten (1010) with a NAND gate and connect the output of the NAND gate to the clear inputs of all the flip-flops.

Notice that only $Q_1$ and $Q_3$ are connected to the NAND gate inputs. This arrangement is an example of *partial decoding*, in which two unique states ($Q_1$=1 and $Q_3$=1) are enough to decode the count of 10. This is because none of the other states (zero through nine) have $Q_1$ and $Q_3$ high at the same time. Consequently, when the counter goes into count ten (1010) the decoding gate output goes LOW and asynchronously RESETS all of the flip-flips.



CamScanner

# Asynchronously Clocked Modulus-12




**Design of an Asynchronous Counter**

**Step 1:** Determine number of flip-flop required → $\text{Number of flip-flops} = \left\lceil \frac{\log \text{MOD}}{\log 2} \right\rceil$

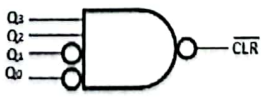
**Step 2:** Make a circuit connection

- connect the circuit as ***in Generalization of an Asynchronous Counter***
- The circuit will depend whether count up or down and positive or negative edge triggered flip-flop is used.

*Step 1 and 2 is enough for max MOD asynchronous counter, but for a truncated counter an extra step is required*

**Step 3:** Design a decoder to decode the MOD of the truncated sequence

- Use a NAND gate, at the input connect the binary equivalent of the MOD number
  - ✓ E.g. : MOD 12 , need to clear all flip-flop when output =12 ($Q_3Q_2Q_1Q_0$ = 1100)




  - ✓ There is a case where we don't need to decode all the output if there is a unique bit pattern
    - E.g. for MOD 12, need to use only $Q_3Q_2$ because the counting sequence is 0 to 11 where there is no $Q_3Q_2$ = 11 in that sequence.

- The output of the NAND gate is connected to all the $\overline{\text{CLR}}$ (CLEAR) input of the flip-flop.



CamScanner

# Advantage and Disadvantage of AC (1)



- Advantage:
  - The design step is simple (easy)
- Disadvantage:
  - Propagated delay accumulate as in ripple binary adder which may cause a missing counting state
    - Especially at high speed (frequency) operation, therefore this kind of counter cannot operates at high frequency.
    - The maximum operating frequency $f_{max} = \frac{1}{Nt_{pd}}$, where N is the number of flip-flop an $t_{pd}$ is the propagation delay of the flip-flop

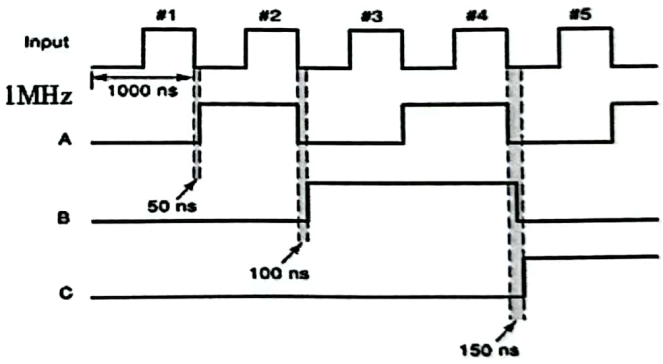
**Effect of propagation delay on AC**

Assume 3 bits counter using a flip-flop with a propagation delay of 50ns.

Case 1 :

Operating at low frequency of 1 MHz.

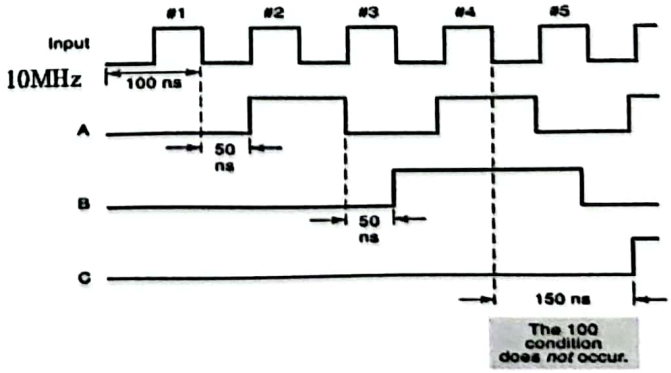
For each stage there is a 50 ns delay and it accumulates up to 150ns at the 3rd stage. The accumulated delay is still lower than the period of the signal which is 1000ns , therefore there is no effect to the counting sequence.




Case 2:

Operating at a higher frequency of 10 MHz.

The clock period is only 100ns. For a 3 stage flip-flop the accumulated delay is the same as before which is 150ns. But, now the accumulated delay is more than the clock period, therefore there is a missing count, 100 doesn't exist!






CamScanner

# Advantage and Disadvantage of AC (2)



**Example**:

If a propagation delay of a mod 32 asynchronous recycle UP counter using JK FF is 40ns, what is the maximum operating frequency?

**Solution**: Number of flip-flops $= \left\lceil \frac{\log \text{MOD}}{\log 2} \right\rceil$

$= \frac{\log 32}{\log 2}$

$= \frac{\log 2^5}{\log 2}$

$= \frac{5 \log 2}{\log 2}$

$N = 5$

$f_{max} = \frac{1}{Nt_{pd}}$

$= \frac{1}{5(40\text{ns})}$

$= \frac{1}{5(40\text{x}10^{-9}\text{s})}$

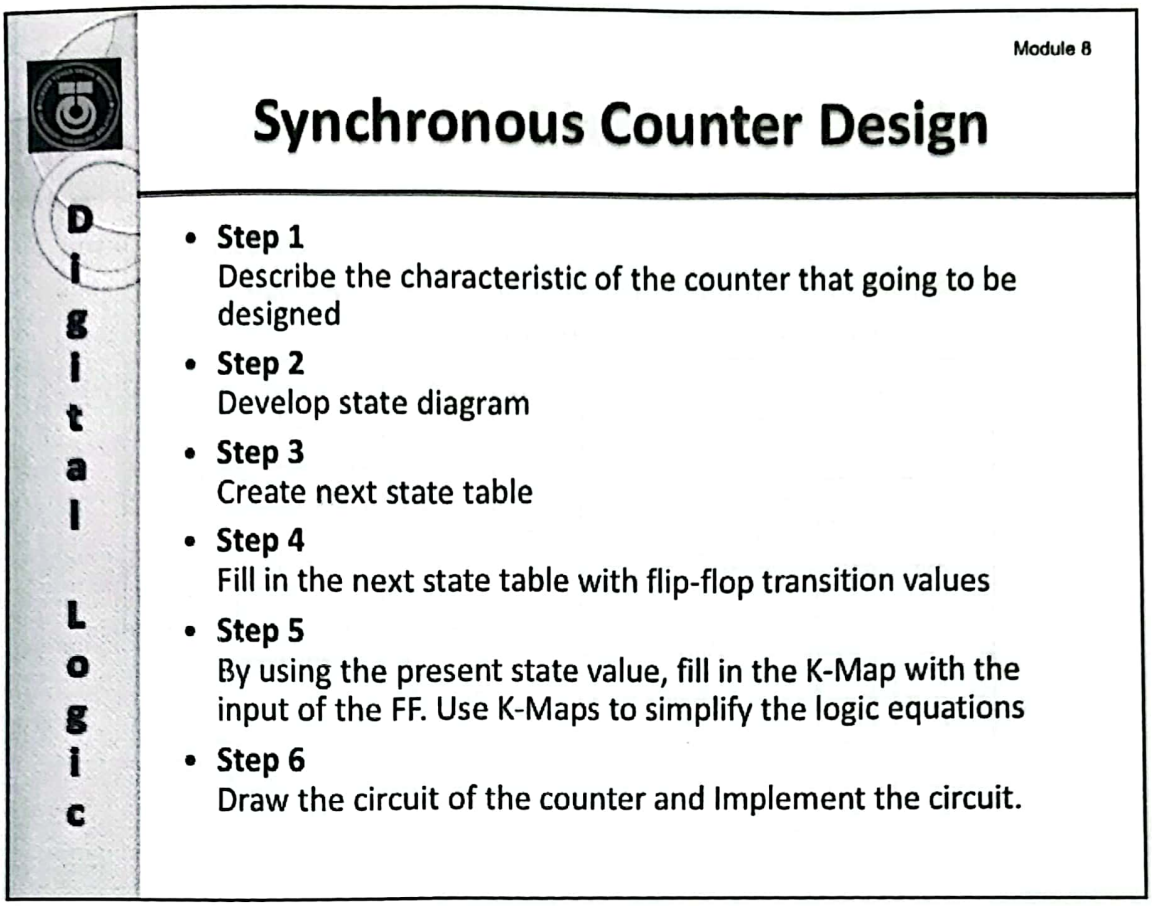
$f_{max} = 5$ MHz

**DIY:**

Design a mod 64 Asynchronous recycle UP counter using JK FF. If propagation delay is 30 ms, what is the maximum operating frequency?



CamScanner




The design of a synchronous sequential circuit starts from a set of specifications and culminates in a logic diagram or a list of Boolean functions from which a logic diagram can be obtained. In contrast to a combinational logic, which is fully specified by a truth table, a sequential circuit requires a state table for its specification. The first step in the design of sequential circuits is to obtain a state table or an equivalence representation, such as a state diagram.

A synchronous sequential circuit is made up of flip-flops and combinational gates. The design of the circuit consists of choosing the flip-flops and then finding the combinational structure which, together with the flip-flops, produces a circuit that fulfils the required specifications. The number of flip-flops is determined from the number of states needed in the circuit.



CamScanner

# F/F Excitation Table

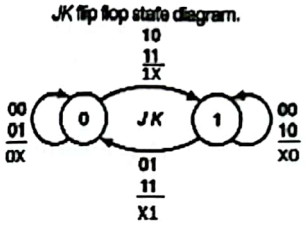
- While designing synchronous counter excitation table is needed as in Step 4.
- Following is an excitation table for JK Flip-Flop

| FF State | | Present State | Next State |
|---|---|---|---|
| J | K | $Q_n$ | $Q_{n+1}$ |
| 0 | 0 | 0 | 0 |
| 0 | 0 | 1 | 1 |
| 0 | 1 | 0 | 0 |
| 0 | 1 | 1 | 0 |
| 1 | 0 | 0 | 1 |
| 1 | 0 | 1 | 1 |
| 1 | 1 | 0 | 1 |
| 1 | 1 | 1 | 0 |

≡

| Present State | Next State | FF State | |
|---|---|---|---|
| $Q_n$ | $Q_{n+1}$ | J | K |
| 0 | 0 | 0 | X |
| 0 | 1 | 1 | X |
| 1 | 0 | X | 1 |
| 1 | 1 | X | 0 |

JK flip flop state diagram.




S-R Excitation Table

| FF State | | Present State | Next State |
|---|---|---|---|
| S | R | $Q_n$ | $Q_{n+1}$ |
| 0 | 0 | 0 | 0 |
| 0 | 0 | 1 | 1 |
| 0 | 1 | 0 | 0 |
| 0 | 1 | 1 | 0 |
| 1 | 0 | 0 | 1 |
| 1 | 0 | 1 | 1 |
| 1 | 1 | 0 | Invalid |
| 1 | 1 | 1 | invalid |

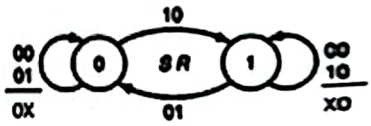
≡

| Present State | Next State | FF State | |
|---|---|---|---|
| $Q_n$ | $Q_{n+1}$ | S | R |
| 0 | 0 | 0 | X |
| 0 | 1 | 1 | 0 |
| 1 | 0 | 0 | 1 |
| 1 | 1 | X | 0 |

S-R Truth Table

| S | R | $Q_{n+1}$ |
|---|---|---|
| 0 | 0 | $Q_n$ |
| 0 | 1 | 0 |
| 1 | 0 | 1 |
| 1 | 1 | - |

SR flip flop state diagram.



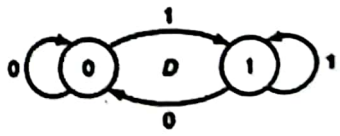

D Excitation Table

| FF State | Present State | Next State |
|---|---|---|
| D | $Q_n$ | $Q_{n+1}$ |
| 0 | 0 | 0 |
| 0 | 1 | 0 |
| 1 | 0 | 1 |
| 1 | 1 | 1 |

D Truth Table

| D | $Q_{n+1}$ |
|---|---|
| 0 | 0 |
| 1 | 1 |

D flip flop state diagram..



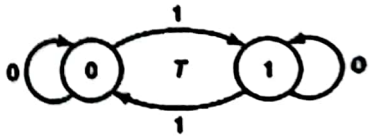

T Excitation Table

| FF State | Present State | Next State |
|---|---|---|
| T | $Q_n$ | $Q_{n+1}$ |
| 0 | 0 | 0 |
| 0 | 1 | 1 |
| 1 | 0 | 1 |
| 1 | 1 | 0 |

T Truth Table

| T | $Q_{n+1}$ |
|---|---|
| 0 | $Q_n$ |
| 1 | $\overline{Q_n}$ |

T flip flop state diagram.



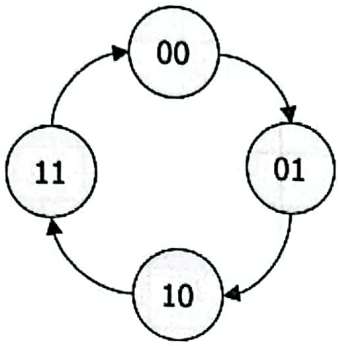



CamScanner

# Design 2-Bit Synchronous Counter

- **Step 1:**
  Design 2-bit synchronous recycle count-up counter using JK FF.

- **Step 2:** Draw state diagram




- **Step 3:** Create next state table

| Present State | | Next State | |
|---|---|---|---|
| $Q1_n$ | $Q0_n$ | $Q1_{n+1}$ | $Q0_{n+1}$ |
| 0 | 0 | 0 | 1 |
| 0 | 1 | 1 | 0 |
| 1 | 0 | 1 | 1 |
| 1 | 1 | 0 | 0 |

**Note on State Table**

A state diagram describes circuit functionality, but does not describe circuit implementation

- Translation to a tabular form is necessary
- The state table should contain
  - ✓ All transitions from each present state to each next state for all variations of the input signals
  - ✓ Example: The counter has three states A, B and C. There is one input that control the counting sequence which is W.
  - ✓ If W = 0, the state sequence are A → C → B → A → C → B → A ... recycle
  - ✓ If W = 1, the state sequence are A → B → C → A → B → C → A ... recycle

| Present state | Next state | |
|---|---|---|
| | W=0 | W=1 |
| A | C | B |
| B | A | C |
| C | B | A |



CamScanner

# ... 2-Bit Synchronous Counter

- **Step 4:** Fill in the **flip-flop transition table**

| Present State | | Next State | | JK Transition | | | |
|---|---|---|---|---|---|---|---|
| $Q1_n$ | $Q0_n$ | $Q1_{n+1}$ | $Q0_{n+1}$ | J1 | K1 | J0 | K0 |
| 0 | 0 | 0 | 1 | 0 | X | 1 | X |
| 0 | 1 | 1 | 0 | 1 | X | X | 1 |
| 1 | 0 | 1 | 1 | X | 0 | 1 | X |
| 1 | 1 | 0 | 0 | X | 1 | X | 1 |

**Note:**

- While filling in the next state table, please refer to the excitation table.

| Present State | Next State | FF State | |
|---|---|---|---|
| $Q_n$ | $Q_{n+1}$ | J | K |
| 0 | 0 | 0 | X |
| 0 | 1 | 1 | X |
| 1 | 0 | X | 1 |
| 1 | 1 | X | 0 |

- Constructing the next-state table depends on the type of flip-flop (D, T, JK) used for the implementation:
  - ✓ For JK flip-flop every transition we need to determine two value which is J and K.
  - ✓ For D and T every transition we need to determine only a single value for D or T.



CamScanner

# ... 2-Bit Synchronous Counter



- **Step 5:** Use K-maps to simplify the logic equations.

| Q1 \ Q0 | 0 | 1 |
|---|---|---|
| 0 | 1 | X |
| 1 | 1 | X |

J0 = 1

| Q1 \ Q0 | 0 | 1 |
|---|---|---|
| 0 | 0 | 1 |
| 1 | X | X |

J1 = Q0

| Q1 \ Q0 | 0 | 1 |
|---|---|---|
| 0 | X | 1 |
| 1 | X | 1 |

K0 = 1

| Q1 \ Q0 | 0 | 1 |
|---|---|---|
| 0 | X | X |
| 1 | 0 | 1 |

K1 = Q0

Note:

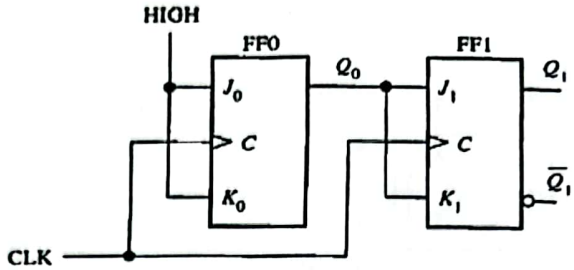
- K-maps are constructed from the state table to simplify the circuit and determine input equation for the FF.
- While mapping to the K-Map, refer to the Present state column not the Next state column of the state table.
- If the implementation is using a JK flip-flop there will be twice the number of K-Maps compared to using a D or T flip-flop, this is because JK flip-flop has two input while the D and T flip-flop has only a single input.



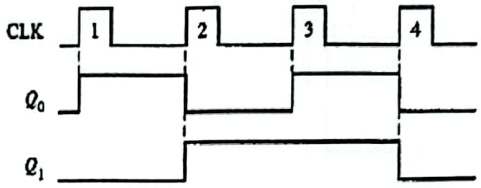
CamScanner

# ... 2-Bit Synchronous Counter

- **Step 6:** Draw the circuit of the counter and Implement the circuit.




- The timing diagram of the implemented the circuit that shows the counting sequence from 0, 1, 2, 3, 0 ( recycle)




- From the timing diagram, we can read the output of the counter as $Q_1Q_0$

| | | | |
|---|---|---|---|
| initial value | - 00 | which is 0 | } has 4 unique state (MOD 4) |
| during Clock 1 | - 01 | which is 1 | |
| during Clock 2 | - 10 | which is 2 | |
| during Clock 3 | - 11 | which is 3 | |
| during Clock 4 | - 00 | which is 0 -- it repeat again (recycle) | |

- Note that
  - ✓ the changes of state occurs at the positive edge of the clock
  - ✓ $Q_0$ always change/TOGGLE every positive edge of the clock when $J_0 = K_0 = 1$ (HIGH)
  - ✓ $Q_1$ will only change/TOGGLE if $Q_0$ before the positive edge is 1 (HIGH), because this will make $J_1 = K_1 = 1$ (HIGH)

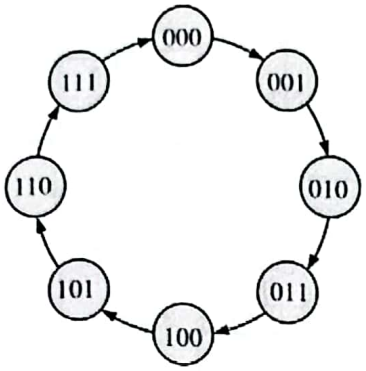


CamScanner

# Design 3-bit Synchronous Counter

Digital Logic

- State diagram




- Next State table

Note: + mean next state

| Present State | | | Next State | | |
|---|---|---|---|---|---|
| Q2 | Q1 | Q0 | $Q2_+$ | $Q1_+$ | $Q0_+$ |
| 0 | 0 | 0 | 0 | 0 | 1 |
| 0 | 0 | 1 | 0 | 1 | 0 |
| 0 | 1 | 0 | 0 | 1 | 1 |
| 0 | 1 | 1 | 1 | 0 | 0 |
| 1 | 0 | 0 | 1 | 0 | 1 |
| 1 | 0 | 1 | 1 | 1 | 0 |
| 1 | 1 | 0 | 1 | 1 | 1 |
| 1 | 1 | 1 | 0 | 0 | 0 |

Next State Transition Table

| Present State | | | Next State | | | JK Transition Table | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|
| Q2 | Q1 | Q0 | $Q2_+$ | $Q1_+$ | $Q0_+$ | J2 | K2 | J1 | K1 | J0 | K0 |
| 0 | 0 | 0 | 0 | 0 | 1 | 0 | X | 0 | X | 1 | X |
| 0 | 0 | 1 | 0 | 1 | 0 | 0 | X | 1 | X | X | 1 |
| 0 | 1 | 0 | 0 | 1 | 1 | 0 | X | X | 0 | 1 | X |
| 0 | 1 | 1 | 1 | 0 | 0 | 1 | X | X | 1 | X | 1 |
| 1 | 0 | 0 | 1 | 0 | 1 | X | 0 | 0 | X | 1 | X |
| 1 | 0 | 1 | 1 | 1 | 0 | X | 0 | 1 | X | X | 1 |
| 1 | 1 | 0 | 1 | 1 | 1 | X | 0 | X | 0 | 1 | X |
| 1 | 1 | 1 | 0 | 0 | 0 | X | 1 | X | 1 | X | 1 |



CamScanner

# ... 3-bit Synchronous Counter

- Create K-map to determine the Boolean expression

| Q2 \ Q1Q0 | 00 | 01 | 11 | 10 |
|---|---|---|---|---|
| 0 | 0 | 0 | 1 | 0 |
| 1 | X | X | X | X |

J2 = Q1Q0

| Q2 \ Q1Q0 | 00 | 01 | 11 | 10 |
|---|---|---|---|---|
| 0 | 0 | 1 | X | X |
| 1 | 0 | 1 | X | X |

J1 =

| Q2 \ Q1Q0 | 00 | 01 | 11 | 10 |
|---|---|---|---|---|
| 0 | 1 | X | X | 1 |
| 1 | 1 | X | X | 1 |

J0 =

**DIY:** Fill in the K-map and get the simplified K inputs of each JK FF.

| Q2 \ Q1Q0 | 00 | 01 | 11 | 10 |
|---|---|---|---|---|
| 0 | X | X | X | X |
| 1 | 0 | 0 | 1 | 0 |

K2 =

| Q2 \ Q1Q0 | 00 | 01 | 11 | 10 |
|---|---|---|---|---|
| 0 | X | X | 1 | 0 |
| 1 | X | X | 1 | 0 |

K1 =

| Q2 \ Q1Q0 | 00 | 01 | 11 | 10 |
|---|---|---|---|---|
| 0 | X | 1 | 1 | X |
| 1 | X | 1 | 1 | X |

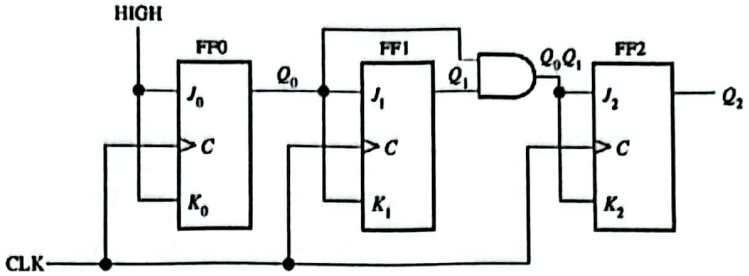
K0 =



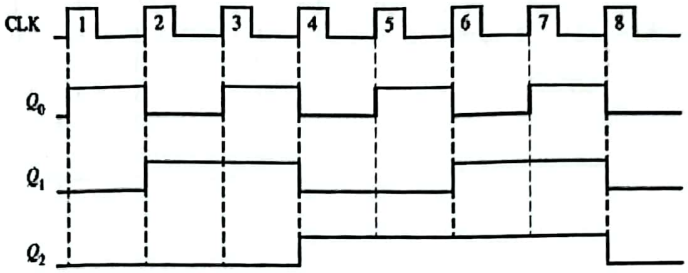
CamScanner

# ... 3-bit Synchronous Counter

- The circuit diagram of 3-bit synchronous counter




- The timing diagram of 3-bit synchronous counter




**Note:**

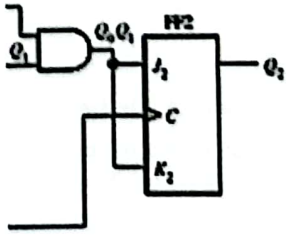
Actually we can design a synchronous counter by observing its timing diagram, based on the above timing diagram:

- $Q_0$ always toggle. To achieve this, we have to operate FF0 in toggle mode by connecting $J_0$ and $K_0$ to HIGH
- $Q_1$ goes to opposite state every time $Q_0$ is HIGH (at the positive edge of clock CLK2, CLK4,CLK6 and CLK8).
  - ✓ From the observation we can achieve this by connecting $Q_0$ to the $J_1$ and $K_1$ inputs of FF1
- $Q_2$ changes state when both $Q_0$ and $Q_1$ is HIGH. (at the positive edge of clock CLK4 and CLK8).
  - ✓ From the observation we can achieve this by 'AND'ing $Q_0$ and $Q_1$to the $J_2$ and $K_2$ inputs of FF2

| FF2 | FF1 | FF0 | |
|---|---|---|---|
| 0 | 0 | 0 | ← |
| 0 | 0 | 1 | |
| 0 | 1 | 0 | |
| 0 | 1 | 1 | |
| 1 | 0 | 0 | |
| 1 | 0 | 1 | |
| 1 | 1 | 0 | |
| 1 | 1 | 1 | (recycles) |

- The most important advantage of synchronous counters is that there is no cumulative time delay because all flip-flops are triggered in parallel. Thus, the maximum operating frequency for this counter will be significantly higher than for the corresponding ripple counter.

- The maximum operating frequency, $f_{max}$ for the above circuit is only limited by the summation of the AND gate and flip-flop propagation delay




$$f_{max} = \frac{1}{\text{total gate delay} + \text{delay of a single flip-flop}}$$

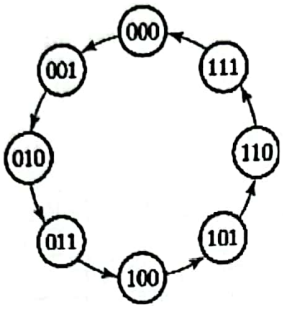


CamScanner

# 3-Bit Synchronous Counter

Design a 3-bit synchronous recycle up counter using T FF.

State Diagram




Next State Table

| Present State | | | Next State | | | Flip-Flop Inputs | | |
|---|---|---|---|---|---|---|---|---|
| $A_2$ | $A_1$ | $A_0$ | $A_2$ | $A_1$ | $A_0$ | $T_{A2}$ | $T_{A1}$ | $T_{A0}$ |
| 0 | 0 | 0 | 0 | 0 | 1 | 0 | 0 | 1 |
| 0 | 0 | 1 | 0 | 1 | 0 | 0 | 1 | 1 |
| 0 | 1 | 0 | 0 | 1 | 1 | 0 | 0 | 1 |
| 0 | 1 | 1 | 1 | 0 | 0 | 1 | 1 | 1 |
| 1 | 0 | 0 | 1 | 0 | 1 | 0 | 0 | 1 |
| 1 | 0 | 1 | 1 | 1 | 0 | 0 | 1 | 1 |
| 1 | 1 | 0 | 1 | 1 | 1 | 0 | 0 | 1 |
| 1 | 1 | 1 | 0 | 0 | 0 | 1 | 1 | 1 |

K-map for $T_{A0}$, $T_{A1}$ and $T_{A2}$.

| $A_2$ \ $A_1A_0$ | 00 | 01 | 11 | 10 |
|---|---|---|---|---|
| 0 | | | 1 | |
| 1 | | | 1 | |

$$T_{A2} = A_1A_0$$

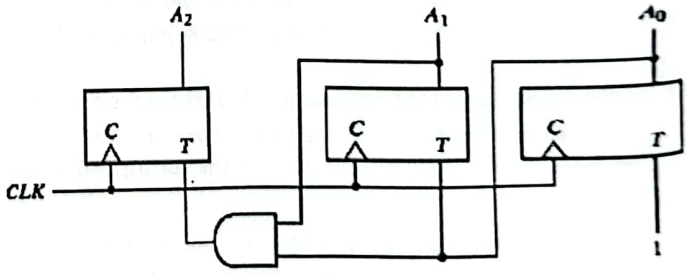
| $A_2$ \ $A_1A_0$ | 00 | 01 | 11 | 10 |
|---|---|---|---|---|
| 0 | | 1 | 1 | |
| 1 | | 1 | 1 | |

$$T_{A1} = A_0$$

| $A_2$ \ $A_1A_0$ | 00 | 01 | 11 | 10 |
|---|---|---|---|---|
| 0 | 1 | 1 | 1 | 1 |
| 1 | 1 | 1 | 1 | 1 |

$$T_{A0} = 1$$

The circuit diagram for 3-bit synchronous counter: $T_{A0}$, $T_{A1}$ and $T_{A2}$




In this example, the K-Map and the circuit diagram is different from the previous examples.

The circuit diagram is drawn in such a way that the MSB is on the left side of the circuit. So that, it can naturally be read with a proper value. (i.e. MSB .....LSB).



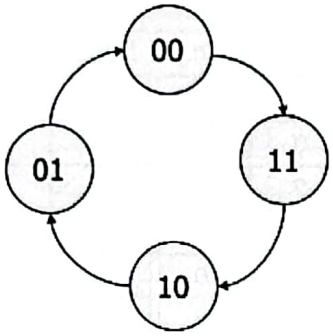
CamScanner

# 2-Bit Down Synchronous Counter



- **Step 1:**
  To design 2-bit down recycle synchronous counter using D FF.

- **Step 2:** Draw state diagram



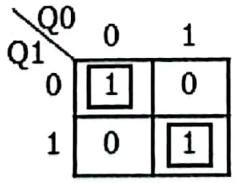

- **Step 3:** Create next state table

| Present State | | Next State | |
|---|---|---|---|
| $Q1_n$ | $Q0_n$ | $Q1_{n+1}$ | $Q0_{n+1}$ |
| 0 | 0 | 1 | 1 |
| 0 | 1 | 0 | 0 |
| 1 | 0 | 0 | 1 |
| 1 | 1 | 1 | 0 |

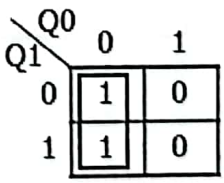
**Step 4:** Draw flip-flop transition table. For D FF, D input is the same as the next state values.

| Present State | | Next State | | D Transition | |
|---|---|---|---|---|---|
| $Q1_n$ | $Q0_n$ | $Q1_{n+1}$ | $Q0_{n+1}$ | D1 | D0 |
| 0 | 0 | 1 | 1 | 1 | 1 |
| 0 | 1 | 0 | 0 | 0 | 0 |
| 1 | 0 | 0 | 1 | 0 | 1 |
| 1 | 1 | 1 | 0 | 1 | 0 |

**Step 5:** Use the K-Map to simplify the equation




D1 = Q1'Q0' + Q1Q0




D0 = Q0'

**DIY:**

**Step 6:** Draw the circuit

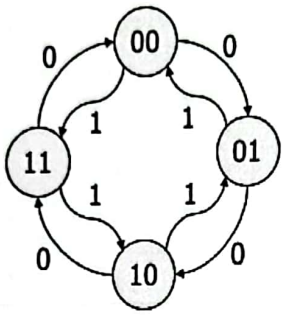


CamScanner

# Up-Down Synchronous Counters



- **Step 1:**
  To design 2-bit up down synchronous counter using D FF based on input mode selector X. When X = 0, it is up counter and down when X = 1.
- **Step 2:** Given the state diagram




- **Step 3:** Create next state table

| | Input, X | Present State | | Next State | |
|---|---|---|---|---|---|
| | | $Q1_n$ | $Q0_n$ | $Q1_{n+1}$ | $Q0_{n+1}$ |
| Count up | 0 | 0 | 0 | 0 | 1 |
| | 0 | 0 | 1 | 1 | 0 |
| | 0 | 1 | 0 | 1 | 1 |
| | 0 | 1 | 1 | 0 | 0 |
| Count down | 1 | 0 | 0 | 1 | 1 |
| | 1 | 0 | 1 | 0 | 0 |
| | 1 | 1 | 0 | 0 | 1 |
| | 1 | 1 | 1 | 1 | 0 |

- Up–down counter is a bidirectional counter that can counts both up and down
  - ✓ The count direction is controlled by an input , as X in the above example
    - If X =1 , the counter will count down
    - If X = 0, the counter will count up
  - ✓ During the operation, the value of X can be changed, and the effect of changing the value will occur at the next clock edge



CamScanner

# .. Up-Down Synchronous Counters



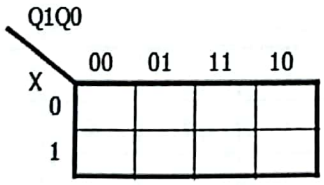
- **Step 4 and 5**:
  Fill in the D1 and D0 by referring to D FF transition table and determine Boolean expression.

| Input, X | Present State | | Next State | | D FF | |
|---|---|---|---|---|---|---|
| | $Q1_n$ | $Q0_n$ | $Q1_{n+1}$ | $Q0_{n+1}$ | D1 | D0 |
| 0 | 0 | 0 | 0 | 1 | 0 | 1 |
| 0 | 0 | 1 | 1 | 0 | 1 | 0 |
| 0 | 1 | 0 | 1 | 1 | 1 | 1 |
| 0 | 1 | 1 | 0 | 0 | 0 | 0 |
| 1 | 0 | 0 | 1 | 1 | 1 | 1 |
| 1 | 0 | 1 | 0 | 0 | 0 | 0 |
| 1 | 1 | 0 | 0 | 1 | 0 | 1 |
| 1 | 1 | 1 | 1 | 0 | 1 | 0 |

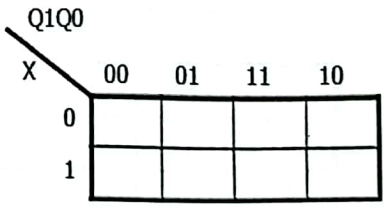
- When mapping to the K-Map , X is one of the input besides the value of Q1and Q0 from the present state column

**DIY:**

Fill in the K-Map and determine the simplified expression for D1 and D0.




D1 =




D0 =

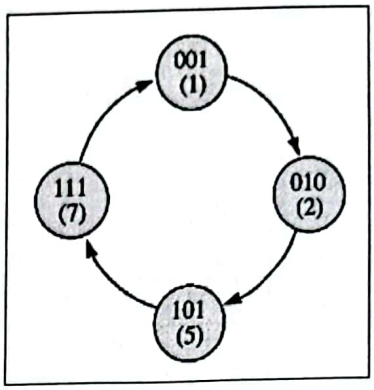
- **Step 6**: Draw the circuit diagram for the 2-bit up-down synchronous counter



CamScanner

# Counter for Arbitrary Sequences

- Design the counter base on the given state diagram using T FF.




From the state diagram, the counter is a 3 bit counter.

For a 3 bits counter, the number of states should be $2^3$ which is 8.

But, in the state diagram there are only 4 states (called **valid state**).

The other 4 states which is 0,3,4 and 6 that does not appear in the state diagram is called the **invalid state**.

Usually we assume an invalid state will never occurs and treat it as don't care, and plotted as X in the next state table as in the first row of the table below. The input to the FF also considered as a don't care.

State table and T FF transition table.

| Present State | | | Next State | | | T FF | | |
|---|---|---|---|---|---|---|---|---|
| $Q2_n$ | $Q1_n$ | $Q0_n$ | $Q2_{n+1}$ | $Q1_{n+1}$ | $Q0_{n+1}$ | T2 | T1 | TO |
| 0 | 0 | 0 | X | X | X | X | X | X |
| 0 | 0 | 1 | 0 | 1 | 0 | 0 | 1 | 1 |
| 0 | 1 | 0 | | | | | | |
| 0 | 1 | 1 | | | | | | |
| 1 | 0 | 0 | | | | | | |
| 1 | 0 | 1 | | | | | | |
| 1 | 1 | 0 | | | | | | |
| 1 | 1 | 1 | | | | | | |

**DIY :** Derive the Boolean expression and draw the circuit diagram.

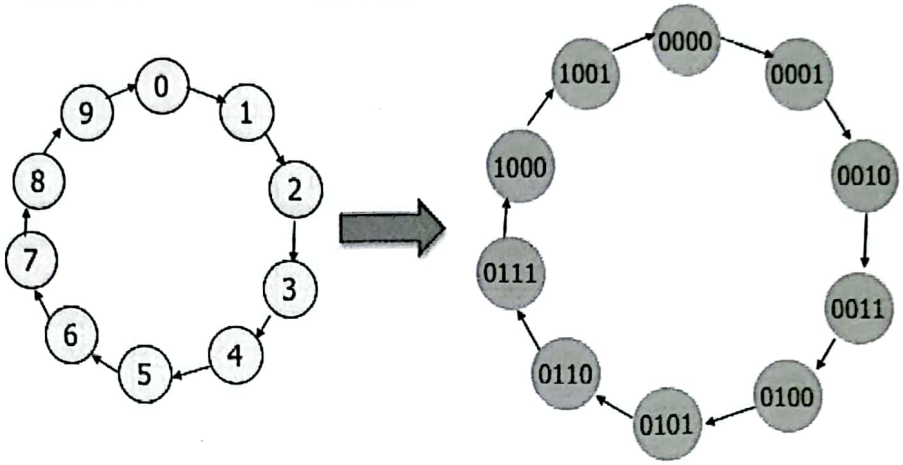


CamScanner

# Synchronous BCD Decade Counter

Digital Logic

- Synchronous decade counter counts from 0 to 9 and then recycles to 0 again.
- 4 FF is required and the unused states that is 10 to 15 are taken as don't care terms.




The maximum possible number of states of a counter is $2^n$, where $n$ is the number of flip-flops in the counter. Regular binary counters have maximum modulus; they progress through all of their possible states. Counters can also be designed to have a number of states that is less than $2^n$. The resulting sequence is called truncated sequence.

A very common modulus for counters with truncated sequences is 10. Counters with 10 states in their sequence is called decade counters.

A BCD counter or decade counter can be constructed from a binary counter by terminating the "ripple-through" counting when the count reaches decimal 9 (binary 1001).



CamScanner

# .. Synchronous BCD Decade Counter



| Present State | | | | Next State | | | | T FF Transition | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|
| Q3 | Q2 | Q1 | Q0 | $Q3_+$ | $Q2_+$ | $Q1_+$ | $Q0_+$ | T3 | T2 | T1 | T0 |
| 0 | 0 | 0 | 0 | 0 | 0 | 0 | 1 | 0 | 0 | 0 | |
| 0 | 0 | 0 | 1 | 0 | 0 | 1 | 0 | 0 | 0 | 1 | |
| 0 | 0 | 1 | 0 | 0 | 0 | 1 | 1 | 0 | 0 | 0 | |
| 0 | 0 | 1 | 1 | 0 | 1 | 0 | 0 | 0 | 1 | 1 | |
| 0 | 1 | 0 | 0 | 0 | 1 | 0 | 1 | 0 | 0 | 0 | |
| 0 | 1 | 0 | 1 | 0 | 1 | 1 | 0 | 0 | 0 | 1 | |
| 0 | 1 | 1 | 0 | 0 | 1 | 1 | 1 | 0 | 0 | 0 | |
| 0 | 1 | 1 | 1 | 1 | 0 | 0 | 0 | 1 | 1 | 1 | |
| 1 | 0 | 0 | 0 | 1 | 0 | 0 | 1 | 0 | 0 | 0 | |
| 1 | 0 | 0 | 1 | 0 | 0 | 0 | 0 | 1 | 0 | 0 | |
| 1 | 0 | 1 | 0 | X | X | X | X | X | X | X | |
| 1 | 0 | 1 | 1 | X | X | X | X | X | X | X | |
| 1 | 1 | 0 | 0 | X | X | X | X | X | X | X | |
| 1 | 1 | 0 | 1 | X | X | X | X | X | X | X | |
| 1 | 1 | 1 | 0 | X | X | X | X | X | X | X | |
| 1 | 1 | 1 | 1 | X | X | X | X | X | X | X | |

**DIY:** Fill in column T0 and the K-maps

| Q3Q2 \ Q1Q0 | 00 | 01 | 11 | 10 |
|---|---|---|---|---|
| 00 | 0 | 0 | 0 | 0 |
| 01 | 0 | 0 | 1 | 0 |
| 11 | X | X | X | X |
| 10 | 0 | 0 | X | X |

T3 =

| Q3Q2 \ Q1Q0 | 00 | 01 | 11 | 10 |
|---|---|---|---|---|
| 00 | 0 | 0 | 1 | 0 |
| 01 | 0 | 0 | 1 | 0 |
| 11 | X | X | X | X |
| 10 | 0 | 0 | X | X |

T2 =

| Q3Q2 \ Q1Q0 | 00 | 01 | 11 | 10 |
|---|---|---|---|---|
| 00 | 0 | 1 | 1 | 0 |
| 01 | 0 | 1 | 1 | 0 |
| 11 | X | X | X | X |
| 10 | 0 | 0 | X | X |

T1 =

| Q3Q2 \ Q1Q0 | 00 | 01 | 11 | 10 |
|---|---|---|---|---|
| 00 | 1 | 1 | 1 | 1 |
| 01 | 1 | 1 | 1 | 1 |
| 11 | X | X | X | X |
| 10 | 1 | 1 | X | X |

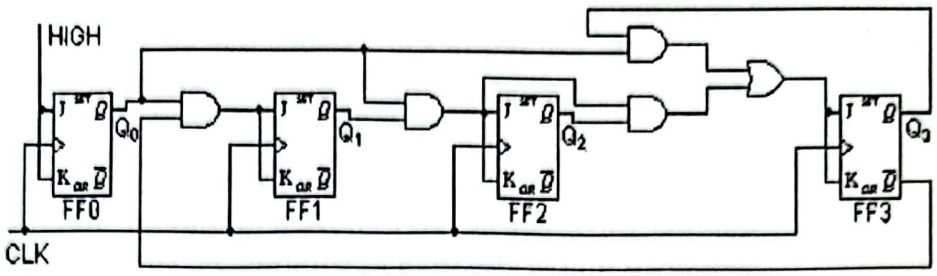
T0 =



CamScanner

# .. Synchronous BCD Decade Counter

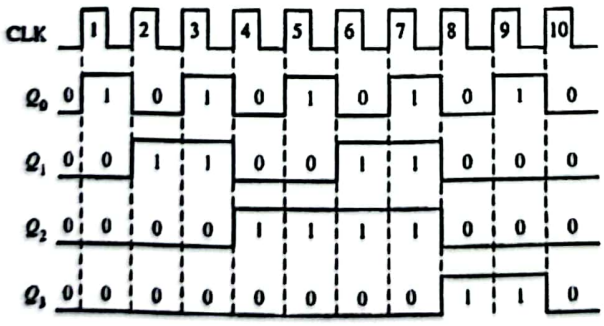





- $Q_0$ toggles on each clock pulse.
- $Q_1$ changes on the next clock pulse each time $Q_0$=1 and $Q_3$=0.
- $Q_2$ changes on the next clock pulse each time $Q_0$=$Q_1$=1.
- $Q_3$ changes on the next clock pulse each time $Q_0$=1, $Q_1$=1 and $Q_2$=1 (count 7), or then $Q_0$=1 and $Q_3$=1 (count 9).

- The circuit above is using a JK flip-flop. But each flip-flop J is connected to K so that J = K which is equivalent to a T flip-flop.
- An exercise from the previous slide ask you to implement the circuit using T flip-flop, Do you get the same answer?

- The timing diagram of the above circuit




| Clock Pulse | Q3 | Q2 | Q1 | Q0 |
|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 |
| 1 | 0 | 0 | 0 | 1 |
| 2 | 0 | 0 | 1 | 0 |
| 3 | 0 | 0 | 1 | 1 |
| 4 | 0 | 1 | 0 | 0 |
| 5 | 0 | 1 | 0 | 1 |
| 6 | 0 | 1 | 1 | 0 |
| 7 | 0 | 1 | 1 | 1 |
| 8 | 1 | 0 | 0 | 0 |
| 9 | 1 | 0 | 0 | 1 |

*From Floyd*



CamScanner

# Recap... Modulus

- The modulus is the number of unique states through which the counter will sequence.
- Maximum possible number of states of counter is $2^n$, $n$ is the number of flip-flops in the counter.
  - Example : Modulus 8 = $2^3$ (Need 3 flip flops)
- Counter can be designed to have a number of states in their sequence that is less than maximum, $2^n$. This is called a truncated sequence.
- One common modulus for counters in truncated sequences is ten (Modulus 10).
  - It is called BCD or decade counters (as explained in previous slides).

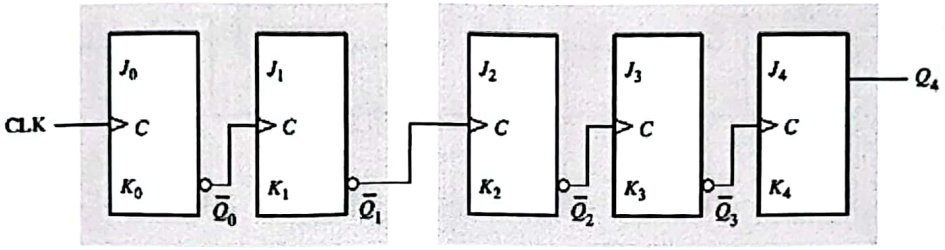


CamScanner

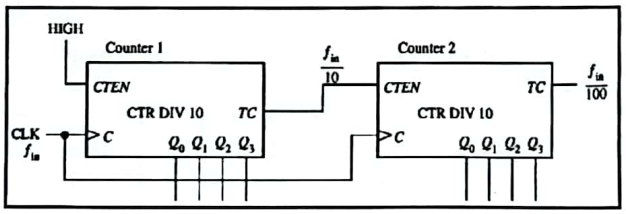
# Cascaded Counter

Digital Logic

- Counter can be connected to achieve higher modulus operation.
- Cascading means that the last-stage output of one counter drives the input of the next counter.
- Effect of cascading a counter is that it total MOD is a product of multiplying each MOD of the counter
  - Example: Two counters, modulus-4 and modulus-8 connected in cascade, will produce a MOD 32 (that is 8 x 4) counter
- Example: Two counters, modulus-4 and modulus-8 connected in cascade, can achieve count until 32 CLK (modulus-32 (that is 4 × 8)).




- Modulus-100 counter can be build by using two cascaded decade (MOD10) counters.
- This counter can be viewed as a frequency divider.
  - ✓ A counter with MOD *N* will divide the frequency by *N*
- For a modulus 100 counter It will divides the input clock frequency by 100.




- Counter can also be used as a frequency divider.

- Counter with MOD *n* will divide the input clock frequency by *n*. From the above figure, Counter 1 and Counter 2 will divide the input clock by 10. If we cascade both counter we will end up with mod 100 counter which will divide the clock ($f_{in}$) by 100.

- *CTEN = Counter Enable*

  If *CTEN* =1 then the counter will count in a proper sequence i.e. 0, 1, ..., 9 for MOD 10 counter (CTR DIV 10)

- *TC=Terminal Count*

  When the counter reaches the max count, TC will be equal to 1. This terminal is use to enable Counter 2 so that the next clock cycle will increment the count in Counter 2.

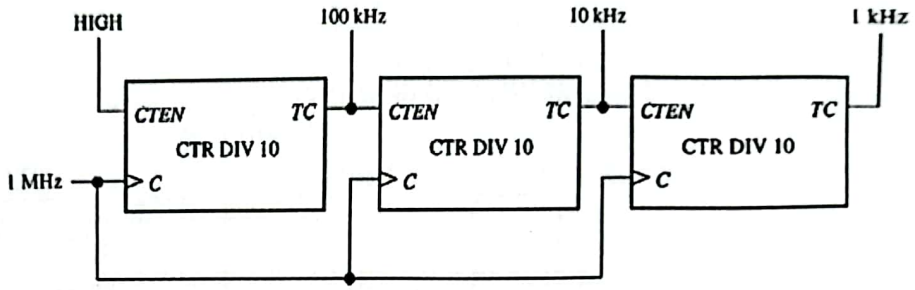


CamScanner

# Modulus-1000 Counter

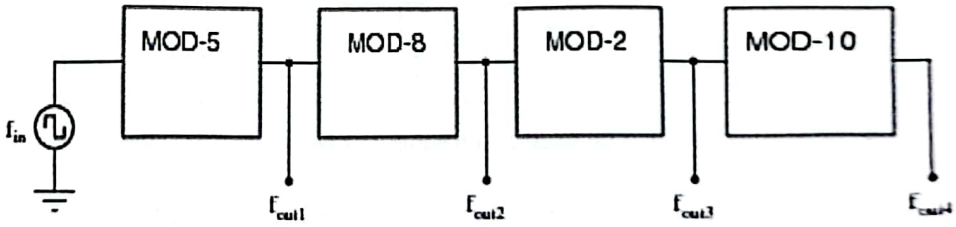
Digital Logic

- Three cascaded decade counters forming a divide-by-1000 frequency divider.
  - Total MOD = 10 x 10 x 10 =1000




- If given a clock with a frequency of 1 MHz and you wish to obtain 100KHz, 10KHz, and 1KHz, a series of cascaded decade counters can be used as in the above figure
  - Divide the 1 MHz signal by 10, the output is 100KHz. Then divide the 100 KHz signal by10, the output is 10KHz. Further division by 10 gives the 1 KHz frequency.

- When counters are used as frequency dividers, they are often drawn as simple boxes with one input and one output each, like this:




**DIY:**

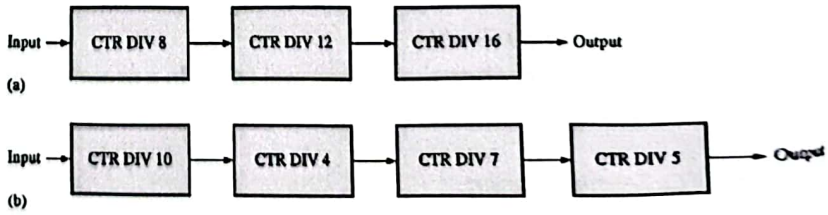
1. Calculate the four output frequencies ($f_{out1}$ through $f_{out4}$) given an input frequency of 25 kHz:

   Answer: $f_{out1}$ = 5 kHz, $f_{out2}$ = 625 Hz, $f_{out3}$ = 312.5 Hz, $f_{out4}$ = 31.25 Hz

2. Find:

   (a) Overall Modulus of (a)

   (b) Overall Modulus of (b)




   (c) How many cascaded counter are required to divide a clock frequency by 100,000?

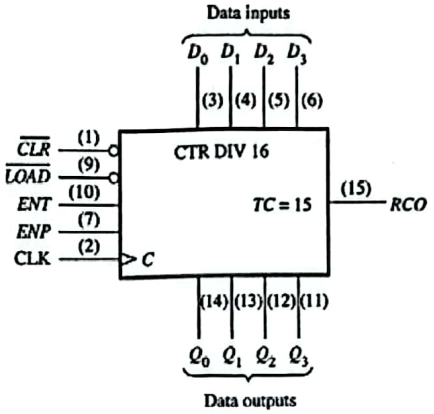


CamScanner

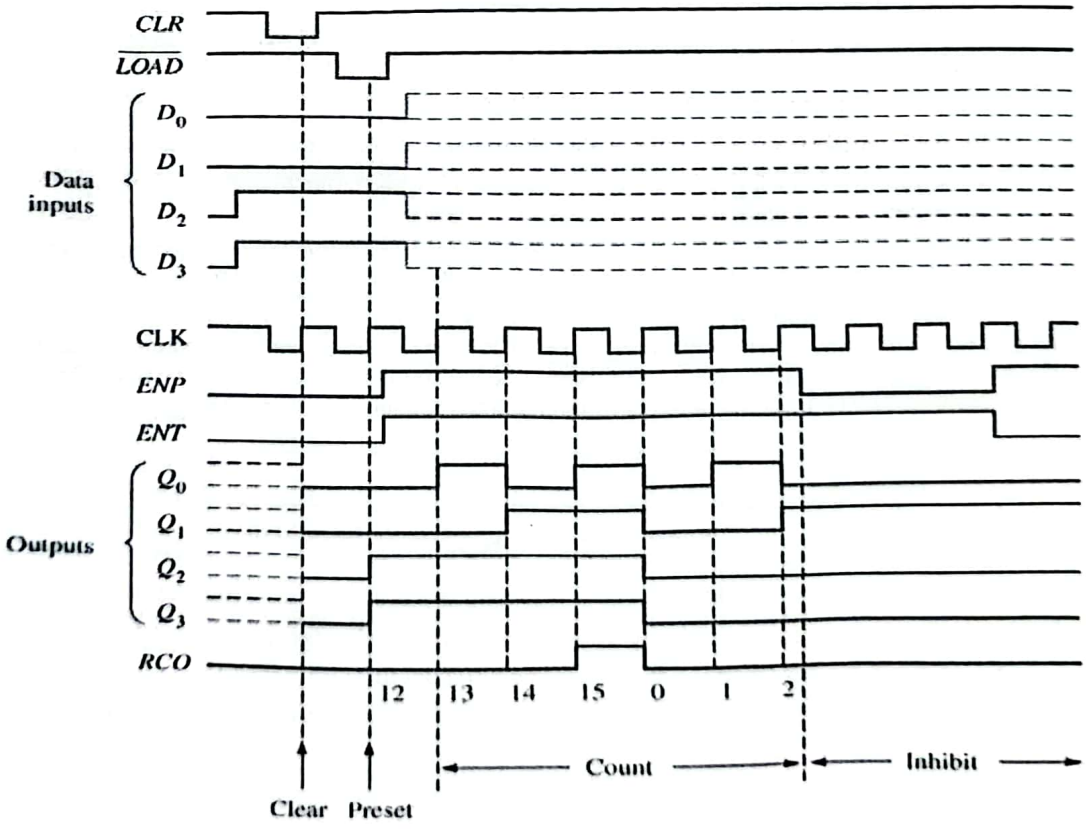
# 74HC163:
# 4-Bit Synchronous Binary Counter

Digital Logic

- There is a dedicated counter IC, e.g 74HC163 which is a MOD 16 counter IC
- The starting counting sequence can be change by setting the initial value at $D_3D_2D_1D_0$
  - To load the initial value, LOAD' must be 0




The timing diagram to highlight the function of 74HC163 pins

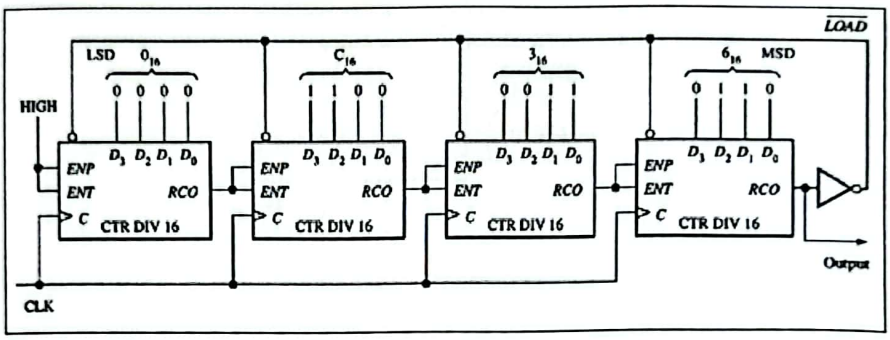




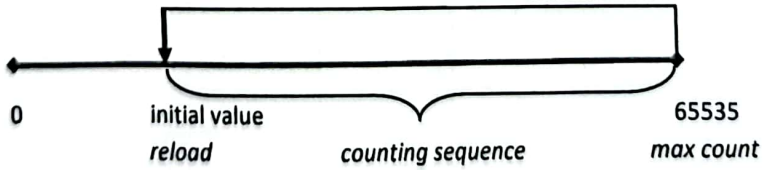

CamScanner

# Cascaded Counter with Truncated Sequence

- The cascaded counter below has $16^4$ = 65,536 states (full modulus for four cascaded CTR DIV 16).
- If we need a modulus 40,000 counter only. So, how?
  1. Determine the initial value : 65536-40000 = 25536 ($63C0_{16}$) make the counter starts at this count.
  2. Therefore preset the cascaded counter to $63C0_{16}$ by setting the value of $D_3D_2D_1D_0$ as shown below.




- An IC 74HC163 is a 4 bit synchronous binary counter,which is a MOD 16 counter
- This IC can be use to produce a bigger MOD by cascading the IC
  - ✓ If the IC is cascaded as the figure above it will produce a MOD of 16 ×16 × 16 × 16 or $16^4$ = MOD 65536.
  - ✓ It counting sequence is 0, 1, 2, 3 .... 65534, 65535
  - ✓ The ENP and ENT is an active HIGH enable pin and must be connected to HIGH to enable the IC.
  - ✓ At the LSB stage , the ENP and ENT always connected to HIGH, the other stage this input is connected to the output RCO from the previous stage.
  - ✓ RCO is a Ripple Carry Out, which will be HIGH if the counter achieve the maximum count, which is 15 for this IC. The output of RCO is use to enable the next stage so that it can increment it count when the previous stage has a maximum count
- All counter is preloaded with an initial value. This initial value will increment to the maximum count, at this point the RCO of the MSB will be 1 and inverted to become 0 and send to an active LOW LOAD' pin. This will reload the initial value to the counter and the counting sequence repeats.



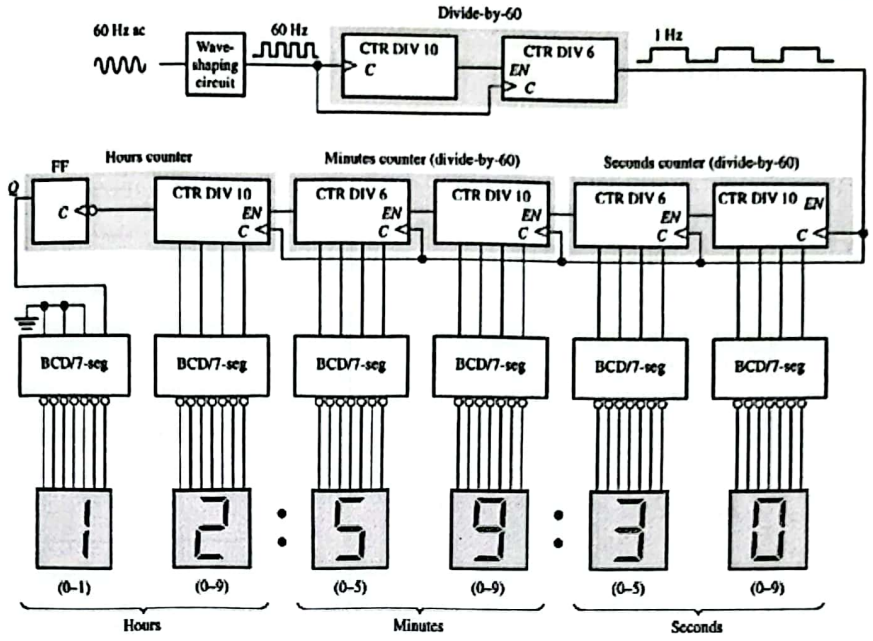

- Previously, the design of a truncated counter is by limiting the maximum count
  - ✓ Example: Decade counter – limit the maximum count to 9 rather than 15
- In the above design, the maximum count is not limited but the minimum count is varied so that the number of state (MOD) changed.



CamScanner

# Counter Applications: Digital Clock






The circuit shows one of the application of a counter in digital clock.

The source frequency of 60Hz is from the wall socket. It is not a square wave , a wave shaping circuit will convert that signal to a 60Hz square wave.

***Seconds Counter*** : The 60Hz square wave is divided by 60 by MOD 60 counter to get a 1Hz (which change every second) square wave – this will be use as a clock to a second counter.

***Minutes Counter:*** To provide a clock for a minute counter the 1 Hz square wave is divided by another cascaded counter with MOD 60 (mod 10 × mod 6).

***Hours Counter:*** The clock frequency of a minute counter is divided by another MOD 60 counter.

The clock division will make sure that the an hour will be incremented by one every 60 minutes and a minutes is incremented after every 60 seconds.

To display the value of the counter a BCD to 7 Segment decoder is used. The decoder will drive a 7-segment display and display the time.

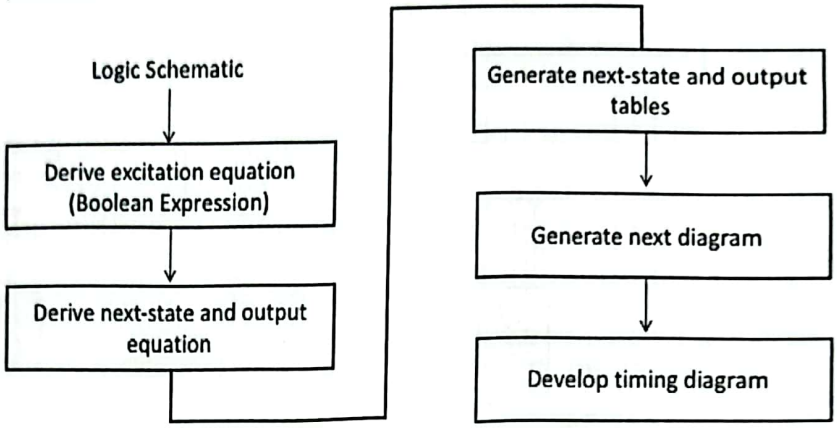


CamScanner

Digital Logic

# Analysis of Sequential Circuits

- Behavior of a sequential circuit is determined from the inputs, the outputs and the states of its flip-flops.
- Both the output and the next state are a function of the inputs and the present state.




**Step 1:**
Start with the logic schematic from which we can derive excitation equations for each flip-flop input.

**Step 2:**
To obtain next-state equations, we insert the excitation equations into the characteristic equations. The output equations can be derived from the schematic.

***Flip-flop characteristic equation:***

$$\text{active HIGH SR}: Q_{next} = S + \overline{R}Q, SR = 0$$
$$\text{JK}: Q_{next} = J\overline{Q} + \overline{K}Q$$
$$\text{D}: Q_{next} = D$$
$$\text{T}: Q_{next} = T\overline{Q} + \overline{T}Q$$

**Step 3:**
Once we have our output and next-state equations, we can generate the next-state and output tables as well as state diagrams.

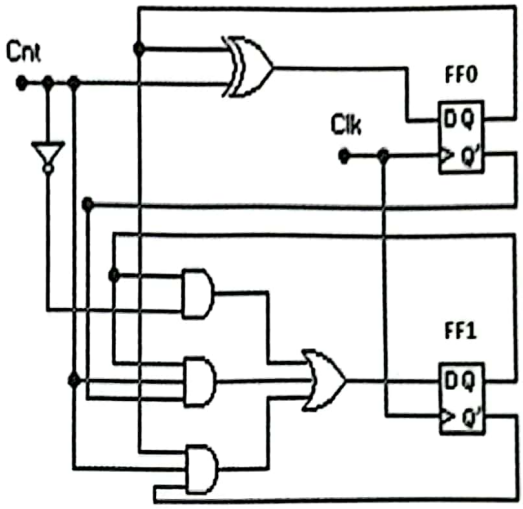
**Step 4:**
When we reach this stage, we use either the table or the state diagram to develop a timing diagram which can be verified through simulation.



CamScanner

# Method 1

- Counter is a sequential circuit. The steps in analyzing the counter can be use to analyze any sequential circuit.
- Analyze and determine the characteristic of the counter:




**Step 1:** Boolean expressions for the inputs of each flip-flops in the schematic.

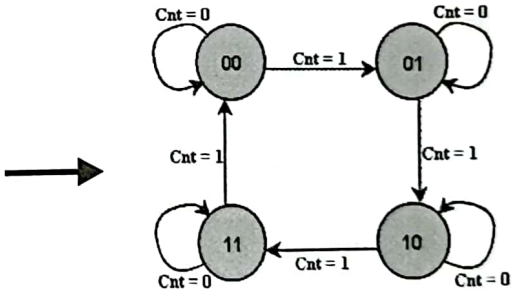
$$D0 = Cnt \oplus Q0 = \overline{Cnt} \bullet Q0 + Cnt \bullet \overline{Q0}$$
$$D1 = \overline{Cnt} \bullet Q1 + Cnt \bullet \overline{Q1} \bullet Q0 + Cnt \bullet Q1 \bullet \overline{Q0}$$

**Step 2:** Derive the next-state equations by converting these excitation equations into flip-flop characteristic equations. In the case of D flip-flops, $Q_+ = D$.

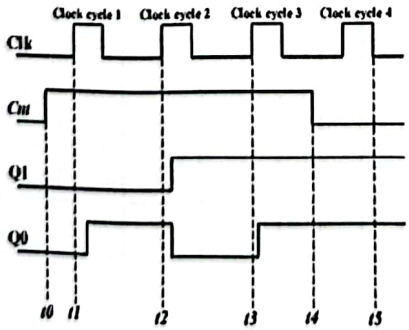
$$Q0_+ = D0 = Cnt \oplus Q0 = \overline{Cnt} \bullet Q0 + Cnt \bullet \overline{Q0}$$
$$Q1_+ = D1 = \overline{Cnt} \bullet Q1 + Cnt \bullet \overline{Q1} \bullet Q0 + Cnt \bullet Q1 \bullet \overline{Q0}$$

**Step 3:** From the equation, produce a state table and a state diagram

| Input, Cnt | Present State | | Next State | |
|---|---|---|---|---|
| | Q1 | Q0 | $Q1_+ = D1$ | $Q0_+ = D0$ |
| 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 1 | 0 | 1 |
| 0 | 1 | 0 | 1 | 0 |
| 0 | 1 | 1 | 1 | 1 |
| 1 | 0 | 0 | 0 | 1 |
| 1 | 0 | 1 | 1 | 0 |
| 1 | 1 | 0 | 1 | 1 |
| 1 | 1 | 1 | 0 | 0 |




**Step 4:** Determine the timing diagram




**We can conclude:**

a) From the counter circuit:
- 2- bit counter because there are two FFs in the design.
- Sycnhronous counter because the FFs have a common clock.

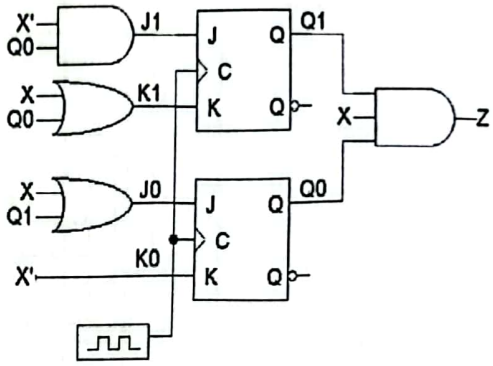
b) From the counter state diagram:
- MOD 4 because has 4 state (i.e. $2^2 = 4$), not a truncated counter.
- Count Up counter when Cnt = 1, and stay in the previous state when Cnt = 0.



CamScanner

# Method 2

- Analyze the following circuit that uses JK flip-flop with one input, *X* and one output, *Z*.




There is one input X, one output Z, and two flip-flops $Q_1Q_0$.

**Step 1:** The Boolean expression of each flip-flop and output Z

$J_1 = X' Q_0$ $J_0 = X + Q_1$ $Z = Q_1Q_0X$

$K_1 = X + Q_0$ $K_0 = X'$

**Step 2:** Produce a state table with JK FF transition using the Boolean expression.

| Input, X | Present State | | Next State | | JK FF Transition | | | | Output, Z |
|---|---|---|---|---|---|---|---|---|---|
| | $Q_1$ | $Q_0$ | $Q_{1+}$ | $Q_{0+}$ | $J_1$ | $K_1$ | $J_0$ | $K_0$ | |
| 0 | 0 | 0 | | | 0 | 0 | 0 | 1 | 0 |
| 0 | 0 | 1 | | | 1 | 1 | 0 | 1 | 0 |
| 0 | 1 | 0 | | | 0 | 0 | 1 | 1 | 0 |
| 0 | 1 | 1 | | | 1 | 1 | 1 | 1 | 0 |
| 1 | 0 | 0 | | | 0 | 1 | 1 | 0 | 0 |
| 1 | 0 | 1 | | | 0 | 1 | 1 | 0 | 0 |
| 1 | 1 | 0 | | | 0 | 1 | 1 | 0 | 0 |
| 1 | 1 | 1 | | | 0 | 1 | 1 | 0 | 1 |



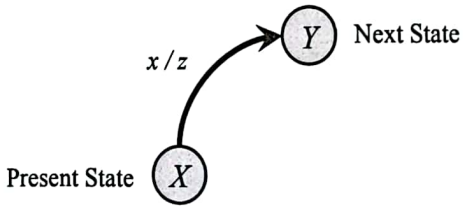
CamScanner

# Method 2 (continue)

**Step 3:** Complete the state table with next state values using JK excitation table, then draw the state diagram.

| Input, X | Present State | | Next State | | JK FF Transition | | | | Output, Z |
|---|---|---|---|---|---|---|---|---|---|
| | $Q_1$ | $Q_0$ | $Q_{1+}$ | $Q_{0+}$ | $J_1$ | $K_1$ | $J_0$ | $K_0$ | |
| 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 1 | 0 |
| 0 | 0 | 1 | 1 | 0 | 1 | 1 | 0 | 1 | 0 |
| 0 | 1 | 0 | 1 | 1 | 0 | 0 | 1 | 1 | 0 |
| 0 | 1 | 1 | 0 | 0 | 1 | 1 | 1 | 1 | 0 |
| 1 | 0 | 0 | 0 | 1 | 0 | 1 | 1 | 0 | 0 |
| 1 | 0 | 1 | 0 | 1 | 0 | 1 | 1 | 0 | 0 |
| 1 | 1 | 0 | 0 | 1 | 0 | 1 | 1 | 0 | 0 |
| 1 | 1 | 1 | 0 | 1 | 0 | 1 | 1 | 0 | 1 |

The drawing of a state diagram for a sequential circuit with an output must have




- A circle to represent a Present state – the state of the sequential circuit at that moment
- A circle to represent a Next state – the state that the sequential circuit going to be
- An arrow that show the changes from the Present state to Next state
  - ✓ for each arrow must write $x / z$, where
    - ❑ $x$ is the **input** that cause the transition from Present state to the Next state
    - ❑ $z$ is the **output** of the sequential circuit (not the state) when the transition from Present state to Next state occurs

**DIY:**

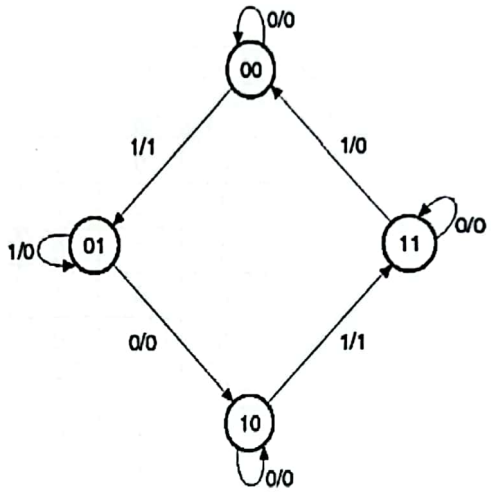
Draw the state diagram.



CamScanner

# Example: Design

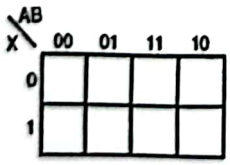
Design a clocked sequential circuit using D flip-flop that operates according to the state diagram given:



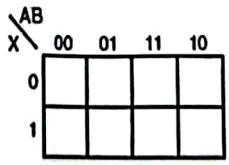

The transition table:

| Input | Present State | | Next State | | Output |
|---|---|---|---|---|---|
| X | A | B | A | B | Y |
| 0 | 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 1 | 1 | 0 | 0 |
| 0 | 1 | 0 | 1 | 0 | 0 |
| 0 | 1 | 1 | 1 | 1 | 0 |
| 1 | 0 | 0 | 0 | 1 | 1 |
| 1 | 0 | 1 | 0 | 1 | 0 |
| 1 | 1 | 0 | 1 | 1 | 1 |
| 1 | 1 | 1 | 0 | 0 | 0 |

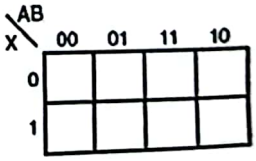
Since the design requires only D flip-flop, then $D_A = A_+$ and $D_B = B_+$.




$D_A =$




$D_B =$




$Y =$



CamScanner

Digital Logic

# MODULE 9
## SHIFT REGISTERS

**Objectives:**

1. To introduce the characteristics and function of a shift register
2. To highlight different types of shift register
3. To introduce shift register IC
4. To explain shift register counter



CamScanner

Digital Logic

# MODULE CONTENT

Basic Shift Register functions

Shift Register:

- Serial In/Serial Out (SISO)
- Serial In/Parallel Out (SIPO)
- Parallel In/Serial Out (PISO)
- Parallel In/Parallel Out (PIPO)

Bidirectional Shift Registers

Shift Register counters

- Ring Counter
- Johnson Counter

CamScanner

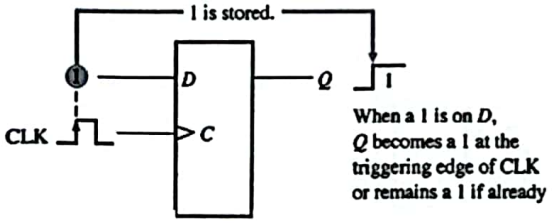
# Basic Shift Register (SR) Functions

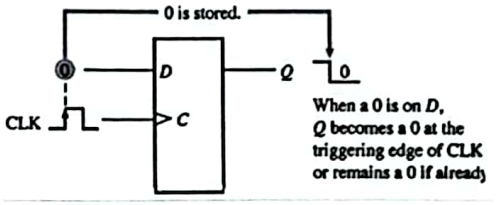
Digital Logic

- SR is a register in which binary data can be stored, and this data can be shifted to the left or right when a signal is applied.
- It consist of arrangements of FFs.
- Digital System : Important in application involving :
  - Data storage
  - Data transfer/movement

**Data Storage**

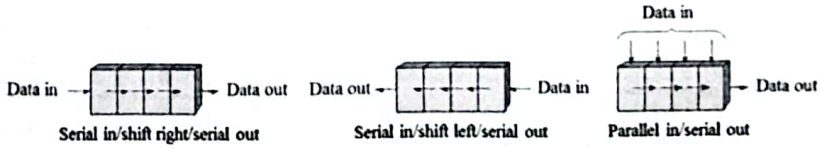
Flip-flop can be use as a storage element. Each flip-flop can store 1 bit data. If more bits required than more flip-flop is used.

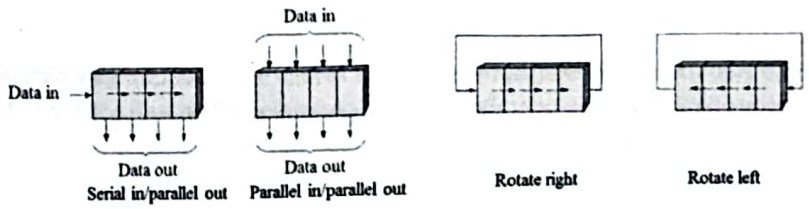



**Data transfer/movement**

A register that is capable of shifting data one bit at a time is called a shift register. The logical configuration of a serial shift register consists of a chain of flip-flops connected in cascade, with the output of one flip-flop being connected to the input of its neighbor. The operation of the shift register is synchronous; thus each flip-flop is connected to a common clock. Using D flip-flops forms the simplest type of shift-registers. Shifting an N bit shift register will lose all the data after 2N clock cycles. If want to maintain the data use rotate.



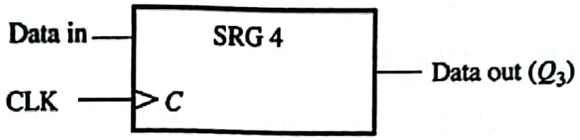






CamScanner

# Serial In/Serial Out (SISO) ..

- Accepts data serially, through *Data in* one bit at a time on a single line.
- Every clock pulse one bit of *Data in* will enter the shift register and at the same time one bit will be shifted to Q3
- Output produced in serial form.

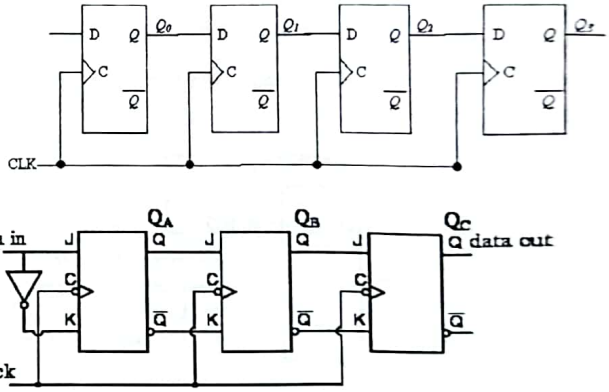



Logic symbol for an 4-bit serial in/serial out shift register

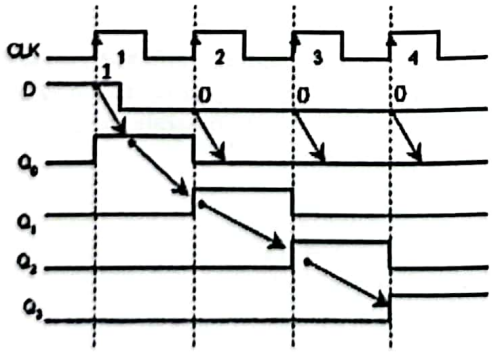
The circuit diagram shows a serial in-serial out shift register implemented using D flip-flops and JK flip-flop configured as D flip-flop.

Assuming that the register is initially clear (all 0's) the timing shown the waveform diagram when '1000' is shifted through the register.

Let us now take a closer look at the effect of shifting the binary digit '1' through the register over time. Again, we assume that the register is initially reset and the bit enters the left-most stage.




Serial-in, serial-out shift register using type "JK" storage elements




| Clock | FF0 | FF1 | FF2 | FF3 | Value |
|---|---|---|---|---|---|
| Initially | 0 | 0 | 0 | 0 | 0 |
| 1 | 1 | 0 | 0 | 0 | 8 |
| 2 | 0 | 1 | 0 | 0 | 4 |
| 3 | 0 | 0 | 1 | 0 | 2 |
| 4 | 0 | 0 | 0 | 1 | 1 |

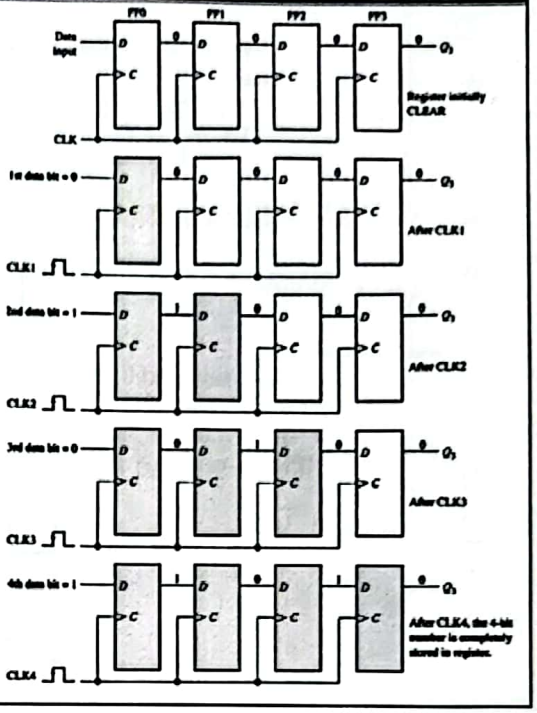
Note that at each time step from t=1 onwards, the right shift results in a state change which indicates a division by two operation. Similarly, if we operate in reverse chronological order (i.e. from t=4 to t=1), the left shift will result in a multiplication be two.



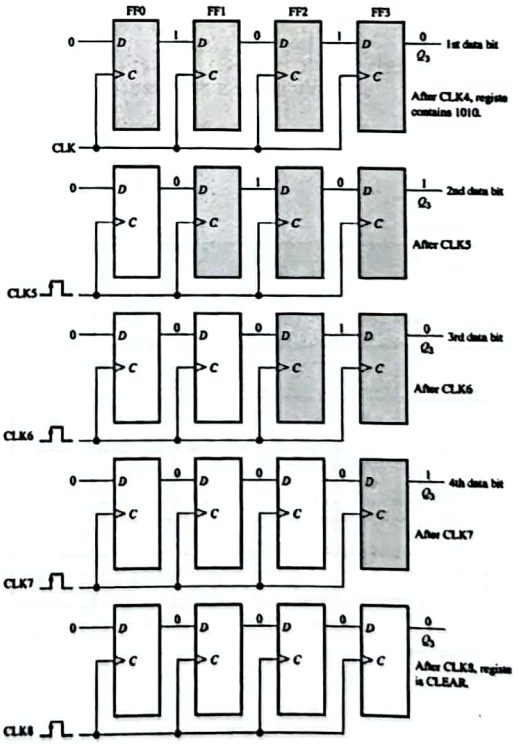
CamScanner

# .. Serial In/Serial Out (SISO)

- Four bits (1010) being entered serially into the register
- Every clock cycle a data is shifted to the right one position
- For n bit SISO it requires n clock cycle to completely entering the shift register
  - i.e. for the first data entered to appear at the output



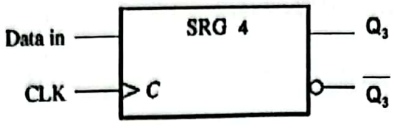

- Once all data is shifted in , every clock cycle will make the data shifted another one position to the right
- On the 5th clock cycle, the first 0 is shifted out and lost, its position replace by the 2nd data, at the left most flip-flop new data entering the SISO
- This kind of shifting repeat every clock cycle.
- Assuming every time a new data is 0, at the 8th clock cycle all flip-flop in SISO will be replaced by a new data
- Therefore for n bit SISO
  - It requires n clock cycle to shifted in all the data set (n bit)
  - It requires another n clock cycle to shifted out all the data set (n bit)






CamScanner

# Example : SISO



Data shifted in

| Clock, t | FF0 | FF1 | FF2 | FF3 |
|---|---|---|---|---|
| Initially | 0 | 0 | 0 | 0 |
| 1 | 1 | 0 | 0 | 0 |
| 2 | 1 | 1 | 0 | 0 |
| 3 | 0 | 1 | 1 | 0 |
| 4 | 1 | 0 | 1 | 1 |

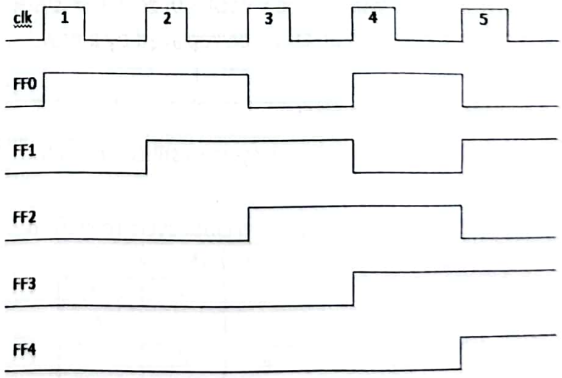
Data shifted out

| Clock, t | FF0 | FF1 | FF2 | FF3 | data lost |
|---|---|---|---|---|---|
| Initially | 1 | 0 | 1 | 1 | X |
| 1 | 0 | 1 | 0 | 1 | 1 |
| 2 | 0 | 0 | 1 | 0 | 1 |
| 3 | 0 | 0 | 0 | 1 | 0 |
| 4 | 0 | 0 | 0 | 0 | 1 |

**Example**: 5-bit Serial In Serial Out (Right Shift Register)

- Data input: 11010
- Initially all FF s are cleared.
- MSB are shifted in first.

| Clock | FF0 | FF1 | FF2 | FF3 | FF4 |
|---|---|---|---|---|---|
| Initially | 0 | 0 | 0 | 0 | 0 |
| 1 | 1 | 0 | 0 | 0 | 0 |
| 2 | 1 | 1 | 0 | 0 | 0 |
| 3 | 0 | 1 | 1 | 0 | 0 |
| 4 | 1 | 0 | 1 | 1 | 0 |
| 5 | 0 | 1 | 0 | 1 | 1 |



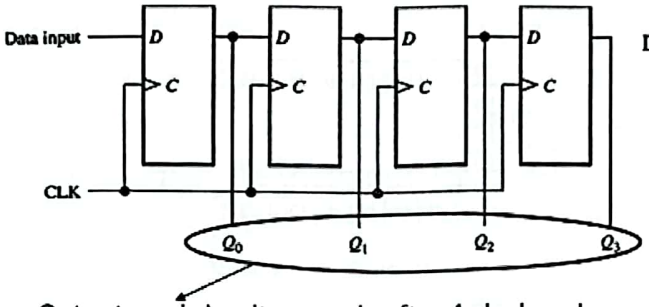



CamScanner

# Serial In Parallel Out (SIPO)

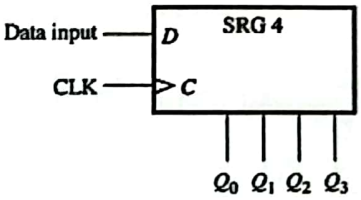
- Data bits are entered serially one bit at a time
- Output is parallel, all bits are available simultaneously but only after N clock cycles for an N bit SIPO
- For SISO , output only at $Q_3$, for SIPO output $Q_3Q_2Q_1Q_0$

Circuit for 4 bit SIPO




Output read simultaneously after 4 clock cycle

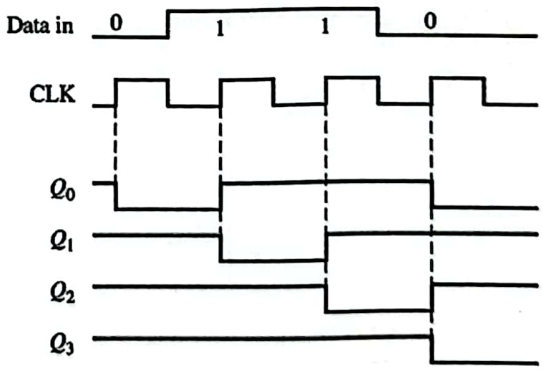
Symbol for 4 bit SIPO




- For a 4 bit SIPO, it can store 4 bit data and shift the data at every clock cycle
- Data entered serially, only after 4 clock cycle the data entered can be read at the output
- Usually MSB data enter the SIPO first (actually which data entered first will depend on the design), after 4 clock cycle it will appear at $Q_3$

| Clock | FF0 | FF1 | FF2 | FF3 |
|---|---|---|---|---|
| Initially | 0 | 0 | 0 | 0 |
| 1 | 1 | 0 | 0 | 0 |
| 2 | 1 | 1 | 0 | 0 |
| 3 | 0 | 1 | 1 | 0 |
| 4 | 1 | 0 | 1 | 1 |

$Q_0$ $Q_1$ $Q_2$ $Q_3$



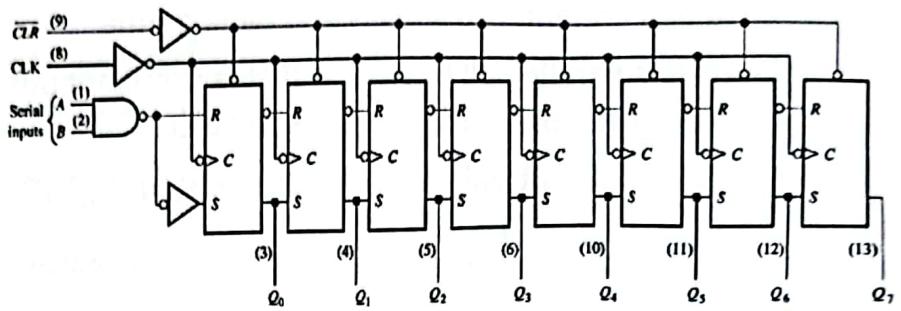

**DIY:**

- Determine what data is entered to SIPO register?
- Circle on the timing diagram the valid output of the SIPO register?

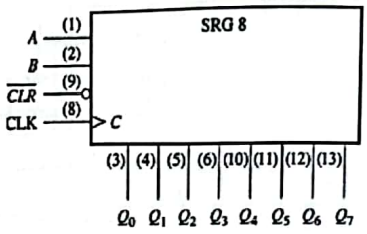


CamScanner

# IC 74HC164: 8-Bit Shift Register




(a) Logic diagram




(b) Logic symbol

This is an example of an IC for 8-bit SIPO.

- Serial data can be input at pin A or B.
- CLR' is an active LOW input. If the logic level is 0, it will clear all output to 0000 0000.
- Data entered at pin A or B, will appear at $Q_0$ after the first clock pulse. The next clock pulse will shift the data to the right.
- The data will appear at $Q_7$ after the 8-th clock pulse.
- A valid parallel data only appear at all output after 8 clock cycle.

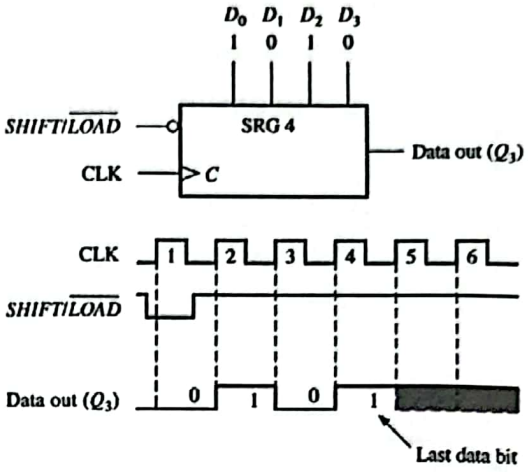


CamScanner

# Parallel In Serial Out (PISO)

Digital Logic

- Inputs entered simultaneously into respective stages on parallel lines
- Outputs are one bit at a time
- *SHIFT/LOAD*
  - = 0 → Loads the input values
  - =1 → shifts them out at clock pulse

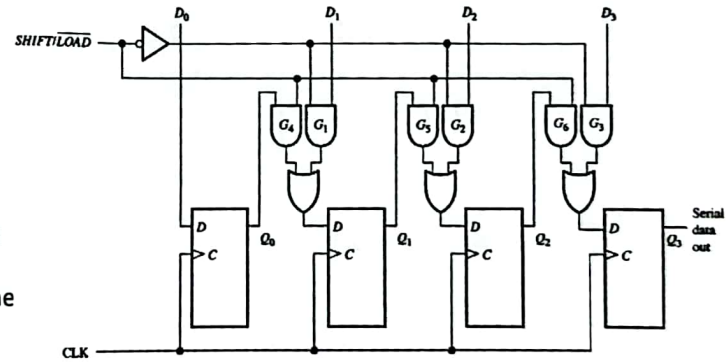



Operation of PISO involves the following 2 steps:

**1. LOAD MODE:**

a) Input data must be available at parallel input pins: $D_3$ $D_2$ $D_1$ $D_0$

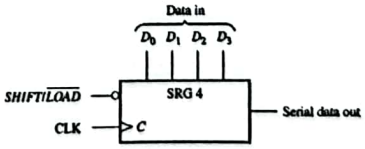
b) Load the initial parallel value by making $SHIFT/\overline{LOAD} = 0$. This will make the data loaded to the 4 flip-flop inside the shift register at the positive edge of the clock. The MSB of the data will appear at Data out.




(a) Logic diagram

**2. SHIFT MODE:**

If $SHIFT/\overline{LOAD} = 1$, data will be shifted on every positive edge of the clock cycle. After 4 clock cycle the LSB data will appear at the Data out. At the 5th clock cycle all the 4 bit data set will be lost and replaced by a new set of 4 bit data.




(b) Logic symbol



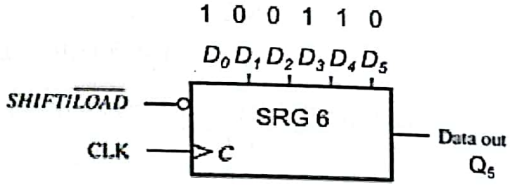
CamScanner

# Example: PISO

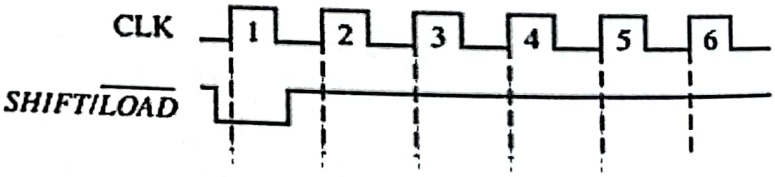


Complete the timing diagram of 6–bit PISO shift register (SRG 6) for the data input 011001. The register initially all is 0s and assume MSB shifted out first.

a) Draw the shift register logic symbol with the inputs.




b) Complete the timing diagram.




Data out ($Q_5$)

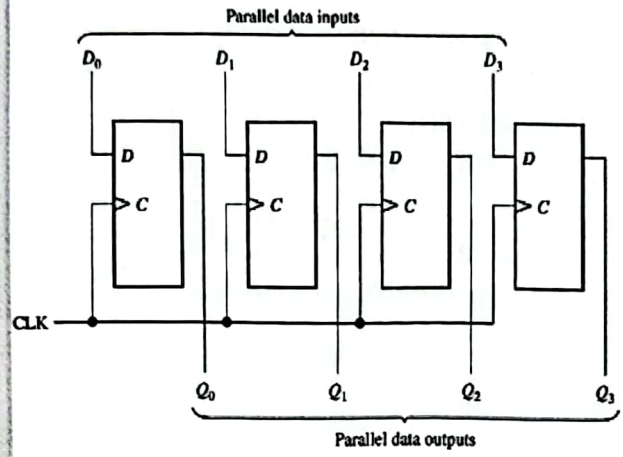


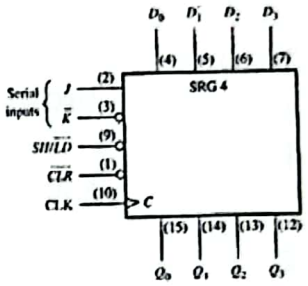
CamScanner

# Parallel In Parallel Out (PIPO)



- Input and output done in parallel.
- Enter all inputs - Bits appear on the parallel outputs.

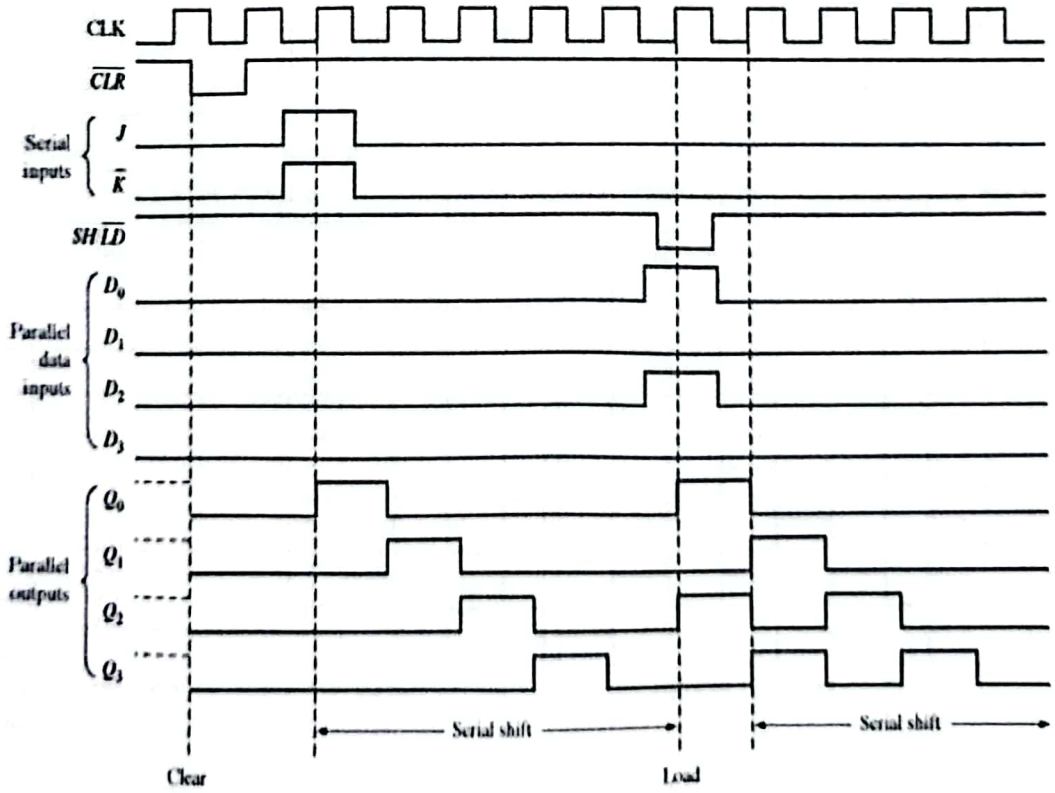



The 74LS195A 4-bit parallel access shift register

- Data input parallel, this data doesn't appear at the output
- The data only appear at the output at the positive edge of the clock.

- The 74HC195 can be used for parallel in/parallel out operation, serial in/serial out and serial in/parallel out operations.
- Q3 is the output when it is used for parallel in/serial out operation.

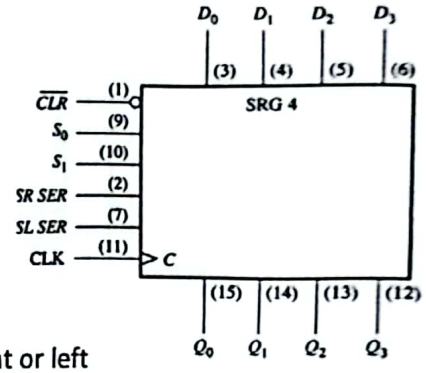





CamScanner

# Universal Bidirectional Shift Register

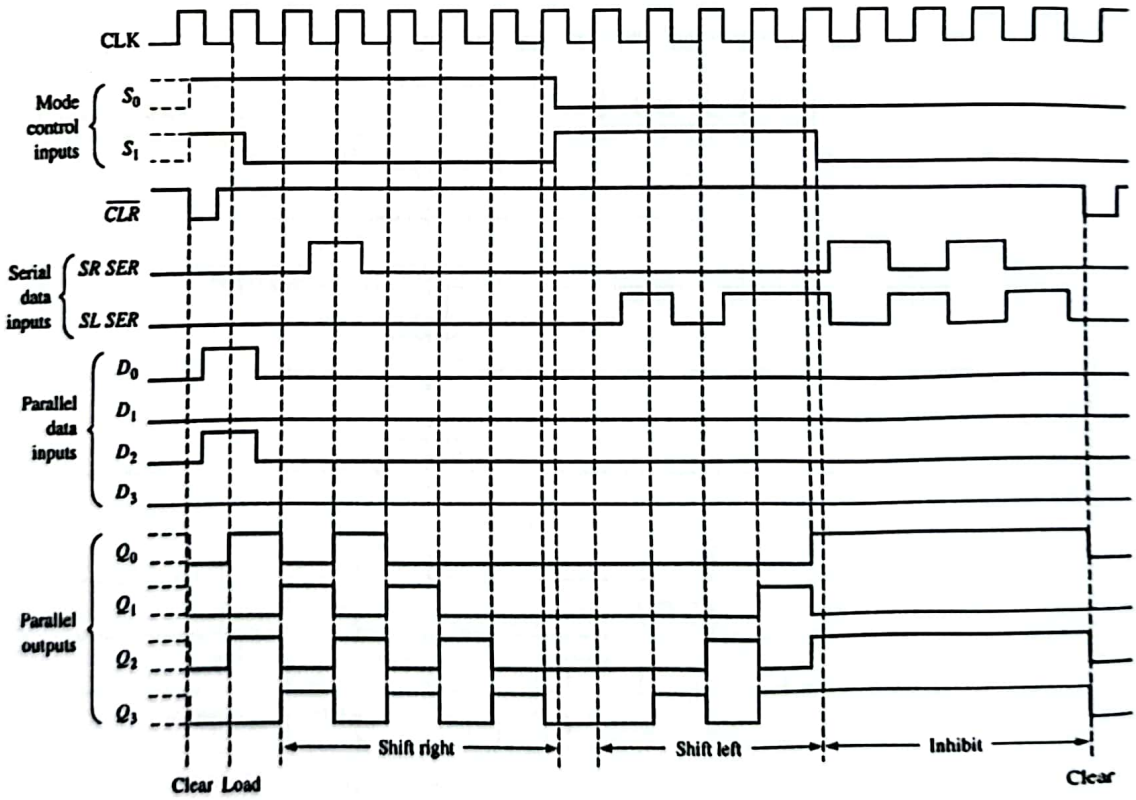
- 74HC194 is a universal bidirectional shift register
  - It has both serial and parallel input and output capability
    - Can implement SISO,SIPO,PISO,PIPO
    - SISO- in: SR SER , out: $Q_3$ for right shift
      - in: SL SER , out: $Q_3$ for left shift
    - SIPO – in: either SL SER or SR SER
      out: $Q_3Q_2Q_1Q_0$
    - PISO – in: $D_3D_2D_1D_0$,
      out : either SL SER or SR SER
    - PIPO – in: $D_3D_2D_1D_0$ , out: $Q_3Q_2Q_1Q_0$
  - Bidirectional - data can be shifted to right or left




Mode of Operation controlled by $S_1S_0$

- $S_1S_0 = 00$ - Clock Inhibit , clock has no efect to the output, output don't change
- $S_1S_0 = 01$ - Shift Right, serial data entered from shift right serial input (SR SER)
- $S_1S_0 = 10$ - Shift Left, serial data entered from shift left serial input (SL SER)
- $S_1S_0 = 11$ – Parallel Load, load the parallel data from $D_3D_2D_1D_0$

The timing diagram of a universal bidirectional shift register






CamScanner

# Shift Register Counters

Digital Logic

- A shift register with serial output connected back to the serial input to produce special sequences.
- 2 most common types:
  - Ring counter
  - Johnson counter

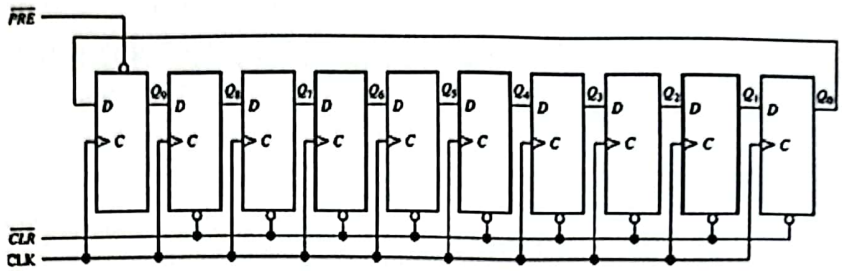
- In some cases, we want a counter that provides individual digit outputs rather than a binary or BCD output.
  - This is usually a case if we want to use a counter to control something rather than counting something
- We can do this by adding a decoder circuit to the binary counter, so that we can detect the state of interest and use it to control something
- However, in many cases it is much simpler to use a different counter structure, that will permit much simpler decoding of individual digit outputs.
  - This is the main reason why we have Ring and Johnson counter.



CamScanner

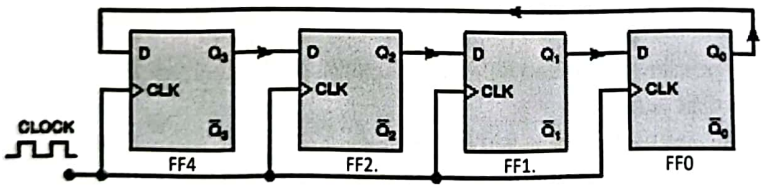
# Ring Counter

- Ring Counter is a Serial-In Serial-Out (SISO) shift register but with the final output fed back to the first input



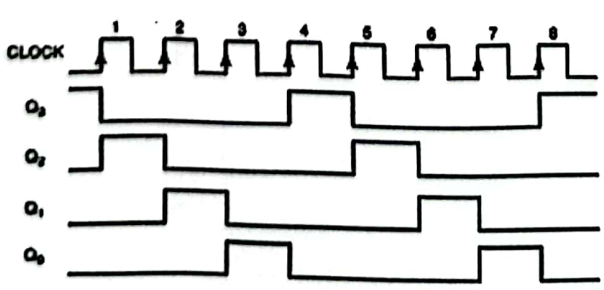

- The first FF ($Q_9$) is preset to '1' by making PRE' = 0 momentarily, all other flip-flop are cleared (0) by making CLR'= 0 --representing state 0
- The '1' is shifted round the ring, so next clock cycle $Q_8 = 1$, while all other flip-flop output is '0' --represents a state one

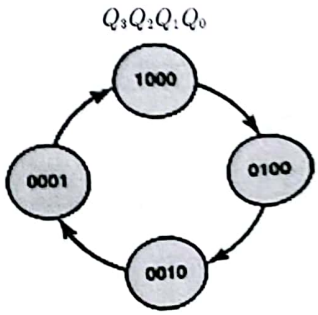
- For n bit ring counter, the number of states it has is n (MOD n)
- For a 4-bit ring counter, there are 4 flip-flops which make it a MOD 4 counter
  - Initially, PRE' = 0 to set $Q_3 = 1$
  - It will recycle after 4 clock cycle




| $Q_3$ | $Q_2$ | $Q_1$ | $Q_0$ | CLOCK pulse |
|---|---|---|---|---|
| 1 | 0 | 0 | 0 | 0 |
| 0 | 1 | 0 | 0 | 1 |
| 0 | 0 | 1 | 0 | 2 |
| 0 | 0 | 0 | 1 | 3 |
| 1 | 0 | 0 | 0 | 4 |
| 0 | 1 | 0 | 0 | 5 |
| 0 | 0 | 1 | 0 | 6 |
| 0 | 0 | 0 | 1 | 7 |
| . | . | . | . | . |
| . | . | . | . | . |

- If only one flip-flop is HIGH while all other LOW, every state will have a unique '1' based on the position of '1'
  - This mean that the '1' can be use to decode any state without the need to use a decoder
  - If we want to decode state 0, get the input from the output of $Q_3$, whenever $Q_3 = 1$ means the state is state 0
  - To decode state 2, get the input from the output $Q_1$






CamScanner

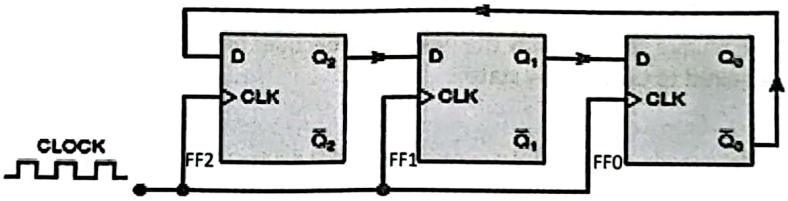
# Johnson Counter

Digital Logic

- A Johnson counter also called switchtail ring counter, twisted-ring counter, walking-ring counter, or Moebius counter is a modified ring counter
- In the Johnson counter the last complemented output is fed back in as an input to the first FF
- Can be implemented with any types of FF, but configured as D flip-flop.
- Number of unique states are 2 times the number of bits (FF)
  - 4 bits ➔ 4*2 = 8 states
  - 5 bits ➔ 5*2 = 10 states
- Johnson counter will produce a modulus of 2n; (n = number of stages)

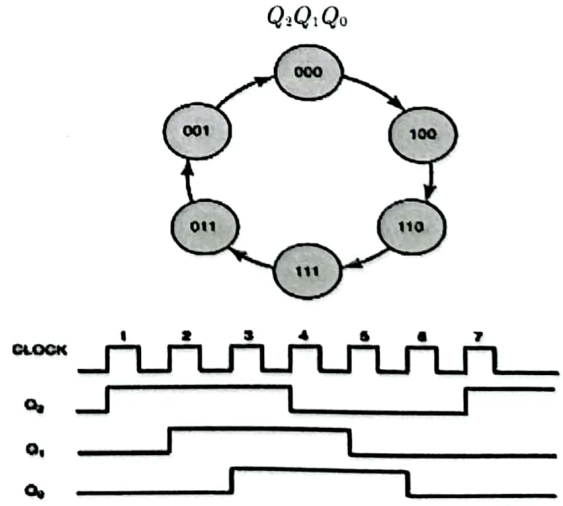
**3 bit Johnson counter**

- Circuit: requires 3 D flip-flop, where $Q_2'$ is connected back to the input of the first flip-flop




- The timing diagram and a table showing the operation of Johnson Counter
  - After the 6th clock cycle, the pattern repeat again, therefore 6 unique states (MOD 6)
  - The counting sequence is not binary.

| Clock, t | FF2 | FF1 | FF0 | $\bar{Q}_0$ |
|---|---|---|---|---|
| Initially | 0 | 0 | 0 | 1 |
| 1 | 1 | 0 | 0 | 1 |
| 2 | 1 | 1 | 0 | 1 |
| 3 | 1 | 1 | 1 | 0 |
| 4 | 0 | 1 | 1 | 0 |
| 5 | 0 | 0 | 1 | 0 |
| 6 | 0 | 0 | 0 | 1 |
| 7 | 1 | 0 | 0 | 1 |
| 8 | 1 | 1 | 0 | 1 |






CamScanner

# Johnson Counter characteristic



- For n bit flip-flop the MOD is 2n
- For a Johnson counter there always exist 2 bit that are unique compared to the other state
  - This mean that it only requires 2 input decoder to decode a state

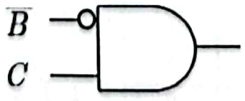
| Sequence number | Flip-flop outputs A | B | C | E | AND gate required for output |
|---|---|---|---|---|---|
| 1 | 0 | 0 | 0 | 0 | $A'E'$ |
| 2 | 1 | 0 | 0 | 0 | $AB'$ |
| 3 | 1 | 1 | 0 | 0 | $BC'$ |
| 4 | 1 | 1 | 1 | 0 | $CE'$ |
| 5 | 1 | 1 | 1 | 1 | $AE$ |
| 6 | 0 | 1 | 1 | 1 | $A'B$ |
| 7 | 0 | 0 | 1 | 1 | $B'C$ |
| 8 | 0 | 0 | 0 | 1 | $C'E$ |

Count sequence and required decoding

- The a above table is for 4 bit Johnson Counter:
  - For a 4 bit counter there is 8 states
  - Each state have a unique 2 input variable as shown on the rightmost column
  - These unique state simplify the design of a decoder because a decoder with only 2 input is required to decode any state

- For example, to decode state 7 to drive an active HIGH device, we can use the following decoder

| Sequence number | Flip-flop outputs A | B | C | E | AND gate required for output |
|---|---|---|---|---|---|
| 7 | 0 | 0 | 1 | 1 | $B'C$ |






CamScanner

# Comparison of a counter

- For a given n flip-flop, the binary counter can produce the most state and ring is the worst
- In term of decoding , Ring counter is the best because there is no need for a decoder to decode each state, binary counter is the worst because require a more complex decoder
- Johnson always in the middle when comparing those feature

| | MOD for n flip-flop | Decoder Input |
|---|---|---|
| Ring | n | no need for a decoder |
| Johnson | 2n | 2 input decoder |
| Binary | $2^n$ | usually > 2 input decoder |



CamScanner